



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Entendiendo las Redes Convolucionales

Realizado por
Alberto Perea León

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
María José Jiménez Rodríguez

En el departamento de
Matemática Aplicada I

Convocatoria de mayo, curso 2025/26

Dedico este trabajo a mis padres, quienes siempre se han esforzado por hacerme una persona de provecho. A los docentes que luchan por inculcarnos y motivarnos a descubrir. Y a la curiosidad, fuente de energía que alimenta a las mentes investigadoras.

Agradecimientos

Me gustaría expresar mis más sinceros agradecimientos a mi madre, Victoria; mi padre, Victor; y mi hermano, Miguel. Ellos, junto con un pequeño conjunto de amigos han formado el núcleo de confianza y apoyo fundamental para mi desarrollo académico, profesional y personal a lo largo de parte de mi vida. Algunos de estos amigos son Jaime Carbó, Carmen Ruíz y Gabriela Vega. Por último, me gustaría hacer mención especial a mi tutora María José Jiménez Rodríguez, quien me ha guiado desde el inicio y ha sabido motivarme y reconducirme en los diferentes aspectos del proyecto.

En este documento, quedan reflejadas las horas de esfuerzo y dedicación de estos últimos años de formación. Sin embargo, los malos ratos, de agobio y estrés; o los buenos, de celebración y de risas; esos momentos no se pueden plasmar en un papel, y no por ello son menos importantes. Que quede constancia, siempre los llevaré conmigo, al igual que a vosotros.

Resumen

Las redes neuronales convolucionales se han consolidado como la arquitectura de referencia en el campo de la visión por computador, pero, a día de hoy resulta bastante complejo desentrañar su funcionamiento interno y entender qué y cómo aprenden realmente. Esto se debe en gran medida a que el conocimiento del modelo se distribuye en millones de parámetros, lo que dificulta comprender qué información almacenan sus capas internas y por qué ciertas arquitecturas funcionan mejor que otras.

Este trabajo aborda esa pregunta de forma progresiva. Partiendo de las bases teóricas de las redes neuronales artificiales, se construye una red neuronal densa capaz de abordar problemas de clasificación simples. Sin embargo, esta arquitectura presenta ciertas carencias al trabajar con información espacial, ya que trata cada píxel de forma independiente, ignorando las relaciones de proximidad entre ellos. Motivado por esta limitación, se introduce la operación de convolución y cómo esta da lugar a una arquitectura más eficiente para trabajar con imágenes.

El estudio se realiza sobre el conjunto de datos MNIST (Modified National Institute of Standards and Technology), imágenes de dígitos escritos a mano, comparando un perceptrón multicapa y una red convolucional en tres aspectos: su rendimiento en clasificación, su robustez ante ruido gaussiano y el comportamiento de dos herramientas de interpretabilidad, los mapas de características, representaciones internas del modelo que revelan qué detecta cada filtro convolucional, y los mapas de saliencia, visualizaciones que señalan qué píxeles de la imagen de entrada influyen más en la decisión del modelo.

La experimentación permite concluir que la arquitectura convolucional es más eficiente y precisa que la red densa, logrando mayor exactitud con la mitad de parámetros, y además significativamente más robusta ante el ruido. Esta resistencia reside en las representaciones y abstracciones que aprende progresivamente en sus capas internas gracias a la convolución. Los mapas de saliencia, aunque útiles para identificar qué regiones activan a ambos modelos, no capturan este mecanismo y resultan insuficientes para explicar el porqué de la robustez ante perturbaciones. Para eso es necesario observar directamente las activaciones internas, no los gradientes sobre la imagen de entrada.

Palabras clave: redes neuronales convolucionales, aprendizaje profundo, interpretabilidad, mapas de saliencia, robustez al ruido, MNIST, mapas de características.

Índice general

1	Introducción	1
1.1.	Contexto	1
1.2.	Motivaciones	2
1.3.	Objetivos	2
1.4.	Estructura del proyecto	3
2	Planificación	4
2.1.	Estimación inicial y dedicación real	4
2.2.	Análisis	4
3	Redes Neuronales Artificiales	5
3.1.	Arquitectura de una Red Neuronal	6
3.1.1.	Las capas	7
3.1.2.	La neurona	7
3.2.	Entrenamiento	12
3.2.1.	Retropropagación	14
3.2.2.	Generalización y Regularización	20
3.3.	Limitaciones de las Redes Neuronales Tradicionales	23
4	Redes Neuronales Convolucionales	25
4.1.	Arquitectura básica	27
4.2.	Funcionamiento interno de las redes convolucionales	32
4.2.1.	Filtrado, stride y padding	32
4.3.	Aprendizaje y optimización de filtros	34
4.4.	Representaciones internas y activaciones	35
4.4.1.	Visualización de representaciones internas	35
4.4.2.	Herramientas de visualización	36
5	Experimentación	42
5.1.	Entorno experimental y herramientas de desarrollo	42
5.2.	Conjunto de datos: MNIST	42
5.3.	Métricas de Evaluación	44
5.4.	Implementación de los modelos	46
5.4.1.	Red Neuronal Densa (FCNet)	46
5.4.2.	Red Neuronal Convolucional (ConvNet)	46
5.5.	Comparativa FCNet vs ConvNet	47
5.6.	Análisis de representaciones internas	52
5.6.1.	Mapas de características	52
5.6.2.	Visualización de activaciones por capa	53
5.6.3.	Mapas de saliencia	54
5.7.	Análisis bajo ruido gaussiano	55

5.7.1.	Resultados de exactitud bajo ruido	56
5.7.2.	Robustez de la CNN frente al ruido	57
5.7.3.	Fragilidad de los mapas de saliencia bajo ruido	59
5.7.4.	Persistencia de las representaciones internas bajo ruido	60
6	Conclusiones y Líneas Futuras	62
6.1.	Eficiencia y precisión de las representaciones convolucionales	62
6.2.	Robustez arquitectónica frente a perturbaciones gaussianas	62
6.3.	Representaciones internas como indicador de robustez	63
6.4.	Límites de los mapas de saliencia como herramienta de diagnóstico .	64
6.5.	Limitaciones	64
6.6.	Líneas Futuras	65
A	Implementación de los Modelos	67
B	Desarrollo Matemático de la Retropropagación	69
	Lista de Acrónimos	74
	Bibliografía	74

Índice de figuras

3.1. Red neuronal totalmente conectada (3–4–2).	7
3.2. Diagrama funcionamiento neurona simple.	8
3.3. Comparación de las diferentes funciones de activación.	11
3.4. Preparación del conjunto de datos MNIST en los subconjuntos de entrenamiento, validación y pruebas.	13
3.5. Esquema de <i>K-fold Cross Validation</i> con $K = 5$: en cada iteración, un fold distinto (naranja) se usa para validación y los restantes (verde) para entrenamiento.	13
3.6. Descenso del gradiente para el error cuadrático. Cada paso acerca x al mínimo global ($x = 0$).	15
3.7. Descenso del gradiente en 3D del error cuadrático. Cada paso acerca (x, y) al mínimo global $f(x, y) = 0$	17
3.8. Diagrama de la función $f(x, y, z) = (x + y)z$, para $(x, y, z) = (2, 3, 4)$	19
3.9. Diagrama de regla de la cadena de la función $f(x, y, z) = (x + y)z$, para $(x, y, z) = (2, 3, 1)$	19
3.10. Repercusión del sobreajuste en el entrenamiento de un modelo.	21
3.11. Efecto de la regularización en un caso de sobreajuste.	22
3.12. Ilustración del dropout: algunas neuronas (en gris con aspa) se desactivan de forma aleatoria durante el entrenamiento, forzando a la red a no depender de unidades específicas.	22
3.13. Aplanado de una imagen para servir de entrada a una red neuronal.	24
3.14. Aplanado de un dígito de MNIST.	24
4.1. Escalado de los parámetros de un modelo tradicional.	25
4.2. Arquitectura LeNet-5 [5].	26
4.3. Arquitectura AlexNet [6].	27
4.4. Convolución discreta entre $f = [0, 0, 1, 1, 0, 0]$ y filtro de detección de bordes $g = [-1, 0, 1]$	28
4.5. Convolución con filtro de Sobel en ejes x, y para detectar bordes.	29
4.6. Salida de una capa convolucional con 32 filtros de 3×3	29
4.7. Comparación entre una red lineal y una con activación ReLU. La introducción de no linealidad permite modelar funciones más complejas.	30
4.8. Operación de Max Pooling con una ventana de 2×2	31
4.9. Ejemplo de average pooling con $\text{stride} = 1$ y $\text{stride} = 2$	33
4.10. Ejemplo de average pooling con padding = 1.	34
4.11. Mapa de características obtenido al pasar un kernel de una capa convolucional por una imagen de entrada.	37
4.12. Pase hacia adelante en una red convolucional.	38
4.13. Características extraídas del número 5 en las capas <i>Conv1</i> (dos primeras filas) y <i>Conv2</i> (dos últimas filas).	39

4.14. Ejemplo de mapa de saliencia generado a partir del gradiente de la salida con respecto a la imagen de entrada.	40
5.1. Captura de ejemplos aleatorios del conjunto de entrenamiento de MNIST.	43
5.2. Distribución de muestras de MNIST en los conjuntos de entrenamiento, validación y pruebas	44
5.3. Curvas de aprendizaje comparativas de FCNet y ConvNet. Evolución de la pérdida (izquierda) y la precisión (derecha) en entrenamiento y validación a lo largo de las épocas.	49
5.4. Matriz de confusión de ConvNet sobre el conjunto de pruebas (izquierda) y precisión por clase (derecha). El dígito 9 ostenta la peor precisión de predicción.	50
5.5. Matriz de confusión de ConvNet sobre el conjunto de pruebas (izquierda) y precisión por clase (derecha). Los dígitos 6, 7 y 9 presentan la mayor tasa de error, por debajo del 99 %.	50
5.6. Comparación de los pares más confundidos en la CNN.	51
5.7. Mapas de activación para los 8 filtros más activos de <i>Conv1</i> y <i>Conv2</i> sobre el número nueve.	53
5.8. Mapa de saliencia del dígito uno sobre un MLP.	54
5.9. Mapa de saliencia promediado de cada uno de los dígitos de MNIST sobre un MLP.	54
5.10. Mapa de saliencia del dígito uno sobre una CNN.	55
5.11. Mapa de saliencia promediado de cada uno de los dígitos de MNIST sobre un MLP.	55
5.12. Efecto del ruido gaussiano sobre un dígito del conjunto de pruebas para valores de σ crecientes, desde la imagen original ($\sigma = 0,0$) hasta una perturbación severa ($\sigma = 0,3$).	56
5.13. Exactitud de FCNet y ConvNet en función del nivel de ruido gaussiano σ . La diferencia entre ambos modelos supera los 22 puntos porcentuales a $\sigma = 0.30$	57
5.14. Ilustración de los mecanismos de robustez de la CNN.	58
5.15. Saliency maps de FCNet (fila central) y ConvNet (fila inferior) sobre el mismo dígito a distintos niveles de ruido σ	59
5.16. Estabilidad del saliency map ante ruido gaussiano: correlación de Spearman (izquierda) y distancia L2 normalizada (derecha), promediadas sobre 20 repeticiones por nivel de σ	60
5.17. Mapas de activación de los 8 filtros más activos de <i>Conv1</i> para la imagen limpia (fila superior) y con ruido $\sigma = 0.2$ (fila inferior). Los patrones estructurales detectados se mantienen pese a las perturbaciones en los píxeles de entrada.	61
5.18. Similitud entre los mapas de activación de la imagen limpia y la ruidosa en <i>Conv1</i> (características locales) y <i>Conv2</i> (características abstractas), promediada sobre 20 repeticiones. Ambas capas mantienen valores superiores a 0.8 hasta $\sigma = 0.25$	61

1. Introducción

Para empezar, debemos contextualizar desde qué punto parte este proyecto y por qué es relevante hoy en día, así como las motivaciones e inquietudes por las que nace. Asimismo, se describirá la estructura del proyecto de forma clara y se explicará cómo se abordará el estudio.

1.1. Contexto

En las últimas décadas, la inteligencia artificial (IA) se ha convertido en una tecnología ampliamente conocida y utilizada en áreas como la investigación y el desarrollo. Este auge está estrechamente ligado al avance de técnicas de aprendizaje automático (machine learning) que han dotado a las redes neuronales artificiales con la capacidad para aprender representaciones y relaciones complejas a partir de datos [1, 2].

Inicialmente, las redes neuronales estaban inspiradas en el funcionamiento del cerebro humano y buscaban abstraer los principios del procesamiento de la información de nuestra mente, centrándose más en la capacidad computacional [7, 8]. Este enfoque ha demostrado ser capaz de abordar con éxito problemas que resultarían difíciles por medio de algoritmos clásicos, como el reconocimiento de patrones, la clasificación, la predicción o la aproximación de funciones [9]. No obstante, la eficacia de una red neuronal no depende exclusivamente de la disponibilidad de datos o de la capacidad de cómputo, sino de la arquitectura empleada y su capacidad de adaptación al problema planteado [10].

Particularmente, las redes neuronales convolucionales (CNN, Convolutional Neural Networks) han surgido como una arquitectura altamente eficaz frente a datos con estructura espacial, como las imágenes. Su diseño pretende aprender representaciones jerárquicamente: las primeras capas capturan características locales simples, mientras que las profundas combinan esas características para formar conceptos más abstractos [2, 10]. Convirtiéndose las CNN en el estándar por antonomasia en el campo de la visión por computador.

Sin embargo, a pesar de su elevado rendimiento, estos modelos suelen ser percibidos como cajas negras, ya que el conocimiento que adquieren durante el entrenamiento queda distribuido en millones de parámetros difíciles de interpretar de forma directa con el resultado. La falta de transparencia plantea diversas cuestiones acerca de qué tipo de información se almacena en sus capas internas y cómo se construyen las relaciones que permiten tomar decisiones al modelo [1].

1.2. Motivaciones

A pesar de que el desarrollo y la evaluación de modelos se ha centrado en métricas de rendimiento como la precisión o la función de coste, estos indicadores no ofrecen información suficiente acerca del comportamiento interno, por lo que acabamos tratando a estos modelos como cajas negras que reciben información y producen un resultado, como por arte de magia. Dos modelos con resultados similares pueden aprender representaciones internas muy distintas, lo que dificulta la comprensión de sus errores, sesgos y limitaciones [2].

Por tanto, desde un punto de vista científico y práctico, resulta muy conveniente entender el aprendizaje interno de los modelos CNN. Comprender estas representaciones nos permitirá no solo mejorar la interpretación y la confianza en los modelos, sino también optimizar las arquitecturas ya existentes, diagnosticar fallos en el entrenamiento y producir sistemas más robustos [10, 9].

Por tanto, este trabajo surge bajo la necesidad de analizar e interpretar el funcionamiento interno de las redes neuronales convolucionales prestando especial atención a las representaciones aprendidas en sus capas intermedias y finales, evaluando cómo estas contribuyen al proceso de toma de decisiones del modelo.

1.3. Objetivos

El objetivo principal de este trabajo es comprender el funcionamiento interno de las redes neuronales convolucionales. Partiendo de las bases teóricas de las redes tradicionales se realizará un análisis de su comportamiento bajo condiciones adversas. Para ello, se ha desarrollado una base de código experimental disponible en <https://github.com/perealberto/understanding-cnn>, que ha servido como herramienta de exploración y verificación a lo largo del trabajo. Como resultado, se han producido dos notebooks que condensan lo aprendido y que pueden servir como recurso didáctico:

1. **Notebook `entendiendo_redes_convolucionales.py`**: recorre de forma guiada el camino desde la neurona artificial hasta la arquitectura convolucional. Incluye implementación práctica con PyTorch, visualización de los filtros aprendidos y una comparativa experimental entre una red densa y una CNN sobre el conjunto de datos MNIST.
2. **Notebook `robustez_ruido.py`**: analiza el comportamiento de ambos modelos ante imágenes degradadas por ruido gaussiano, estudia la fragilidad de los mapas de saliencia y visualiza la persistencia de las representaciones internas de la CNN para explicar el mecanismo de robustez.

Ambos notebooks se apoyan en un módulo auxiliar `src/` que encapsula las definiciones de los modelos (`models.py`) y las funciones de cálculo de gradientes y visualización (`grad.py`), de forma que el código permanece limpio y centrado en el razonamiento.

1.4. Estructura del proyecto

Con el objetivo de situar al lector dentro del marco de este estudio, en este apartado se presenta una breve guía de las diferentes secciones que componen el proyecto, así como el propósito de cada una de ellas.

Para comenzar, se realiza un recorrido teórico por las bases de las redes neuronales artificiales y del aprendizaje profundo, estableciendo el marco conceptual necesario para comprender el resto del trabajo.

1. **Introducción teórica a las redes neuronales artificiales.** En este bloque se presenta una visión general de las redes neuronales tradicionales. Se describen su arquitectura básica, el modelo de neurona artificial, la organización en capas y los mecanismos de entrenamiento mediante retropropagación del error. Asimismo, se abordan conceptos clave como la generalización, la regularización y las principales limitaciones de estas arquitecturas cuando se aplican a datos de alta dimensionalidad, como las imágenes.
2. **Introducción teórica a las redes neuronales convolucionales.** A partir de las limitaciones expuestas en el apartado anterior, se introducen las redes neuronales convolucionales como una evolución natural de las redes neuronales tradicionales para el procesamiento de datos con estructura espacial. En esta sección se describe en detalle su arquitectura, el funcionamiento interno de las capas convolucionales y el proceso de aprendizaje y optimización de los filtros.
3. **Representaciones internas y herramientas de visualización dentro del capítulo de CNN.** Como subsección integrada en el capítulo de redes neuronales convolucionales, se estudian las representaciones internas aprendidas por el modelo y se introducen técnicas de análisis como los mapas de características, la visualización de activaciones por capa y los mapas de saliencia, con el objetivo de entender qué información utiliza la red para tomar decisiones.

Una vez establecido el marco teórico y las herramientas de análisis, el proyecto avanza hacia una fase experimental. En esta parte se describe el conjunto de datos empleado, las métricas de evaluación utilizadas y la implementación práctica tanto de una red neuronal tradicional como de una red neuronal convolucional aplicadas a la tarea de clasificación de dígitos manuscritos. Finalmente, se analizan y discuten los resultados obtenidos, extrayendo conclusiones sobre el aprendizaje interno de los modelos estudiados y proponiendo posibles líneas de trabajo futuro.

2. Planificación

2.1. Estimación inicial y dedicación real

Categoría	Estimado (h)	Real (h)	Diferencia (h)
Desarrollo	110	150	+40
Documentación	155	130	−25
Investigación	20	30	+10
Tutoría	15	25	+10
Total	300	335	+35

Tabla 2.1: Comparativa entre la planificación inicial y la dedicación real.

2.2. Análisis

El trabajo se desarrolló a lo largo de aproximadamente dos años, lo que permitió madurar, consolidar y afianzar los conocimientos requeridos del enfoque final obtenido. Esto contribuye a explicar que las horas reales superen en 35h al supuesto inicial.

En la estimación inicial se asignaron más horas a la documentación que al desarrollo, bajo la suposición de que redactar la memoria en $\text{\LaTeX}$ con rigor académico sería la tarea más costosa. La realidad fue la contraria. La base de código y los materiales interactivos que combinan texto, código y visualizaciones pedagógicas, ocuparon más tiempo del estimado. Asimismo, el desarrollo terminó siendo la categoría más costosa, con 40h adicionales sobre lo previsto.

Debido a la prolongación en el tiempo del proyecto se vieron incrementados los supuestos para la parte de investigación y las tutorías en un total de 10h, cada una. Hay que destacar que las tutorías permitieron orientar el proyecto cuando se produjeron bloqueos sobre la dirección o desarrollo del mismo.

3. Redes Neuronales Artificiales

En la actualidad, el término Red Neuronal Artificial (RNA, del inglés Artificial Neural Network, ANN) se ha vuelto tan popular y prácticamente igual de utilizado que el término IA, llegando a tratarlos casi como sinónimos. Nada más lejos de la realidad, nos remontamos a la década de los 40, cuando Alan Turing plantea un sistema que posteriormente sería desarrollado por Warren McCulloch y Walter Pitts, asentando así las bases de lo que hoy conocemos como IA, concretamente, en la rama del aprendizaje automático (del inglés Machine Learning, ML). Estos sistemas de computación están basados en el funcionamiento de las redes neuronales biológicas, es decir, el cerebro humano; intentan abstraer el complejo funcionamiento interno y centrarse en cómo nuestro cerebro procesa la información [8].

Nuestro cerebro es una compleja red de pequeños núcleos, llamados neuronas que están conectadas entre sí mediante brazos llamados axones. Estas se encargan de procesar la información que captamos del entorno mediante nuestros sentidos y nos permiten aprender en base a la experiencia y la exposición. Imaginemos a un niño pequeño que intenta aprender a diferenciar entre un círculo y un cuadrado. Cuando ve estas formas por primera vez, sus sentidos (vista, principalmente) captan la información visual, que es procesada en distintas regiones del cerebro como la corteza visual, encargada de interpretar patrones y características del entorno. A medida que el niño observa repetidamente estas figuras y recibe estímulos externos (como alguien que le dice “esto es un círculo”), su cerebro comienza a establecer conexiones entre lo que ve y lo que aprende. Estas conexiones ocurren entre neuronas, y cuanto más se repite la experiencia, más fuerte se vuelve la sinapsis entre ellas: este es el fundamento del aprendizaje. Además, la memoria del niño funciona de forma distribuida; puede recordar inmediatamente lo que ha aprendido (memoria a corto plazo), pero con la repetición y el uso frecuente, esa información se consolida y pasa a la memoria a largo plazo. Este proceso inspira a las redes neuronales artificiales, que intentan imitar cómo los humanos procesan información sensorial, aprenden mediante la repetición ajustando la “fuerza” de sus conexiones, y almacenan conocimiento de forma gradual y jerárquica.

Si bien es cierto que las redes neuronales no son tan complejas como sus homólogas biológicas, también poseen una estructura interna cuyo estudio es realmente importante. A grandes rasgos, podemos diferenciar tres factores importantes en la definición de una RNA: su arquitectura, es decir, la conexión entre sus neuronas, cómo están agrupadas y qué formas tienen de procesar la información; el algoritmo de aprendizaje utilizado, que definirá la forma en la que se calculan los pesos de las conexiones; y las funciones de activación, que permitirán introducir no linealidad en el modelo y decidir si una neurona se activa o no ante ciertos estímulos [9]. La arquitectura de modelos es una subárea de la IA que se encarga del estudio del funcionamiento y la eficacia de la configuración de

un modelo en función del conjunto de datos. Gracias a su desarrollo, podemos saber qué estructuras y configuraciones son mejores para ciertos tipos de problemas.

Siguiendo el ejemplo del niño pequeño que aprende a diferenciar formas, en el mundo de la IA existen problemas o conjuntos de datos que han sido ampliamente utilizados para verificar y testear el desarrollo de modelos. A lo largo del trabajo hablaremos sobre diferentes tipos de algoritmos y configuraciones, por lo que será necesario implicar a un sujeto de pruebas para ilustrar y comparar el rendimiento de cada enfoque. En concreto, el conjunto de datos será MNIST (Modified National Institute of Standards and Technology), un conjunto clásico y accesible que cuenta con un total de 70.000 imágenes de dígitos escritos a mano. Habiendo servido durante años como punto de partida en el desarrollo y evaluación de modelos de reconocimiento visual, este conjunto nos permitirá simular un proceso de aprendizaje progresivo en el que podamos observar cómo los distintos modelos son capaces de identificar patrones, adaptarse a la información y mejorar su precisión con la experiencia [11].

3.1. Arquitectura de una Red Neuronal

Conceptualmente, una RNA está formada por unidades, llamadas neuronas, interconectadas y organizadas en lo que se denominan capas. Cada neurona se encarga de procesar desde una hasta múltiples señales de entrada, mediante la combinación de dos operaciones básicas: una combinación lineal ponderada de las entradas, y la aplicación de una función de activación no lineal. El resultado se transmite a las neuronas de la capa siguiente, de forma que la información siempre fluye unidireccionalmente desde la capa de entrada hasta la de salida. Por ejemplo, al procesar una imagen (representada como un vector de números) en la primera capa, cada unidad recibe una parte de la información de entrada, la procesa y entrega su resultado a la siguiente, así sucesivamente hasta que llegamos a la última capa y, por tanto, a la salida de la red. Combinando estos tratamientos, podemos lograr que una red sencilla sea capaz de, en primer lugar, detectar bordes; luego, que las capas intermedias los combinen para reconocer formas; y finalmente, que la última capa genere una clasificación final. Esto es, a rasgos generales, la estructura de una RNA [7].

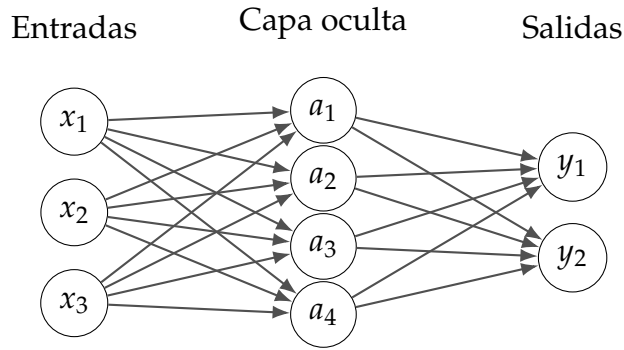


Figura 3.1: Red neuronal totalmente conectada (3–4–2).

3.1.1. Las capas

Como hemos mencionado, existen diferentes tipos de capa: la capa de entrada, es la interfaz de la red, recibe datos vectorizados y los transmite al resto de capas [12]; las capas ocultas, que realizan transformaciones no lineales mediante matrices de pesos y funciones de activación; la capa de salida, encargada de traducir y codificar el resultado de la red a un formato interpretable en el contexto del problema. El aprendizaje ocurre mediante el ajuste dinámico de los pesos de las conexiones durante el entrenamiento, permitiendo a las capas ocultas extraer características jerárquicamente más abstractas de los datos [7].

La cantidad de neuronas de una capa oculta puede variar dependiendo del caso, pero no existe ningún método que nos permita determinar su número óptimo. Numerosos estudios destacan que solo la experimentación durante el entrenamiento permite aproximar cuántas unidades se requieren para una tarea específica. Además, en ciertos escenarios complejos puede requerirse la incorporación de múltiples niveles ocultos, aunque en la mayoría de las aplicaciones, dos capas ocultas suelen ser suficientes [8].

3.1.2. La neurona

La neurona es la unidad computacional básica, es en ella donde se realizan todas las operaciones para determinar la salida de la red. Su estructura consta de tres componentes: canales de entrada, núcleo y canales de salida.

Los canales de entrada son las interfaces por las que la neurona recibe las señales $x_1, x_2, \dots, x_n$. Normalmente, se representa con un vector cuyas posiciones tienen un peso w_i asociado (la memoria de la red), con lo que se determina qué entradas son más importantes. Luego, el núcleo, en el que se aplican dos funciones: la función de combinación (de las entradas y sus pesos), y la función de activación, que aplica una transformación no lineal. Por último, el canal de salida por el que transmite el resultado a las neuronas de la capa siguiente, sirviendo de entrada para nuevos cálculos [12].

Los pesos w_i constituyen la "memoria" de la red y se optimizan durante el entrenamiento para minimizar errores. Aunque la arquitectura es fija, la capacidad de aprendizaje proviene de la adaptación de estos parámetros con cada iteración [2].

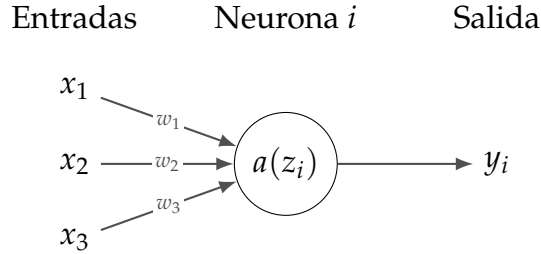


Figura 3.2: Diagrama funcionamiento neurona simple.

Función de Combinación

La función de combinación transforma las entradas $\vec{x} = [x_1, x_2, \dots, x_n]^T$ utilizando sus pesos asociados $\vec{w} = [w_1, w_2, \dots, w_n]$, se representa con la operación punto ($\cdot$). Su forma estándar es la suma ponderada:

$$z = \vec{w} \cdot \vec{x} = \sum_{i=1}^n (w_i \cdot x_i) \quad (3.1)$$

Existen diferentes modificaciones de esta función, como aplicar funciones de agregación como el $\max(z)$ o el $\min(z)$, o combinaciones booleanas como la conjunción $\bigwedge_i (w_i \wedge x_i)$ o la disyunción $\bigvee_i (w_i \vee x_i)$, todo dependerá del contorno del problema. Sin embargo, la suma ponderada (3.1) es ampliamente utilizada por su diferenciabilidad¹, propiedad esencial para el entrenamiento mediante el descenso del gradiente [2].

Función de Activación

El propósito de una función de activación es el de introducir **no linealidad** en el modelo, permitiendo que la red aprenda relaciones complejas en los datos. Sin funciones de activación, una red neuronal sería equivalente a una simple regresión lineal (combinación lineal de todas las capas $L_T = L_1 + L_2 + \dots + L_n$), incapaz de resolver problemas complejos como el reconocimiento de imágenes o el procesamiento del lenguaje natural [2, 14].

Su nombre proviene de la capacidad que tiene para decidir si la neurona *se activa* o no. Como se puede observar en la figura 3.2, toma el valor z de salida de la combinación (3.1) y le aplica una operación matemática no lineal $a = \sigma(z)$, donde σ representa la función de activación. Decimos que la neurona se activa cuando

¹La diferenciabilidad es un concepto fundamental en el cálculo infinitesimal y la matemática en general. Se refiere a la propiedad de una función de poder ser derivada en un punto específico [13].

supera el umbral de la función de activación, y consecuentemente, decimos que no se activa cuando no supera dicho umbral. Por ejemplo, en un problema binario, en el que las salidas deben ser 0 o 1, podemos utilizar 3.2 como función de activación [10], donde th es el umbral de activación.

$$\begin{cases} 0 & \text{si } \vec{w} \cdot \vec{x} \leq th \\ 1 & \text{si } \vec{w} \cdot \vec{x} > th \end{cases} \quad (3.2)$$

Debido a la búsqueda de una notación más simplificada y cómoda, el umbral de activación se suele desplazar al primer término de la desigualdad y reemplazarlo por lo que se conoce como el sesgo de la neurona ($b \equiv -th$), tal y como en 3.3 [10].

$$\begin{cases} 0 & \text{si } \vec{w} \cdot \vec{x} + b \leq 0 \\ 1 & \text{si } \vec{w} \cdot \vec{x} + b > 0 \end{cases} \quad (3.3)$$

Esta modificación permite expresar el umbral como si fuese otra entrada más a la neurona con 1 como peso asociado. Simplificando nuestra función de combinación a 3.4.

$$z = \vec{w} \cdot \vec{x} + b \quad (3.4)$$

Los sesgos son un factor clave para la activación de una neurona o no. Cuanto mayor sea su valor más sencillo será para una neurona activarse, mientras que si toma un valor muy negativo, puede llegar a ser muy difícil que esa neurona se active [10]. Tanto los pesos w_i y como los sesgos b se ajustan iterativamente durante el entrenamiento. Este proceso permite a la red priorizar entradas relevantes asignando pesos altos a características discriminativas, suprimir ruido atenuando señales irrelevantes (pesos cercanos a cero) y modelar compensaciones, ya que el sesgo actúa como umbral de activación base.

Perceptrón

Existen diferentes tipos de funciones de activación y elegiremos una u otra en función de el contorno del problema y el tipo de dato que vayamos a utilizar. El ejemplo más simple para definir la activación de una neurona es la función escalón (utilizada en 3.3), utilizada en neuronas binarias. Cuando la combinación de entradas de la neurona es mayor al umbral, la activación es 1; si es menor, es 0 (aunque el rango puede cambiar, por ejemplo, a $\{-1, 1\}$) [8].

$$\sigma(z) = \begin{cases} 0 & \text{si } z \leq 0 \\ 1 & \text{si } z > 0 \end{cases} \quad (3.5)$$

Una neurona que aplique la suma ponderada, como función de combinación, y la función escalón, como función de activación, se denomina **perceptrón**. Esta arquitectura fue inventada y propuesta por Frank Rosenblatt en 1958 [15] y a día de hoy se utiliza como base de estudio y comprensión del funcionamiento de las redes neuronales. Sin embargo, el perceptrón clásico presenta una limitación para el

aprendizaje automático: su función escalón produce salidas binarias (0 o 1), lo que imposibilita realizar ajustes graduales en los pesos durante el entrenamiento [10], es decir, un pequeño ajuste en los pesos puede ocasionar un gran desajuste en la salida de la red.

Si modificamos ligeramente un peso o sesgo en un perceptrón, su salida puede cambiar abruptamente (de 0 a 1). Esta discontinuidad propaga cambios caóticos en redes multicapa, haciendo inviable optimizar los parámetros de forma sistemática [10]. Para resolver esto, se desarrollaron funciones de activación alternativas con dos propiedades clave:

1. Diferenciabilidad: permitir calcular gradientes suaves para ajustar pesos mediante descenso de gradiente.
2. No linealidad suave: transformar la suma ponderada (z) de manera progresiva, no abrupta.

Sigmoide

La neurona sigmoide² fue la primera en resolver las limitaciones del perceptrón ya que 3.6 genera salidas continuas, a la vez que pequeños cambios Δw_i en sus pesos producen cambios proporcionales Δa en la salida, permitiendo a la red aprender mediante correcciones incrementales (importante para la retropropagación de la red) [10].

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (3.6)$$

A simple vista, la neurona sigmoide parece muy diferente del perceptrón. Sin embargo, si realizamos un pequeño análisis algebraico veremos que la función sigmoide no es más que aplicar un filtro de suavizado a la función escalón. Supongamos que $z \equiv \vec{w} \cdot \vec{x} + b$ es un número positivo muy grande. Entonces $e^{-z} \approx 0$, por tanto, $\sigma(z) = 1$. Es decir, si z es un número positivo grande, la función de activación de la neurona será aproximadamente 1, de forma similar al perceptrón. Por otro lado, si z es un número negativo, entonces $e^{-z} \rightarrow \infty$, y $\sigma(z) \approx 0$. De forma que, cuando z es muy negativo, la función de activación tenderá a valer 0, aproximando el funcionamiento de un perceptrón [10].

Si bien es cierto que el uso de la sigmoide permitió aplicar técnicas como el aprendizaje mediante gradientes, su diseño plantea ciertas lagunas. Al mapear valores extremos de z (positivos o negativos) a regiones cercanas a 0 o 1, la derivada de la función se vuelve casi nula. Este fenómeno se conoce como desvanecimiento del gradiente (*vanishing gradient* del inglés) y dificulta el ajuste de pesos en capas profundas, ya que los gradientes se atenuan exponencialmente (van desapareciendo, como el nombre indica) durante la retropropagación [14]. Además, plantea otro problema relacionado con el intervalo (0,1) de salida de la función, haciendo que todas sus neuronas generen valores positivos, ralentizando

²Es común que, en la literatura técnica, las neuronas se nombren según la función de activación que utilizan.

la convergencia en el entrenamiento. Actualmente, este tipo de neurona se utiliza sobre todo en modelos de clasificación binaria [2].

ReLU

La unidad lineal rectificadora (ReLU, del inglés Rectified Linear Unit) es una función de activación ampliamente adoptada en redes neuronales profundas debido a su simplicidad y eficacia. Definida por la ecuación 3.7, ReLU transforma la salida de la función de combinación z aplicando una no linealidad con umbral en cero:

$$\sigma(z) = \max(0, z) = (z)^+ \quad (3.7)$$

A diferencia de la sigmoide, ReLU preserva la linealidad para valores positivos de z , lo que facilita el flujo del gradiente durante el entrenamiento y mitiga el problema de desaparición del gradiente en redes profundas. Además, su cálculo es computacionalmente eficiente, ya que solo requiere comparar z con cero, evitando operaciones exponenciales costosas como en la sigmoide. A rasgos generales, las redes basadas en ReLU mejoran el rendimiento consistentemente en comparación a las redes basadas en funciones de activación sigmoide [2].

Sin embargo, el uso de las ReLU también conlleva riesgos. Cuando una neurona recibe constantemente entradas negativas ($z \leq 0$), su salida y gradiente se acaban anulando, y como consecuencia, sus pesos también. Este problema, bautizado como **neuronas muertas**, puede inutilizar una sección de la red. Existen variantes como la *Leaky ReLU* o la *Parametric ReLU* que intentan mitigar estos riesgos [1].

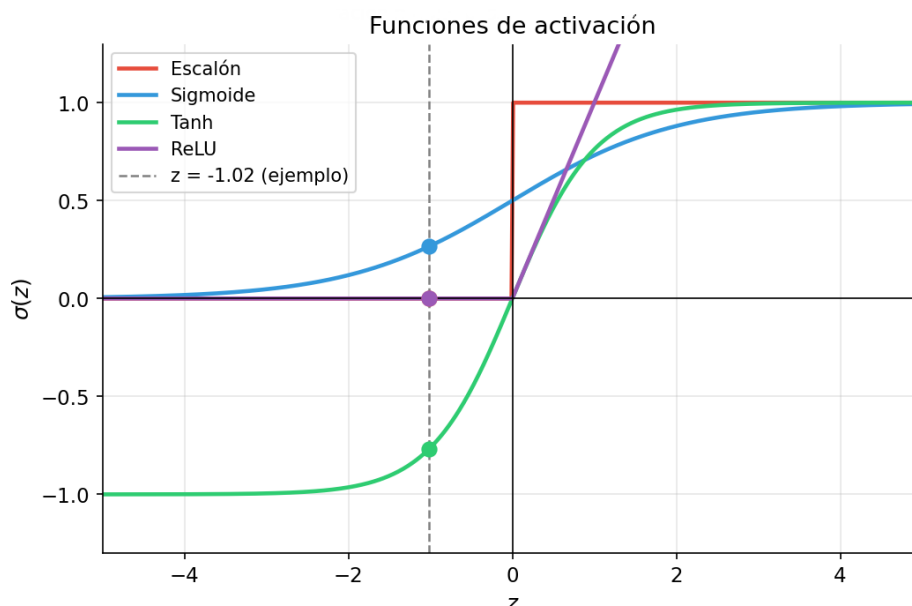


Figura 3.3: Comparación de las diferentes funciones de activación.

3.2. Entrenamiento

A lo largo de la primera sección hemos desglosado superficialmente los componentes principales de una red neuronal tradicional, desde la arquitectura de las neuronas hasta el papel de las funciones de activación. Sin embargo, estos elementos sólo constituyen el esqueleto de datos de las redes.

El proceso de entrenamiento es el mecanismo por el cual un modelo es capaz de asimilar las diferentes características de un conjunto de datos, aprendiendo así, a reconocer patrones complejos, generalizar relaciones entre entradas y salidas, o incluso, a realizar predicciones en datos nunca vistos. Para llegar a esto, el modelo deberá pasar por diferentes etapas en su entrenamiento con el fin de optimizar sus parámetros internos: el conjunto de pesos W^i y el sesgo b de cada neurona [14].

Interiorizar estos conceptos nos permite no solo entender cómo las redes neuronales aprenden, sino que también asienta las bases para diseñar arquitecturas más eficaces y diagnosticar problemas comunes como el sobreajuste, o el estancamiento en mínimos locales.

Preparación de los Datos

A la hora de entrenar entra en juego un factor casi tan importante como el propio modelo, el conjunto de datos de entrenamiento. Al igual que los atletas necesitan una dieta equilibrada y que potencie ciertas vitaminas para exprimir sus capacidades físicas en las competiciones, los modelos necesitan que el conjunto de datos que se utilice para entrenarlos capture la esencia del problema que se quiere resolver.

Generalmente, en la etapa de entrenamiento nos ayudaremos de tres conjuntos de datos: el conjunto de entrenamiento, una colección de datos que el modelo iterará para aprender las características y relaciones del problema; el conjunto de validación, que nos ayudará a evaluar el modelo tras cada entrenamiento y a ajustar sus hiperparámetros³; y el conjunto de pruebas, la colección de datos que nos permitirá verificar el rendimiento y la eficacia del entrenamiento al final del proceso.

³Características configurables de una red neuronal tales como el número o el tamaño de las capas, las funciones de activación, el método de inicialización de pesos, etc. Es decir, todo aquello que no se modifica automáticamente a lo largo del aprendizaje de la red [2, 14].

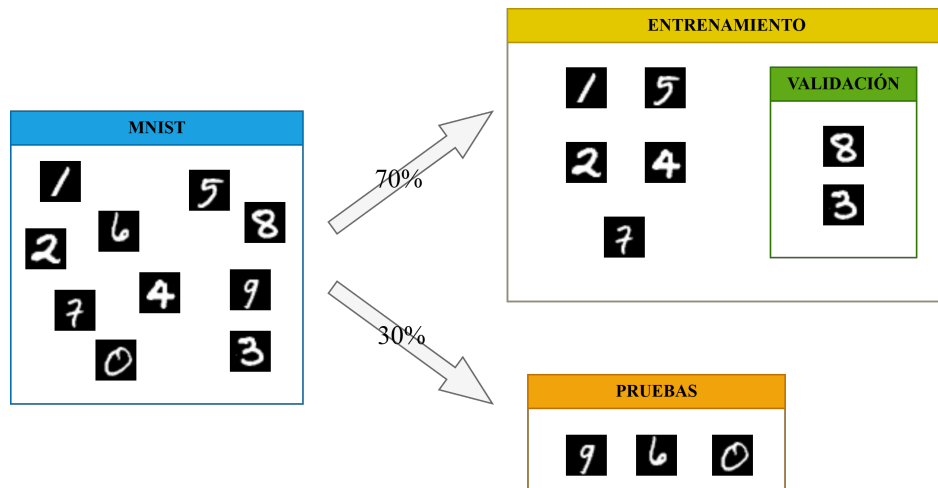


Figura 3.4: Preparación del conjunto de datos MNIST en los subconjuntos de entrenamiento, validación y pruebas.

Normalmente, la separación de datos en sus diferentes conjuntos suele realizarse aleatoriamente, aunque, se intenta mantener la proporción 20-80 o 30-70 para los subconjuntos de pruebas y entrenamiento, respectivamente. Una vez separados los subconjuntos, podríamos utilizarlos sin problema. No obstante, existen diferentes metodologías o prácticas para dividir los datos de manera que los preparemos para el modelo. Entre los más conocidos está el *K-fold Cross Validation* [2, 14].

K-fold Cross Validation. Consiste en segmentar el conjunto de datos en K subconjuntos o *folds* de tamaño similar. En cada iteración $k = 1, \dots, K$, se entrena el modelo con $K - 1$ folds y se valida con el fold restante. Al finalizar, se promedian las métricas para obtener una estimación más robusta del rendimiento y reducir la varianza asociada a una única partición entrenamiento-pruebas.

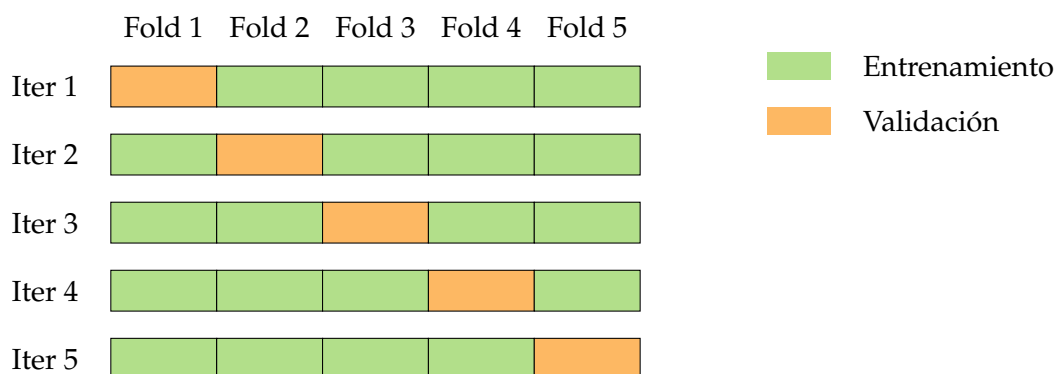


Figura 3.5: Esquema de *K-fold Cross Validation* con $K = 5$: en cada iteración, un fold distinto (naranja) se usa para validación y los restantes (verde) para entrenamiento.

3.2.1. Retropropagación

La magia del aprendizaje automático se fundamenta en un algoritmo conocido como retropropagación (*backpropagation*, en inglés). Esta técnica consiste en utilizar una función de coste al final de la red que mida la discrepancia entre el valor inferido y el real. Posteriormente, propagaremos el error desde el final hasta la primera capa de la red, actualizando sistemáticamente todos los parámetros internos que intervienen en la predicción, de forma que en la siguiente iteración hacia adelante (*forward*, en inglés) se reduzca la magnitud del error [14].

Existen diferentes funciones de pérdida, todas relacionan la salida esperada y con la predicción del modelo $\hat{y}$. En este trabajo nos centraremos en una de las más básicas y utilizadas en la literatura: el error cuadrático.

$$E = \frac{1}{2}(\hat{y} - y)^2 \quad (3.8)$$

A lo largo de la retropropagación, utilizaremos el error calculado por funciones como esta para poder actualizar todos los parámetros. Sin embargo, no todos ellos se actualizarán de la misma forma. Tenemos que ser capaces de discernir qué neuronas estuvieron más implicadas en la decisión de la red, de forma que la magnitud del cambio sea proporcional a la responsabilidad. En el mundo matemático, las herramientas utilizadas para medir la variación de un parámetro respecto de otro es la **derivada**. Esta es la base del algoritmo utilizado por antonomasia en el entrenamiento de modelos de aprendizaje automático: el descenso del gradiente.

Descenso del Gradiente

Imaginemos que somos una persona con los ojos vendados en mitad de una montaña. Si quisiéramos descenderla, bastaría con, iterativamente, dar pequeños pasitos en dirección a donde la pendiente sea menor. Básicamente, este es el razonamiento detrás del descenso del gradiente. Siendo la función del error una función continua, podemos calcular la derivada respecto de uno de sus parámetros para estudiar la pendiente y hallar mínimos locales. Eso, en resumen, es lo que hace nuestra red para aprender.

Hagamos un ejemplo con un sistema en 2D. Supongamos el final de una iteración de nuestro modelo, que nuestra función de coste es el error cuadrático y el valor esperado en la inferencia es 0, nuestra función de coste quedaría $f(x) = \frac{1}{2}(x)^2$. En este punto, siendo la red, tendríamos que calcular cómo varía el error en función del valor de nuestros parámetros. La derivada de nuestra función de coste es $f'(x) = x$. Todo esto se puede escribir en python de forma muy simple utilizando la sintaxis de las expresiones lambda.

```
1 import numpy as np
2 f_error = lambda x: 1/2 * (x)**2
3 df_error = lambda x: x
```

Luego, para representar nuestra función de coste vamos a generar un array de valores que nos permitan ver la gráfica del error centrada. Como la expresión del coste define una parábola con vértice en el 0, crearemos un vector de 1000 valores entre el -1 y el 1 . Además, seleccionaremos un punto aleatorio de este vector, ya que la inicialización de parámetros es aleatoria.

```
1 data = np.linspace(-1, 1, num=1000)
2 rnd_x = np.random.choice(data, 1)
```

Es aquí donde entra en juego nuestro algoritmo de optimización. Vamos a iterar sobre esta función de coste, para cada paso vamos a calcular hacia dónde tiene que moverse nuestro punto para minimizar el error. Para ello, utilizaremos la derivada junto con un factor de cambio que decidimos nosotros. Con él, controlaremos la magnitud del cambio sobre el punto, más tarde ahondaremos en este concepto.

```
1 fig, ax = plt.subplots()
2 ax.plot(data, f_error(data), label='f_error', color='black')
3 ax.scatter(rnd_x, f_error(rnd_x), color='red', label='inicio')
4
5 change_rate = 0.01
6 for _ in range(250):
7     slope = df_error(rnd_x)
8     rnd_x = rnd_x - slope * change_rate
9     step = ax.scatter(rnd_x, f_error(rnd_x), marker='.', color='
        blue')
10
11 ax.scatter(rnd_x, f_error(rnd_x), color='green', label='final')
```

Todas estas iteraciones, nos dan como resultado el descenso de la función de coste definida, bajando su pendiente.

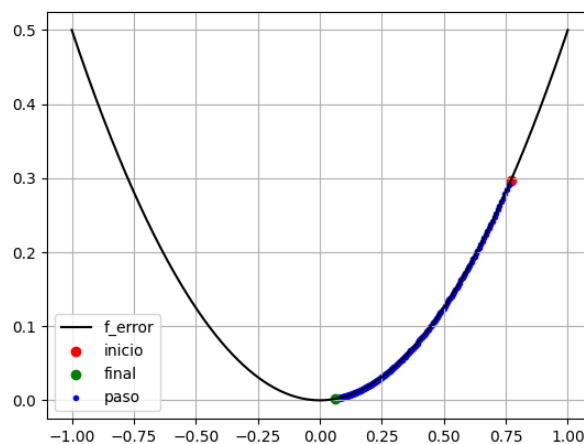


Figura 3.6: Descenso del gradiente para el error cuadrático. Cada paso acerca x al mínimo global ($x = 0$).

Aunque es simplista, este ejemplo nos ayuda a hacernos una idea de cómo funciona por dentro la actualización de los parámetros. Claro que, en una red, trabajamos a nivel multidimensional, donde los parámetros son tensores, contenedores n -dimensionales. En estos casos la notación se vuelve un poco más verbosa, ya que tenemos que emplear el **gradiente** [2].

Por definición, el gradiente de la función de coste ∇E es un vector que apunta hacia la dirección de máximo aumento del error. Su magnitud indica la tasa de cambio en esa dirección, es decir, cuanto mayor sea, mayor variación habrá. Por tanto, si buscamos $-\nabla C$, nuestro vector gradiente estará apuntando hacia donde el valor del error se minimiza, que, de hecho, es lo que buscamos para el aprendizaje de la red. Si modificamos un poco el código anterior, podemos realizar un ejemplo en 3D. En este caso, usando dos vectores, $\vec{x}$ e $\vec{y}$, la función del error la expresaremos con lo que en la literatura se denomina *one-hot encoding*. Esta notación consiste en relacionar las clases de predicción con las posiciones de un vector. Aquellas clases que son correctas se marcan con el valor 1 en la posición correspondiente, el resto a 0. Traduciendo el caso anterior, en el que el valor esperado era un 0. El error se calcularía como $f(x) = \frac{1}{2}((x-1)^2 + y^2)$.

```

1 f_error = lambda x, y: 1/2 * ((x - 1)**2 + y**2)
2 dfdx_error = lambda x: x - 1
3 dfdy_error = lambda y: y
4
5 data = np.linspace(-10, 10, 1000)
6 x, y = np.meshgrid(data, data)
7 rnd_x = np.random.choice(data, 1)
8 rnd_y = np.random.choice(data, 1)
9
10 fig, ax = plt.subplots()
11 contour = ax.contourf(x, y, f_error(x, y), levels=10)
12 ax.scatter(rnd_x, rnd_y, color='red', label='inicio')
13
14 change_rate = 0.01
15 for _ in range(250):
16     slope_x = dfdx_error(rnd_x)
17     slope_y = dfdy_error(rnd_y)
18     rnd_x = rnd_x - slope_x * change_rate
19     rnd_y = rnd_y - slope_y * change_rate
20     step = ax.scatter(rnd_x, rnd_y, color='blue', marker='.')
21
22 ax.scatter(rnd_x, rnd_y, color='green', label='final')
```

Ajustando el resto del código a este formato obtendríamos una representación tridimensional del descenso del gradiente de nuestra función de coste, desde un punto elegido aleatoriamente (punto rojo), paso a paso, hasta el mínimo (global) del error (punto verde).

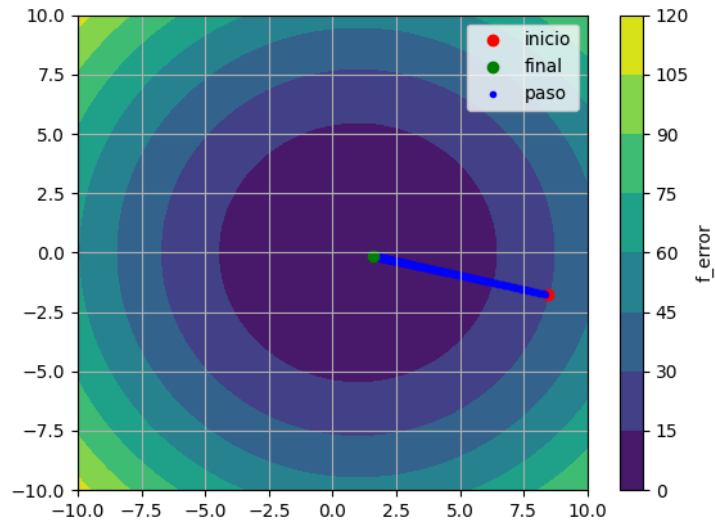


Figura 3.7: Descenso del gradiente en 3D del error cuadrático. Cada paso acerca (x, y) al mínimo global $f(x, y) = 0$.

A grandes rasgos, en cada iteración el algoritmo de retropropagación se encargará de distribuir ese error a lo largo de todas las conexiones de la red, calculando en cada nodo cómo varía la salida de la red respecto a ese parámetro buscando que la función del error se minimice. Es decir, acabamos de convertir el aprendizaje de la red neuronal en un problema de optimización [1].

Actualización de los Parámetros

Como hemos visto a lo largo de la implementación del descenso del gradiente, una de las etapas más importantes es la actualización de los parámetros. Gracias a esto, el modelo es capaz de aprender por cuenta propia, ajustándose automáticamente para ser capaz de inferir con precisión. Comúnmente, a la hora de actualizar los parámetros se utiliza lo que se denomina la **regla delta** en la literatura. Esto no es más que una formalidad para referirse al incremento o el decremento del valor de los parámetros.

$$\Delta x = \eta \frac{dE}{dx} \rightarrow x = x - \frac{dE}{dx} \quad (3.9)$$

En esta expresión redescubrimos un factor muy importante en la actualización de los pesos. Previamente, en el descenso del gradiente, regulamos la magnitud del cambio calculado en la derivada con un ratio de cambio. Este ratio de cambio expresa la "velocidad" a la que queremos que nuestro modelo aprenda. En la literatura se conoce como **tasa de aprendizaje** y en función de su valor, podemos provocar que el modelo aprenda tan lento que nunca llegue a adquirir todas las características del problema, pudiendo incluso estancarse en mínimos locales; o por el contrario, si la tasa es muy grande, el modelo difícilmente encontrará el

mínimo en el error. La elección de la tasa de aprendizaje determina la velocidad y la estabilidad del entrenamiento. Un valor óptimo permitirá a la red descender suavemente hacia configuraciones de error mínimo [2, 14, 1].

Con este ejemplo hemos podido comprender mejor cómo funciona la optimización de parámetros en una red monocapa. Pero habría que plantearse qué ocurre cuando tenemos más de una capa. En este caso, el descenso del gradiente es una técnica que nos permite saber cómo serán las variaciones del error en función de los parámetros de la capa directamente anterior a la salida. Es decir, al utilizar redes multicapa sólo podríamos afinar los parámetros de las neuronas de la penúltima capa de la red. Es aquí donde, la regla de la cadena, entra en juego, permitiéndonos calcular fácilmente el gradiente de arquitecturas complejas [2].

La Regla de la Cadena

Recapitulando un poco, hemos logrado adquirir las herramientas necesarias para calcular cómo corregir el error de predicción de nuestro modelo. Sin embargo, nuestros modelos son complejas estructuras de cálculo, pareciese que el cálculo final del error no tuviese nada que ver con aquella entrada en la primera capa. Ciertamente es que nuestros modelos son complejas redes de nodos interconectados, donde cada salida depende de una serie de operaciones intermedias. Para poder ajustar los parámetros desde la última capa hasta la primera necesitamos una manera sistemática de propagar esas derivadas hacia atrás a lo largo de la red.

Aquí entra en juego la regla de la cadena, una herramienta matemática que nos permite descomponer la derivada de una función en términos de derivadas parciales más simples. En otras palabras, la regla de la cadena actúa como un mecanismo de transmisión: el gradiente fluye desde la salida hasta las entradas atravesando cada operación que conecta ambos extremos. En la literatura, se utilizan una serie de expresiones matemáticas ⁴. No obstante, este trabajo pretende arrojar un pequeño haz de luz e intuición. Por tanto, nos centraremos en la idea sobre la que se construyeron los modelos matemáticos.

Para entenderlo de forma intuitiva, consideremos un ejemplo sencillo $f(x, y, z) = (x + y)z$, que podemos simplificar todavía más como $q = (x + y)$ y $f = qz$. En la figura 3.8 se muestra el grafo que desglosa las operaciones para $f(2, 3, 4)$.

⁴Consultar B.

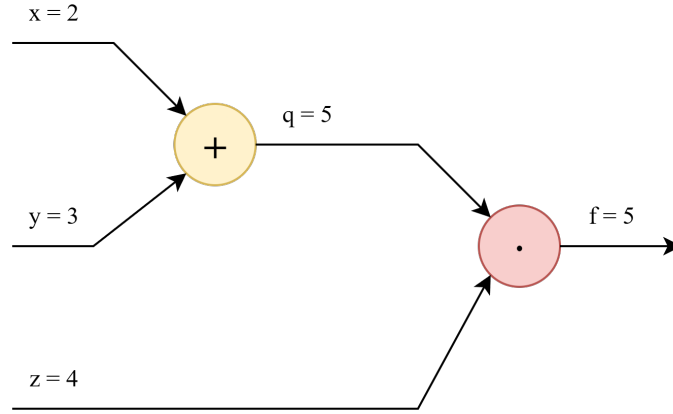


Figura 3.8: Diagrama de la función $f(x, y, z) = (x + y)z$, para $(x, y, z) = (2, 3, 4)$.

Este diagrama, en esencia, representa lo que ocurre en una red al hacer *forward*. Ahora, queremos calcular la responsabilidad de cada entrada dentro de la operación para, en caso de querer corregir los valores, saber la magnitud con la que actualizaremos cada uno. Es decir, tenemos que calcular las derivadas parciales de f respecto de cada entrada: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$ y $\frac{\partial f}{\partial z}$. Para ello, aplicaremos la regla de la cadena de las derivadas. Esta nos dice que podemos descomponer la derivada de una función en términos más simples. En nuestro caso, utilizaremos la expresión $f = qz$ como paso intermedio para calcular las derivadas parciales respecto de x e y .

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} \quad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} \quad (3.10)$$

Para z no tenemos el mismo inconveniente, ya que en la expresión $f = qz$ existe una relación directa entre la función y la variable. Ahora sí, podemos calcular paso a paso las derivadas parciales por la regla de la cadena:

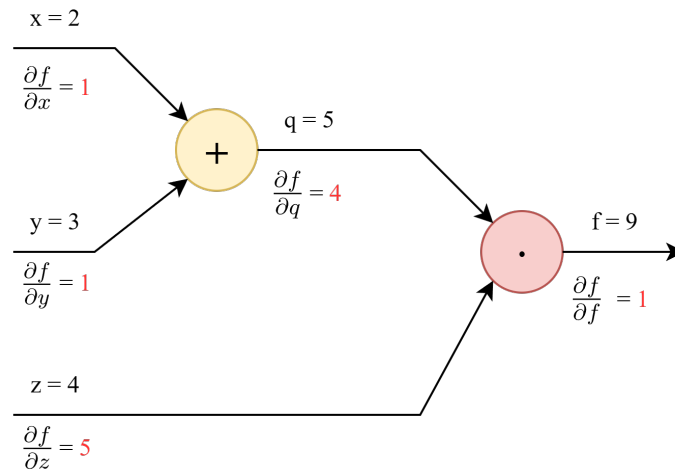


Figura 3.9: Diagrama de regla de la cadena de la función $f(x, y, z) = (x + y)z$, para $(x, y, z) = (2, 3, 1)$.

En definitiva, la retropropagación es el mecanismo que convierte la red en un sistema entrenable: cada parámetro recibe una corrección proporcional a su responsabilidad en el error final, propagada sistemáticamente desde la salida hasta la primera capa.

En un framework como PyTorch, este ciclo completo de propagación hacia adelante, cálculo del error, retropropagación y actualización de pesos se reduce a cinco líneas que se repiten en cada iteración, tal y como aparece en el método `fit()` de la clase `Trainer` en `src/models.py`:

```
1 preds = self.model(x)           # propagacion hacia adelante
2 loss  = self.loss_fn(preds, y)   # calculo del error
3
4 self.optimizer.zero_grad(set_to_none=True) # 1. borrar gradientes
    acumulados
5 loss.backward()                  # 2. retropropagar el
    error
6 self.optimizer.step()            # 3. actualizar los
    parametros
```

Extracto de código 3.1: Ciclo de entrenamiento en `Trainer.fit()` (`src/models.py`). Las tres últimas líneas implementan la retropropagación y la actualización de parámetros.

3.2.2. Generalización y Regularización

Hasta el momento, hemos abordado el aprendizaje de una red neuronal como un problema de optimización, con el propósito principal de entrenar modelos que no solo funcionen bien con los datos utilizados durante el entrenamiento, sino que también sean capaces de realizar predicciones precisas en datos nuevos. Esta capacidad de desempeñarse correctamente en situaciones no vistas previamente se conoce como *generalización* [1]. En esencia, buscamos desarrollar modelos que identifiquen patrones útiles y aplicables a distintos contextos, evitando simplemente memorizar los ejemplos específicos del conjunto de entrenamiento.

Problemas comunes en el entrenamiento

De acuerdo con [Goodfellow et al.](#), la capacidad de un modelo para generalizar correctamente depende tanto de su habilidad para minimizar el error en los datos de entrenamiento como de su eficacia al enfrentarse a las diferencias entre los conjuntos de entrenamiento y prueba. En este contexto, se identifican principalmente los siguientes problemas:

- **Sobreajuste:** ocurre cuando la red neuronal memoriza detalles irrelevantes del conjunto de entrenamiento en lugar de aprender patrones significativos. El resultado es un rendimiento deficiente en datos nuevos. Por ejemplo, en la figura 3.10 podemos ver cómo el modelo se ha especializado en el conjunto de entrenamiento, pero en el de validación no obtiene los mismos resultados.

- **Subajuste:** aparece cuando el modelo es demasiado simple para capturar la complejidad de los datos, lo que impide alcanzar un buen desempeño incluso en el propio conjunto de entrenamiento.
- **Alta varianza y sesgo:** relacionados respectivamente con modelos excesivamente complejos (varianza elevada) o excesivamente simples (alto sesgo).

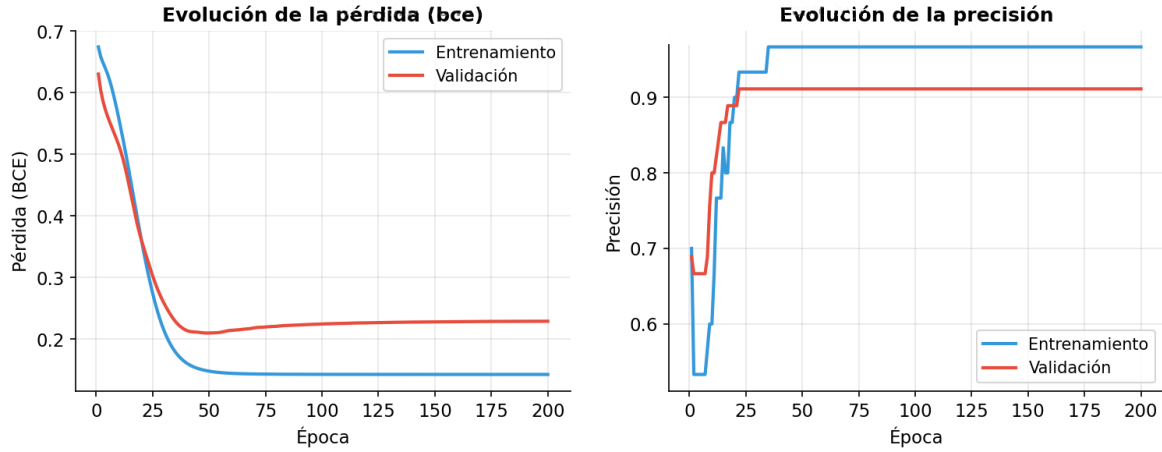


Figura 3.10: Repercusión del sobreajuste en el entrenamiento de un modelo.

Para mitigar estas situaciones, existen diversas técnicas que actúan directamente sobre los parámetros del modelo durante el entrenamiento, conocidas como regularización, entre las más utilizadas se encuentran la L1 y L2, el dropout y la normalización de los batches.

Regularización L1. Consiste en añadir un término proporcional a la suma de los valores absolutos de los pesos en la función de pérdida:

$$\mathcal{L}_{total} = \mathcal{L}_{original} + \lambda \sum_i |w_i| \quad (3.11)$$

Este mecanismo promueve la *esparsidad* en los parámetros, forzando a que muchos de ellos tomen valor cero. De este modo, el modelo tiende a ser más simple y puede realizar una selección implícita de características relevantes.

Regularización L2. También denominada *weight decay*, esta técnica penaliza la magnitud cuadrática de los pesos:

$$\mathcal{L}_{total} = \mathcal{L}_{original} + \lambda \sum_i w_i^2 \quad (3.12)$$

A diferencia de L1, no fuerza explícitamente pesos nulos, pero sí tiende a mantenerlos pequeños. Esto contribuye a estabilizar el entrenamiento y mejorar la capacidad de generalización al evitar que el modelo dependa en exceso de ciertos parámetros.

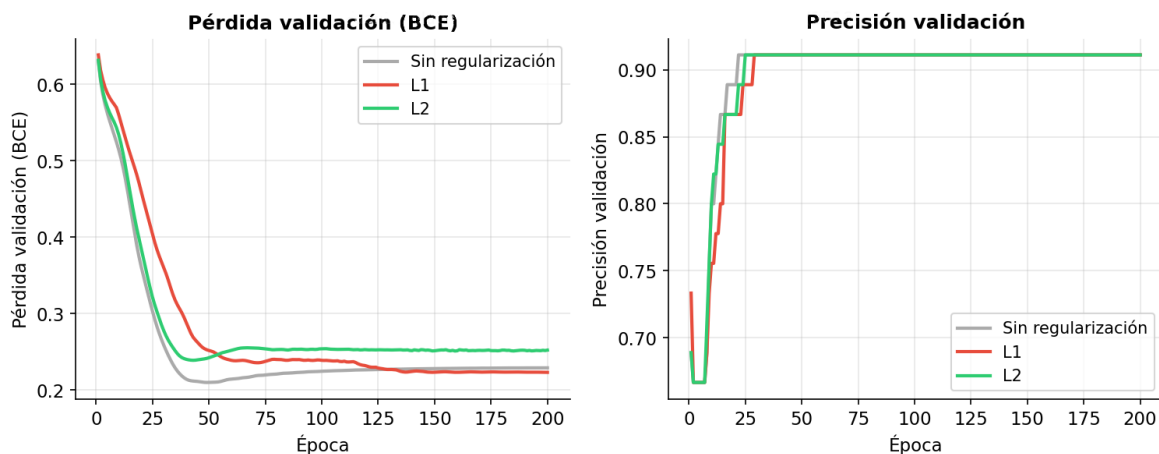


Figura 3.11: Efecto de la regularización en un caso de sobreajuste.

Dropout. Es una técnica estocástica que consiste en "desactivar" de forma aleatoria un subconjunto de neuronas durante cada paso del entrenamiento, el tamaño de ese subconjunto se suele configurar entre un 0,2 y un 0,5 del total de neuronas de la capa. De esta forma, la red no puede depender excesivamente de neuronas concretas y se ve obligada a explorar otros caminos dentro de la red, volviéndose más robusta. En la práctica moderna (*inverted dropout*), durante el entrenamiento se escalan las activaciones para mantener su valor esperado, y en inferencia se utilizan todas las neuronas sin aplicar enmascaramiento adicional [2].

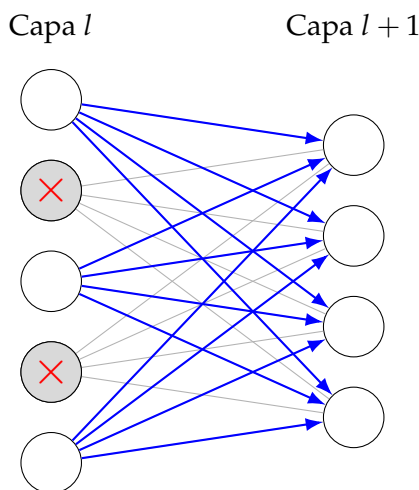


Figura 3.12: Ilustración del dropout: algunas neuronas (en gris con aspa) se desactivan de forma aleatoria durante el entrenamiento, forzando a la red a no depender de unidades específicas.

En PyTorch, el dropout se inserta como una capa más en la secuencia del modelo mediante `nn.Dropout(p)`. Al llamar a `model.train()`, la desactivación de neuronas está activo; mientras que al llamar a `model.eval()`, se desactiva y todas las neuronas contribuyen a la predicción. El código 3.2 muestra su uso en el modelo MLP512x512 de `src/models.py`, que también incorpora normalización de batches y sirve de ejemplo práctico de varias técnicas de regularización combinadas:

```

1 self.drop = nn.Dropout(p_drop)    # p_drop = 0.2 por defecto
2
3 def forward(self, x: torch.Tensor) -> torch.Tensor:
4     x = self.flatten(x)             # (B, 784)
5     x = self.drop(F.relu(self.bn1(self.fc1(x)))) # lineal + BN +
        ReLU + Dropout
6     x = self.drop(F.relu(self.bn2(self.fc2(x)))) # lineal + BN +
        ReLU + Dropout
7     return self.fc3(x)              # logits sin
        activacion final

```

Extracto de código 3.2: Dropout en MLP512x512 (src/models.py).

Batch Normalization. Aplica una transformación que mantiene la media de la salida a 0 y la desviación estándar a 1. No funciona igual durante el entrenamiento que durante la inferencia. Durante el entrenamiento, utiliza la media y la desviación estándar del batch para normalizar los datos de entrada. Durante las inferencias, el modelo utiliza un promedio móvil de la media y la desviación estándar de los batches que vió en el entrenamiento. Esta técnica, facilita la retropropagación de los gradientes del modelo, lo cual ayuda en modelos profundos [1].

En el código 3.2, la capa `nn.BatchNorm1d(512)` normaliza las 512 activaciones de cada muestra antes de que lleguen a ReLU: la secuencia `bn1(fc1(x))` aplica primero la transformación lineal y luego la normalización, antes de la función de activación y del dropout. Para mapas de características convolucionales, la variante bidimensional `nn.BatchNorm2d` normaliza por canal; puede consultarse su uso en el modelo `SmallCNNSolid` de `src/models.py`.

3.3. Limitaciones de las Redes Neuronales Tradicionales

Las redes neuronales densamente conectadas, aunque son efectivas en tareas de aprendizaje general, presentan desafíos al aplicarse a multidimensionales, como las imágenes.

Una de sus principales limitaciones es su incapacidad para aprovechar la estructura espacial. Por ejemplo, en imágenes, trata cada píxel como una entrada independiente, ignorando las relaciones de proximidad entre píxeles adyacentes, fundamental para identificar patrones locales como bordes, texturas o formas.

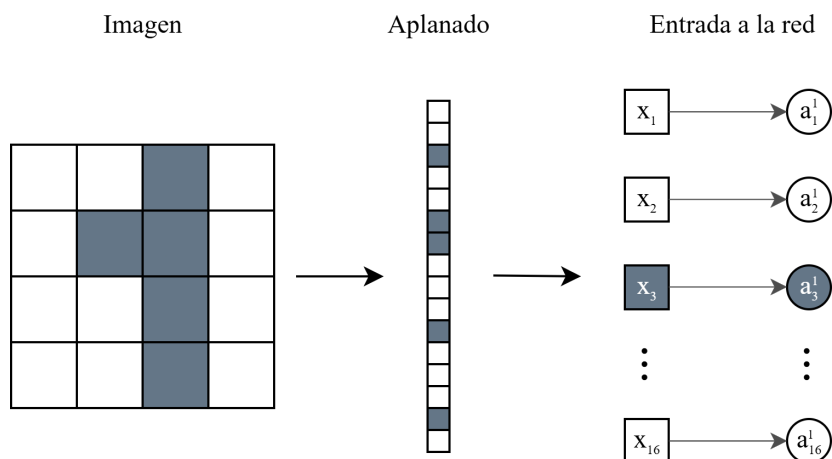


Figura 3.13: Aplanado de una imagen para servir de entrada a una red neuronal.

Adicionalmente, esta condición provoca lo que se conoce como *redundancia en el aprendizaje de características*, donde cada neurona debe aprender pesos específicos para detectar una característica en una posición particular, como por ejemplo la neurona a_3^1 de la figura 3.13, incluso si dicha característica ha aparecido en otra región de la imagen, limitando su generalización.

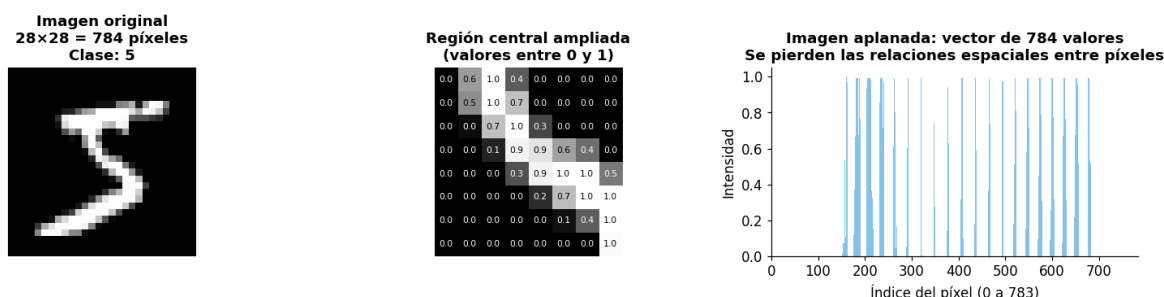


Figura 3.14: Aplanado de un dígito de MNIST.

Por otra parte, intentar modelar problemas que involucren imágenes de gran tamaño (por ej. 100x100 píxeles) puede convertirse en un reto de escalabilidad en el que los parámetros crezcan exponencialmente. Una capa densa con 1000 neuronas requeriría millones de pesos, lo que conlleva un alto coste computacional, riesgo de overfitting y un alto consumo de memoria.

Con la intención de sobrepasar dichas limitaciones, las redes convolucionales introducen mecanismos específicos adaptados a datos visuales. Mediante operaciones de **convolución**, que aplican filtros locales que trabajan con *ventanas* en la imagen, permitiendo al modelo preservar la estructura bidimensional de la entrada y aprender patrones espaciales jerárquicamente.

4. Redes Neuronales Convolucionales

A raíz de las limitaciones encontradas en las redes neuronales tradicionales, el mundo de la visión por computador se vio forzado a buscar otras técnicas más especializadas. No sólo desde el punto de vista de la precisión de los modelos, sino también, del rendimiento. Como hemos comentado en secciones anteriores, los modelos de inferencia tradicional necesitan una gran cantidad de recursos (fig. 4.1) para poder trabajar con información espacial, factor que en muchas ocasiones se convierte en limitante.

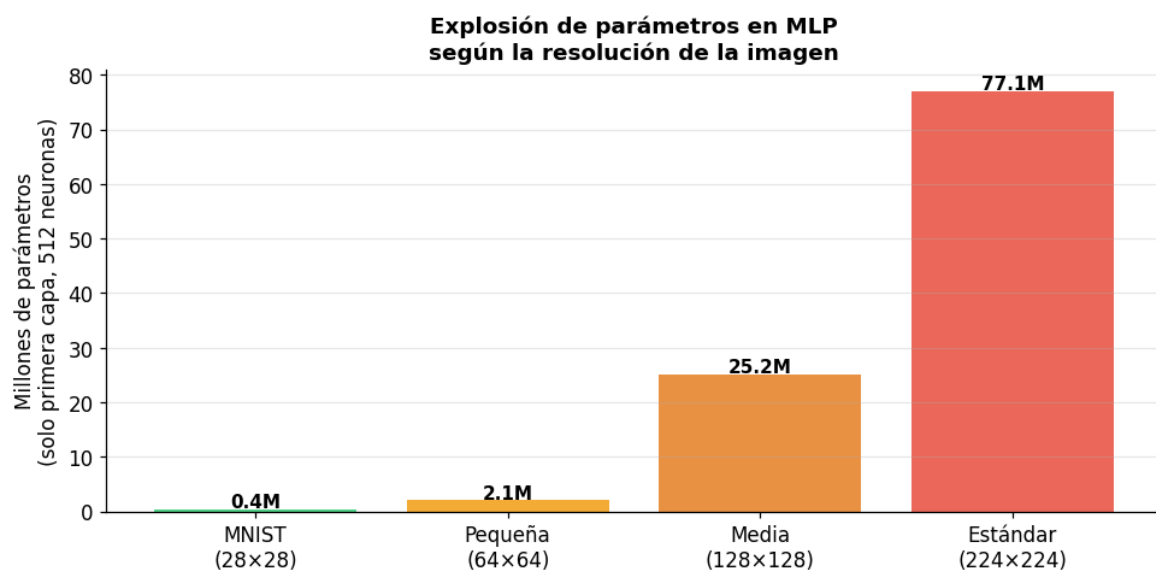


Figura 4.1: Escalado de los parámetros de un modelo tradicional.

La necesidad de un enfoque más eficiente dio lugar al desarrollo de las Redes Neuronales Convolucionales, que basan su funcionamiento en la operación de la convolución. Esta herramienta matemática permite extraer características de conjuntos de datos n -dimensionales guardando las relaciones espaciales entre los nodos, lo que la convierte en la candidata perfecta para superar las limitaciones presentadas por arquitecturas tradicionales. Estas técnicas permiten a los modelos de convolución reconocer patrones locales y elementos como bordes, texturas o contornos y sus relaciones entre diferentes regiones de una imagen de una forma más natural, como hacemos los seres humanos para reconocer objetos como pelotas, coches, animales, etc.

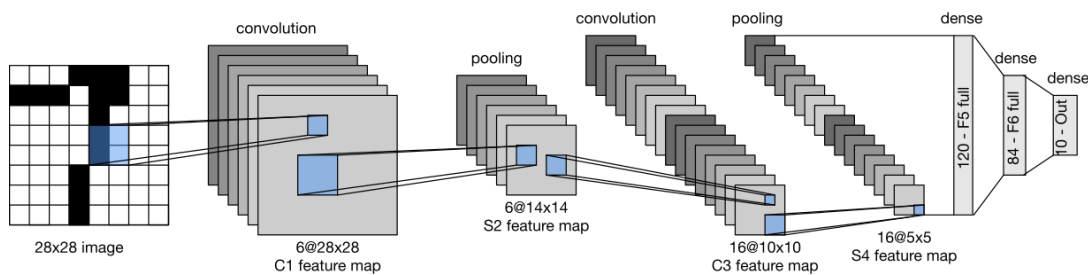


Figura 4.2: Arquitectura LeNet-5 [5].

Por otro lado, la arquitectura convolucional nos permite simplificar mucho la cantidad de parámetros que debe gestionar el modelo para llevar a cabo su tarea, en comparación con modelos tradicionales, como los perceptrones multicapa (MLP, del inglés Multi-Layer Perceptron), donde fácilmente podemos llegar a acumular centenas de miles de parámetros de conexiones entre cada par de neuronas. Esto es gracias a la compartición de pesos en las capas de convolución, que utilizan filtros (cuyos valores son parámetros de la red) que se aplican repetidamente sobre diferentes regiones de la imagen, lo que nos permite, no sólo encontrar patrones similares en diferentes regiones, sino que también reducir drásticamente el consumo de memoria, mejorar la eficiencia y la generalización del modelo.

Otro aspecto fundamental de las redes convolucionales es su capacidad para realizar una descomposición jerárquica de las características presentes en los datos de entrada. En las capas iniciales, los filtros tienden a detectar patrones muy simples, como bordes horizontales, verticales o diagonales. Conforme se avanza en profundidad, las siguientes capas combinan estas representaciones elementales para formar descriptores más complejos, como esquinas, texturas o regiones específicas de un objeto. Finalmente, en las capas más profundas, la red es capaz de representar estructuras de alto nivel a partir de la composición de las características aprendidas previamente. Este proceso refleja, en cierto modo, la manera en que el sistema visual humano descompone la información visual en niveles de complejidad creciente [2, 10, 1, 4].

Algunos de los hitos más relevantes en modelos convolucionales fueron la arquitectura *LeNet-5*, propuesta por [Lecun et al.](#) en 1998, diseñada para el reconocimiento de dígitos y letras manuscritas. Este modelo mostró por primera vez el potencial de las convoluciones para extraer características jerárquicas de las imágenes de forma automática, tanto que esta tecnología acabó implementándose en los cajeros automáticos (ATM, del inglés Automated Teller Machine) para leer cheques [16].

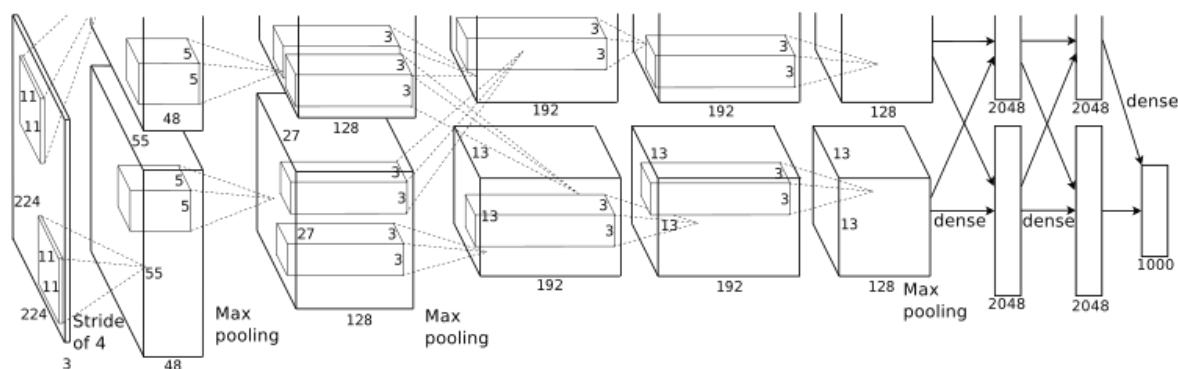


Figura 4.3: Arquitectura AlexNet [6].

Más tarde, en 2012, el interés por las redes convolucionales se disparó cuando *AlexNet* logró una mejora drástica en el concurso de referencia en clasificación y localización de imágenes a gran escala (ILSVRC, del inglés *ImageNet Large Scale Visual Recognition Challenge*) [17]. En concreto, *AlexNet* obtuvo un error top-5 del 15.3% frente al 26.2% del siguiente mejor sistema, lo que supuso una reducción relativa cercana al 42% [6]. Este resultado marcó el inicio de la adopción masiva de los modelos convolucionales y consolidó su papel como arquitectura de referencia en visión por computador.

Posteriormente, propuestas como *Network in Network* introdujeron bloques con microneas *MLPConv* y el uso de *global average pooling*, reforzando la idea de aumentar la capacidad representacional sin depender únicamente de capas densas finales [18].

Actualmente, las redes convolucionales tienen una amplia variedad de aplicaciones prácticas: reconocimiento facial en sistemas de seguridad, detección de objetos en conducción autónoma, análisis médico de imágenes como radiografías o resonancias magnéticas, clasificación de imágenes por satélite o incluso el procesamiento de vídeo en tiempo real [9, 8, 19].

4.1. Arquitectura básica

Las redes convolucionales son famosas por su arquitectura que, a menudo, recuerda a la forma de un embudo. Se desglosa en una serie de capas con orden concreto. Cada una de ellas está especializada en tareas concretas para obtener cierta información de las entradas. Esta configuración permite a la red aprender representaciones progresivamente más abstractas y complejas.

Capa convolucional

Además de darle el nombre a la arquitectura, es en esta capa en la que reside la clave del éxito de los modelos convolucionales. Como mencionamos anteriormente, la principal diferencia con los modelos tradicionales se encontraba en la forma en la que se interpreta y relaciona la información espacial de los datos de entrada a la

red. Esto es posible gracias a la operación de convolución que llevan a cabo en su interior.

La **convolución** es un operador matemático que transforma dos funciones f y g en una tercera función que, de cierta forma, representa la superposición de f y g a lo largo del tiempo [1]. Intuitivamente, puede entenderse como una operación en la que una de las funciones se "desliza" sobre la otra, midiendo en cada posición el grado de similitud entre ambas.

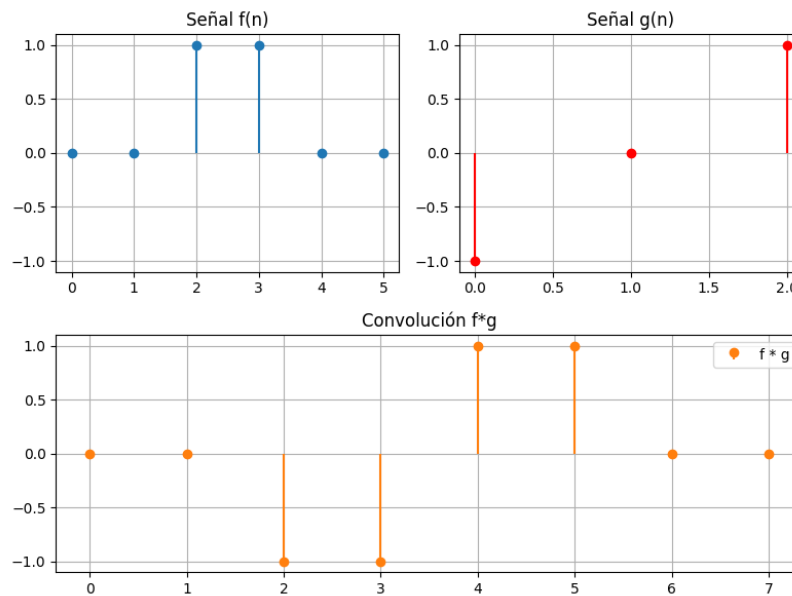


Figura 4.4: Convolución discreta entre $f = [0,0,1,1,0,0]$ y filtro de detección de bordes $g = [-1,0,1]$.

En las capas convolucionales, consiste en desplazar pequeñas matrices denominadas *filtros* o *kernels* a lo largo de la imagen de entrada a la capa y realizar una serie de cálculos entre los valores de nuestra matriz y los de la entrada que nos ayudarán a detectar características importantes [10].

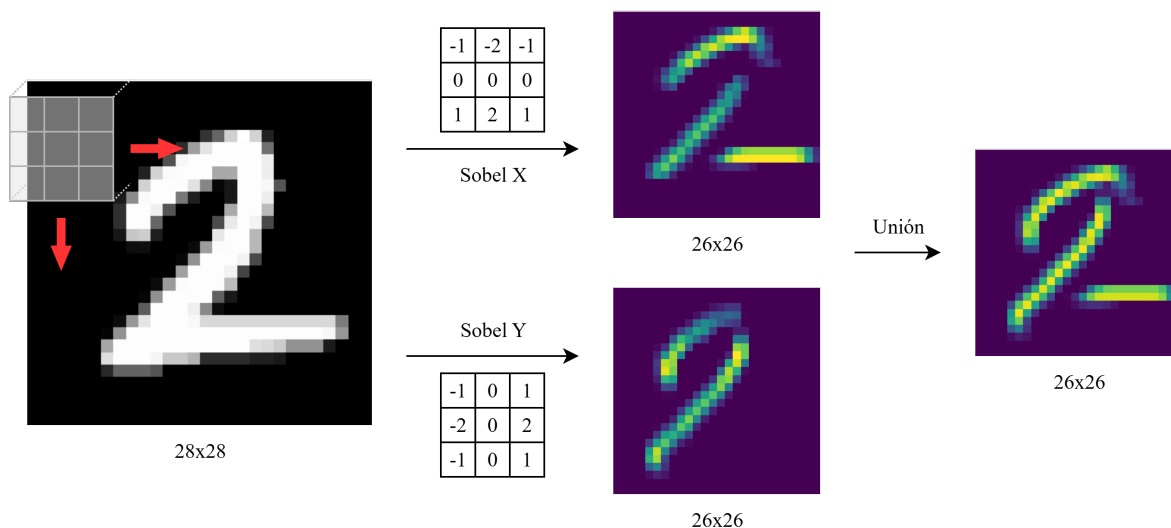


Figura 4.5: Convolución con filtro de Sobel en ejes x, y para detectar bordes.

Por ejemplo, en la figura 4.5, hemos aplicado el filtro de Sobel, kernel conocido por su capacidad para localizar bordes verticales y horizontales, para localizar las regiones de la imagen en las que podemos encontrar dichas características. Cada una de estas detecciones puede ser una de las salidas de un filtro de las primeras capas de convolución de la red. A medida que avanza la información por las diferentes capas convolucionales, se irán detectando más características, consiguiendo que la red sea capaz de captar detalles más concretos, como curvas, o contornos. Claro que, a diferencia de este ejemplo, en una capa convolucional generalmente obtenemos muchas más salidas, concretamente, tantas como filtros pasemos, por ejemplo, la salida de pasar una imagen de 28×28 por una capa convolucional con 32 filtros de 3×3 nos devuelve 32 imágenes de 26×26 (fig. 4.6).

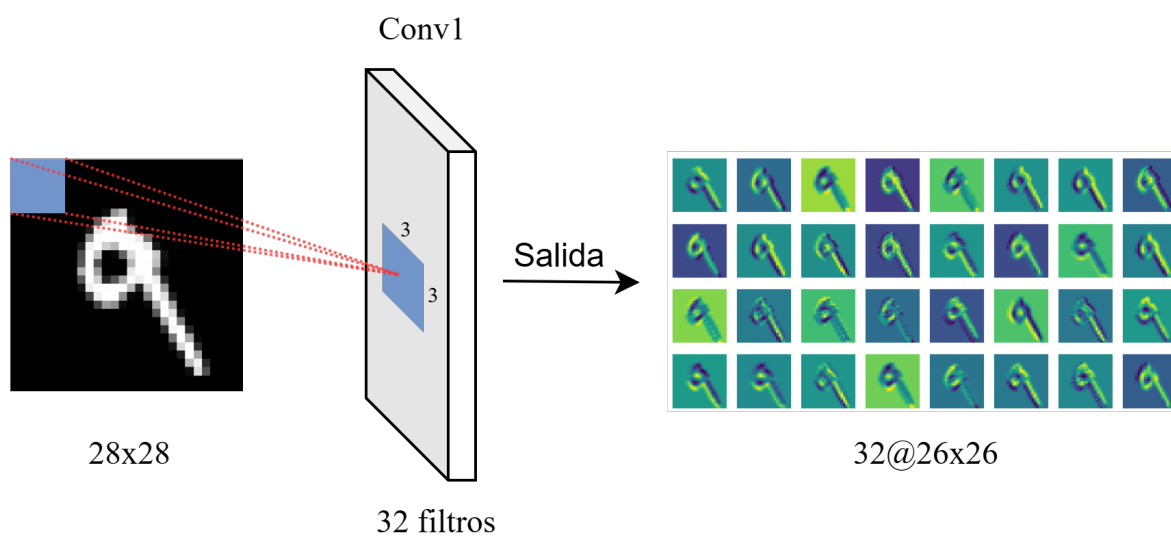


Figura 4.6: Salida de una capa convolucional con 32 filtros de 3×3 .

Capa de activación

Generalmente, las operaciones matriciales que estamos realizando sobre las entradas pueden descomponerse como una combinación lineal de la forma $W_2(W_1x) = (W_2W_1)x$. Esto aporta un inconveniente muy grande, ya que la complejidad de nuestra arquitectura se simplificaría en una combinación lineal de las entradas con los parámetros de la red, provocando que nuestro modelo no sea capaz de aprender patrones más complejos. Para corregir esta necesidad se introducen las funciones de activación. Concretamente, en este trabajo se utiliza ReLU (3.1.2), definida como $f(x) = \max(0, x)$. Donde, para $x > 0$, el comportamiento es lineal, mientras que para $x \leq 0$, la salida es constante, consiguiendo de esta forma introducir no linealidad en el modelo [14, 1].

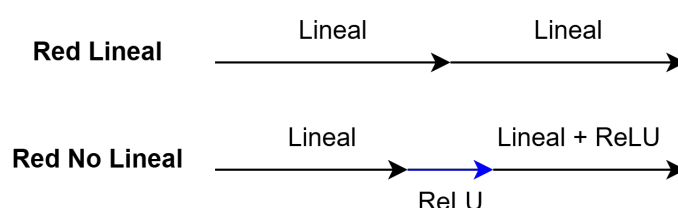


Figura 4.7: Comparación entre una red lineal y una con activación ReLU. La introducción de no linealidad permite modelar funciones más complejas.

Capa de pooling

Anteriormente, mencionamos que los modelos convolucionales tienen la capacidad de hacer una descomposición jerárquica de los datos de entrada. Las capas convolucionales, las de activación y las capas de pooling son las encargadas de extraer las características de nuestras imágenes.

La operación de pooling se refiere a una transformación cuya finalidad es reducir la dimensionalidad de la entrada. Para ello, normalmente se aplica una operación estadística el máximo, la media, o incluso la normalización L2, entre los valores seleccionados por una ventana de tamaño $n \times m$.

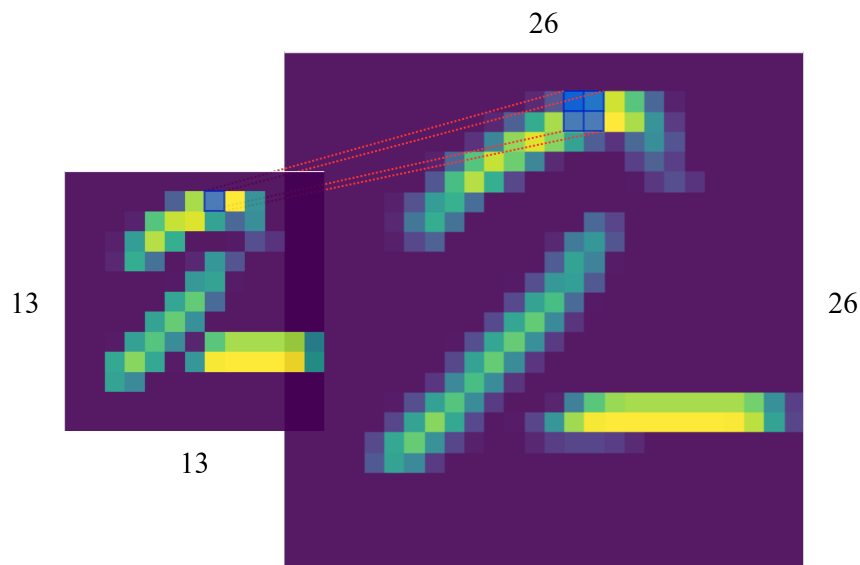


Figura 4.8: Operación de Max Pooling con una ventana de 2×2 .

Además, las operaciones de pooling ayudan a que el modelo sea capaz de extraer y reconocer patrones sin importar pequeñas modificaciones en las entradas. Es decir, aplicando pooling introducimos invarianza ante traslaciones locales de las características, volviendo nuestra arquitectura más robusta. Algunos de los filtros de pooling más utilizados son el *Max Pooling*, o el *Avg Pooling* [1].

En la literatura y en las arquitecturas actuales, lo más común es encontrarnos la tríada de capas convolucional-relu-pooling, ya que esta configuración ayuda a dar robustez al modelo y permite construir estructuras más complejas y profundas sin que el rendimiento de la red se vea agravado.

Capa completamente conexa

Llegamos a la parte final de nuestra arquitectura. Después de haber extraído todas las características aprovechando nuestros operadores espaciales, nuestra red introduce toda esa información en un conjunto de capas completamente conexas, similares a las de modelos tradicionales, donde se calcularán las relaciones entre los patrones extraídos y las clases sobre las que tenemos que predecir. Para ello, la salida de la última capa convolucional se suele aplanar para servir de entrada a la primera capa densa del modelo [10].

El modelo ConvNet de `src/models.py` implementa exactamente esta arquitectura. Su método `forward` traduce directamente la secuencia de bloques descrita: dos etapas de convolución-ReLU-pooling para extraer características, seguidas del aplanado y el clasificador denso:

```
1 def forward(self, x):
2     x = self.pool(F.relu(self.conv1(x))) # Conv1 + ReLU + MaxPool:
      28x28 -> 13x13
```

```

3     x = self.pool(F.relu(self.conv2(x))) # Conv2 + ReLU + MaxPool:
      13x13 -> 5x5
4     x = torch.flatten(x, 1) # aplanado: 64x5x5 = 1600 valores
5     x = F.relu(self.fc1(x))
6     x = F.relu(self.fc2(x))
7     x = self.fc3(x) # logits de salida (sin activacion)
8     return x

```

Extracto de código 4.1: Método forward de ConvNet (src/models.py). Las dos primeras líneas aplican la secuencia convolución-ReLU-pooling; el aplanado y las tres últimas forman el clasificador denso.

Intuitivamente, podemos pensar en la arquitectura de una red convolucional como dos partes especializadas. La primera, que se encarga de extraer características útiles para identificar a las diferentes clases, lo cual tiene un coste computacional muy alto empleando arquitecturas tradicionales. Y la segunda, que se encarga de procesar esa información “masticada” previamente por el bloque convolucional, siendo capaz de inferir la pertenencia a cada clase.

4.2. Funcionamiento interno de las redes convolucionales

4.2.1. Filtrado, stride y padding

Hemos visto que las redes convolucionales se basan en la operación de convolución, donde un filtro se desplaza sobre la entrada y aplica una transformación con la que obtiene características concretas. Además, cada uno de estos filtros puede transformar dimensionalmente la entrada que reciben, reduciéndola, estirándola, etc. Por ello, a la hora de entrenar nuestros modelos, las dimensiones de los filtros, así como la forma en que los desplazamos por las entradas son hiperparámetros¹ de configuración muy importantes, que pueden afectar directamente a la capacidad de la red para aprender y a la eficiencia computacional.

Dimensiones del filtro

El tamaño de nuestros filtros es un factor clave dentro del aprendizaje. Filtros pequeños (2×2 o 3×3) capturan detalles locales de la escena, algo así como si mirásemos desde un microscopio. Por otro lado, los filtros grandes (7×7 o 11×11) permiten captar detalles más generales, con el inconveniente de requerir un mayor número de parámetros. Es decir, la dimensión que seleccionemos, no sólo va a determinar la forma en la que nuestro modelo aprenda, sino que va a afectar directamente en nuestro rendimiento computacional.

¹Los hiperparámetros son parámetros de un modelo o del proceso de entrenamiento que **no se aprenden automáticamente** durante el entrenamiento, sino que los definimos nosotros previamente. Por ejemplo: el número de capas, la tasa de aprendizaje o el número de filtros a utilizar [1].

Además, desplazar un filtro de una determinada dimensión puede conllevar modificar la extensión de nuestra entrada en todos sus ejes. Es el caso de la figura 4.9 al pasar un filtro de tamaño 2×2 a nuestra entrada de 4×4 , reducimos sus dimensiones a 3×3 .

Stride

Define el número de píxeles que avanza el filtro en cada paso. Con $\text{stride} = 1$, el filtro se desplaza píxel a píxel, produciendo una salida de tamaño similar a la entrada. Con $\text{stride} \geq 2$, avanza dando saltos de n píxeles tal que $n \geq 2$, reduciendo la dimensionalidad del resultado. A mayores valores, el stride provoca que la inferencia se haga muy rápido, con el inconveniente de que puede significar perder de información importante en las entradas.

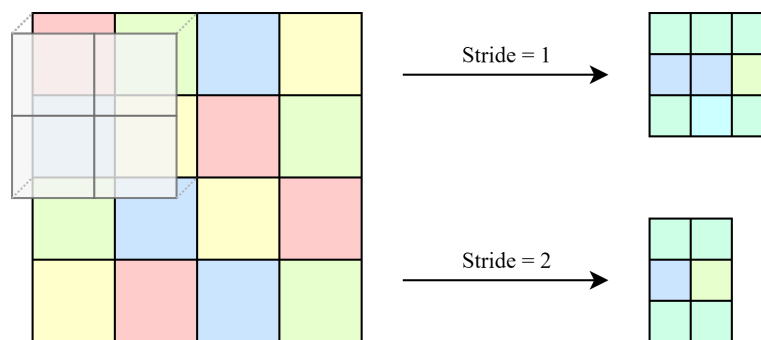


Figura 4.9: Ejemplo de average pooling con $\text{stride} = 1$ y $\text{stride} = 2$.

Padding

En una convolución, al aplicar un kernel sobre una imagen, los píxeles situados en los bordes se "visitan" menos porque el filtro no puede extenderse más allá de los límites de la imagen. Este hiperparámetro nos permite configurar filas y columnas extra de relleno, normalmente a 0, alrededor de la imagen para preservar la información de los bordes. Además, ajustando bien el padding podemos mantener el tamaño de entrada. Por ejemplo, una imagen de 4×4 con un filtro de 3×3 devuelve una imagen de 2×2 . Con el padding adecuado, en este caso $\text{padding} = 1$, puede mantenerse en 4×4 . Es decir, el padding nos permite controlar el equilibrio entre reducir la dimensionalidad y preservar la información.

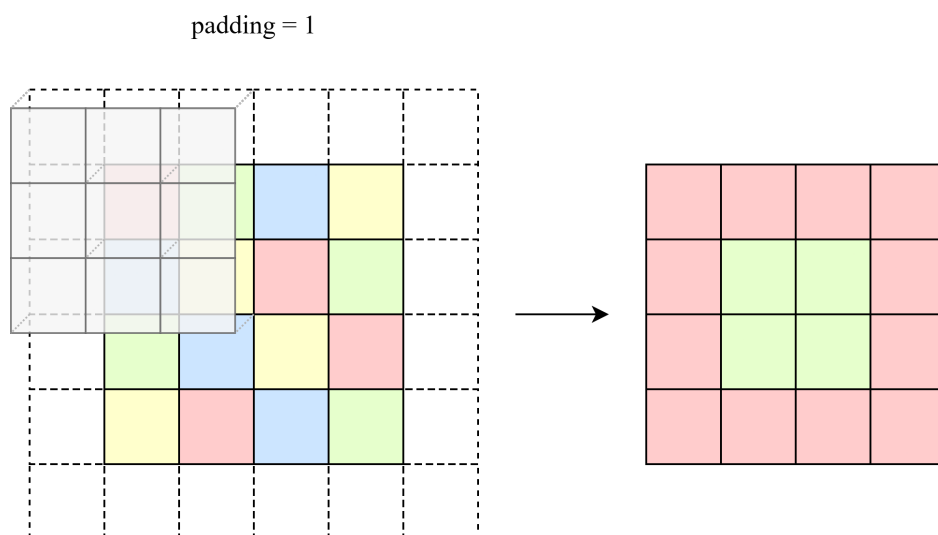


Figura 4.10: Ejemplo de average pooling con **padding** = 1.

De esta forma, diseñar las capas convolucionales de nuestra red se convierte en un meticuloso trabajo artesanal para ajustar estos hiperparámetros de modo que la potencia del modelo y su eficiencia computacional estén equilibrados [2, 14, 1].

4.3. Aprendizaje y optimización de filtros

El verdadero poder de las redes convolucionales reside en su capacidad de aprender de forma automática los filtros adecuados para extraer las características específicas que le permitan discernir adecuadamente entre las diferentes clases. Aquí entra en juego nuestro algoritmo de aprendizaje por antonomasia, la retropropagación. Al igual que los modelos tradicionales ajustan los pesos sus conexiones en base al error obtenido, los modelos convolucionales aprovechan estas iteraciones para afinar sus filtros.

Retropropagación en las capas convolucionales

El aprendizaje de los filtros sigue el mismo principio que en modelos tradicionales: los pesos (en este caso, los valores de los kernels) se inicializan aleatoriamente y se ajustan iterativamente mediante el algoritmo de retropropagación del error (3.2.1). La retropropagación permite calcular el gradiente de la función de pérdida con respecto a cada parámetro de la red, propagando los errores desde la capa de salida hacia las capas anteriores. Este gradiente se usa para actualizar los pesos en la dirección que se minimiza la pérdida. Obteniendo así, un modelo lleno de filtros capaces de detectar los diferentes patrones que presentan nuestros conjuntos de entrenamiento.

Optimización

Para actualizar los parámetros, se utilizan algoritmos de optimización basados en el descenso del gradiente 3.2.1. El descenso de gradiente estocástico (SGD, del inglés Stochastic Gradient Descent) es la técnica clásica, donde los parámetros se actualizan tras cada minibatch de datos. En la práctica, se emplean variantes más sofisticadas como Adam, RMSProp o Momentum, que aceleran la convergencia y mejoran la estabilidad del entrenamiento [2].

4.4. Representaciones internas y activaciones

Las redes neuronales convolucionales aprenden representaciones internas de los datos de entrada a través de las activaciones producidas por sus diferentes capas. Una **activación** es el valor de salida de una neurona o filtro para una entrada concreta, mientras que una representación interna es el patrón conjunto que emerge al considerar muchas activaciones de forma coordinada. Estas representaciones constituyen la base del conocimiento del modelo y reflejan qué información considera útil para la tarea de clasificación.

En las primeras capas convolucionales suelen observarse activaciones asociadas a patrones simples, como bordes, esquinas o texturas básicas. A medida que la información se propaga a capas más profundas, estas respuestas locales se combinan para formar descriptores cada vez más abstractos, como formas u objetos parciales [2].

El estudio de estas representaciones internas permite obtener una primera intuición sobre el comportamiento del modelo y sobre el tipo de información que se va preservando o descartando a lo largo de la red. No obstante, la inspección directa de valores numéricos suele resultar poco intuitiva: para interpretar el proceso de decisión es más útil proyectar esas activaciones en visualizaciones que conecten lo que ocurre dentro de la red con regiones de la imagen de entrada.

4.4.1. Visualización de representaciones internas

Con el objetivo de superar estas limitaciones, se han desarrollado distintas técnicas de visualización que permiten proyectar la información aprendida por la red a un dominio interpretable para el observador. Estas técnicas facilitan el análisis del tipo de características que se activan en cada capa y permiten relacionar las activaciones internas con regiones concretas de la imagen de entrada.

En muchas áreas de conocimiento se utiliza el término caja negra para referirse a sistemas complejos que producen una salida coherente a partir de una entrada dada, pero cuyo funcionamiento interno resulta difícilmente interpretable. Este es el caso de las redes neuronales profundas, donde el conocimiento del modelo se encuentra distribuido en los pesos y activaciones de múltiples capas interconectadas.

Por este motivo, a lo largo del tiempo se han propuesto diversos métodos de análisis que permiten comprender mejor qué ocurre en el interior de estos modelos. El uso de estas herramientas no solo contribuye a una mayor interpretabilidad, sino que también facilita la comparación entre arquitecturas, el diagnóstico de errores y el ajuste más preciso de los hiperparámetros del modelo [2, 1].

4.4.2. Herramientas de visualización

Las técnicas de visualización más utilizadas en la literatura de redes neuronales profundas pueden clasificarse, de forma general, en dos grandes grupos. Por un lado, aquellas que se basan en la información obtenida durante la propagación hacia adelante (*forward*) de la red, como los mapas de características y las activaciones neuronales. Por otro lado, las técnicas que emplean información del gradiente durante la retropropagación para identificar las regiones más influyentes en la decisión del modelo. Los mapas de saliencia, que se estudian en detalle más adelante, son el ejemplo representativo de este segundo grupo utilizado en este trabajo.

La elección de estas tres herramientas responde directamente a los experimentos del capítulo 5. Los mapas de características permiten observar qué información extrae la CNN antes y después de añadir ruido, la visualización por capa permite trazar la evolución de esa información a lo largo de la red, y los mapas de saliencia permiten comparar qué píxeles de la imagen original influyen en mayor o menor medida en la predicción del modelo.

Mapas de características

Un **mapa de características** (del inglés *feature map*) es el tensor de salida que produce un filtro convolucional al aplicarse sobre una entrada dada. Formalmente, para el filtro k -ésimo de una capa convolucional, el valor en la posición (i, j) del mapa resultante se define como:

$$a_{ijk} = \text{ReLU} \left(\sum_{u,v} w_{k,u,v} \cdot X_{i+u,j+v} \right) \quad (4.1)$$

donde $w_{k,u,v}$ son los pesos del filtro k en la posición relativa (u, v) , y $X_{i+u,j+v}$ es el valor de la entrada en la posición correspondiente [2]. La función ReLU se aplica elemento a elemento tras la suma ponderada para introducir no linealidad.

La salida completa de una capa convolucional con K filtros es un tensor tridimensional de dimensiones $H \times W \times K$, donde H y W son las dimensiones espaciales resultantes y K es el número de filtros. Cada filtro produce un único canal en ese tensor, que codifica la respuesta del filtro ante los distintos parches de la entrada. Para visualizar esta información, cada canal se representa como una imagen 2D independiente, de modo que se obtienen K imágenes separadas, una por filtro [2, 4].

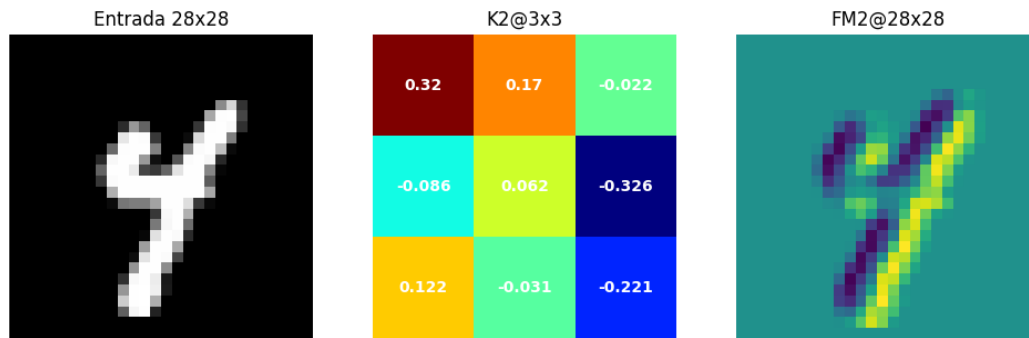


Figura 4.11: Mapa de características obtenido al pasar un kernel de una capa convolucional por una imagen de entrada.

En las primeras capas suelen aparecer activaciones asociadas a bordes, esquinas o texturas simples, ya que los filtros iniciales aprenden a detectar características locales de bajo nivel. Conforme se avanza en profundidad dentro de la red, los mapas de características reflejan representaciones cada vez más abstractas: desde contornos y formas geométricas hasta estructuras que codifican partes de objetos [10, 4].

La visualización de estos mapas permite identificar qué filtros destacan regiones relevantes de la entrada y comprender cómo la red interpreta la información visual. En el contexto de MNIST, es habitual observar filtros tempranos sensibles a trazos verticales u horizontales, mientras que los filtros de capas más profundas responden a combinaciones de trazos que distinguen unos dígitos de otros [2].

Visualización de activaciones por capa

La visualización de activaciones por capa, o también conocidos como mapas de activación, consiste en obtener para una imagen de entrada dada, la salida de cada capa de la red procesada en orden secuencial. A diferencia de la visualización de un único filtro aislado, que examina la respuesta de un solo detector ante distintas entradas, el análisis por capa recorre todas las capas de la arquitectura y recoge, para cada una de ellas, el conjunto completo de sus mapas de características. El resultado es una secuencia de tensores que muestra cómo la representación de la imagen evoluciona desde las primeras capas, cercanas a la entrada, hasta las más profundas, próximas a la capa de clasificación [2].

En las capas iniciales, los mapas de activación conservan una alta similitud visual con la imagen de entrada: los filtros detectan bordes, texturas y gradientes de intensidad reconocibles. A medida que la información avanza por las capas intermedias, los mapas se vuelven progresivamente menos interpretables de forma directa, ya que codifican combinaciones no lineales de características que la red ha aprendido a asociar con distintas clases. En las capas más profundas, los mapas suelen mostrar un alto grado de abstracción, con pocas regiones activas pero muy selectivas frente a la entrada concreta [2].

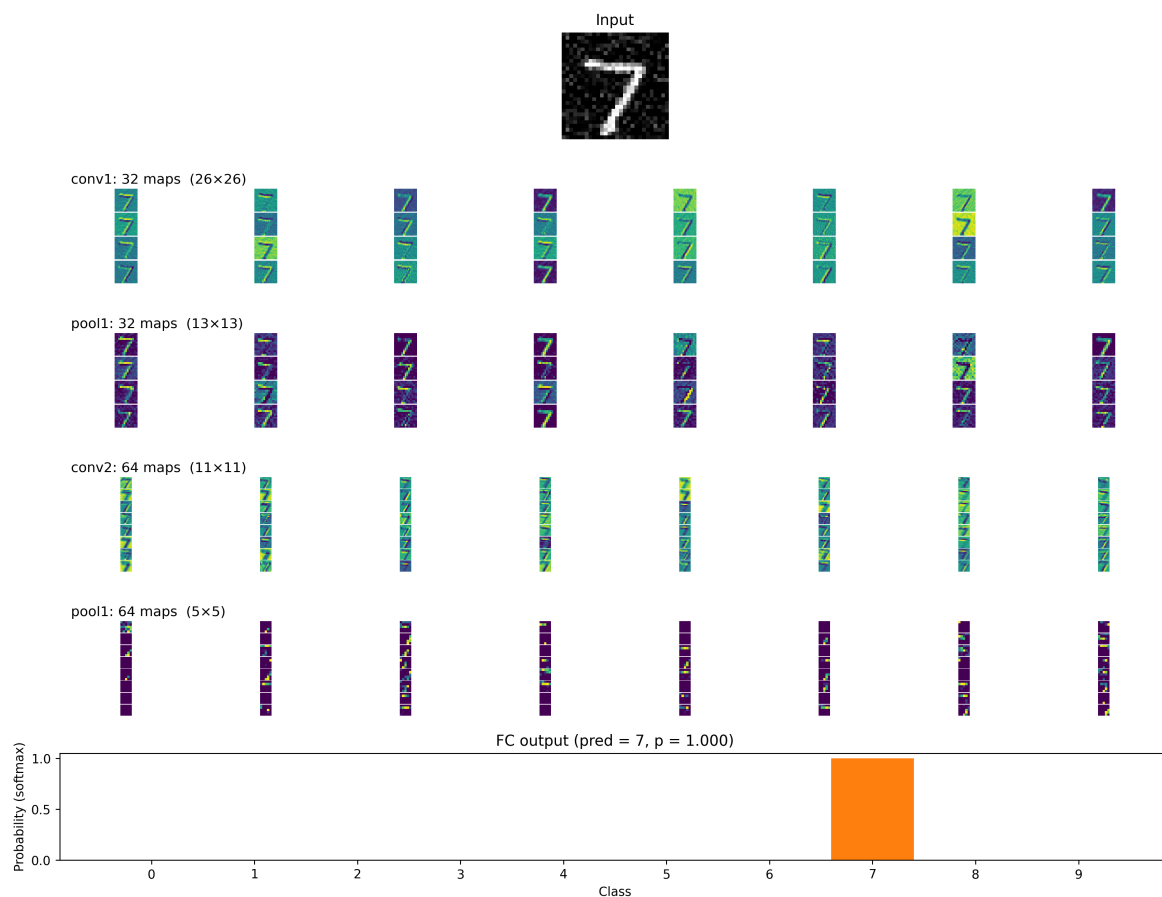


Figura 4.12: Pase hacia adelante en una red convolucional.

La comparación de activaciones generadas por distintos ejemplos de entrada permite analizar la consistencia de los patrones aprendidos y detectar posibles anomalías en el entrenamiento, siendo especialmente útil durante la fase de desarrollo del modelo.

Una forma recomendable de presentar este análisis es comparar, para una misma imagen, las activaciones de una capa temprana, una intermedia y una profunda. Este contraste muestra con claridad la transición entre información local de bajo nivel y representaciones de mayor contenido semántico [2]. Por ejemplo, la figura 4.13, muestra las características extraídas del número 5 en las 2 primeras capas convolucionales de una red.

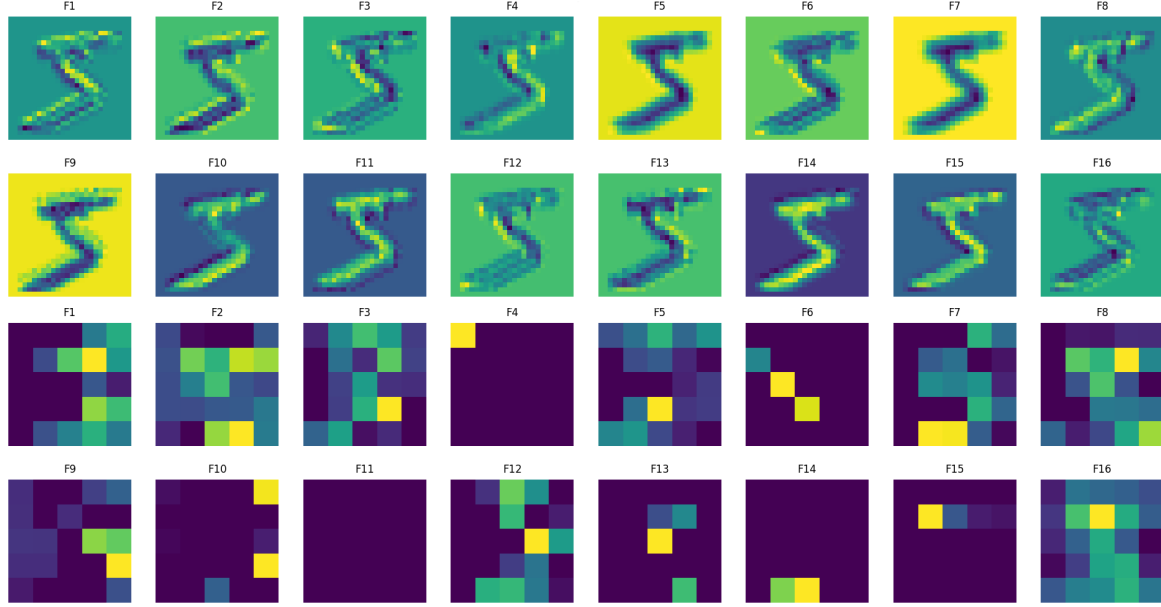


Figura 4.13: Características extraídas del número 5 en las capas *Conv1* (dos primeras filas) y *Conv2* (dos últimas filas).

Mapas de saliencia

Los mapas de características y la visualización por capa operan en el espacio intermedio de la red, sus tensores de salida tienen dimensiones que no coinciden con las de la imagen de entrada original, lo que dificulta identificar directamente qué regiones de la imagen han provocado una determinada activación o predicción. El gradiente proporciona un mecanismo para trasladar esa información de vuelta a la entrada, al calcular la derivada de la salida del modelo con respecto a cada píxel de la imagen inicial, se obtiene una medida de sensibilidad que cuantifica cuánto influye cada píxel en la predicción final.

El fundamento detrás de los mapas de saliencia es que si un píxel concreto tiene una gran influencia sobre la predicción del modelo, entonces una pequeña variación en ese píxel producirá un cambio notable en la puntuación de la clase predicha. El gradiente mide exactamente esa sensibilidad. Por tanto, calcular la derivada de la puntuación de una clase con respecto a cada píxel de la imagen permite identificar qué regiones son más determinantes para la decisión del modelo [3].

Formalmente, dado que las imágenes de entrada pueden tener varios canales, el gradiente se calcula para cada canal por separado. Para obtener un único mapa 2D interpretable, se toma el máximo valor absoluto a lo largo de los canales [3]:

$$M_{ij} = \max_c \left| \frac{\partial S_c}{\partial I_{ij}^{(c)}} \right| \quad (4.2)$$

donde S_c representa la puntuación de la clase predicha, $I_{ij}^{(c)}$ el valor del píxel en la posición (i, j) del canal c , y M_{ij} el valor resultante en el mapa de saliencia.

El valor absoluto se aplica porque lo relevante es la magnitud del efecto de cada píxel sobre la salida, independientemente de si su variación aumenta o disminuye dicha puntuación. El resultado es un mapa de calor en el que los valores de mayor magnitud indican los píxeles a los que la predicción es más sensible.

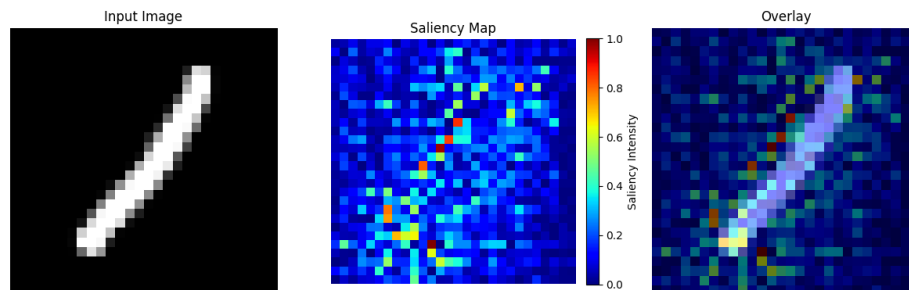


Figura 4.14: Ejemplo de mapa de saliencia generado a partir del gradiente de la salida con respecto a la imagen de entrada.

Esta técnica fue introducida por [Simonyan et al. \(2014\)](#) para redes de clasificación de imágenes, siendo uno de los primeros métodos de visualización basados en gradiente aplicados a redes convolucionales profundas.

La función `compute_saliency()` de `src/grad.py` implementa directamente la fórmula anterior. El fragmento clave consiste en activar el cálculo de gradientes sobre la imagen de entrada, propagar hacia atrás la puntuación de la clase objetivo y leer el gradiente resultante:

```

1 x = x.clone()
2   .detach()
3   .requires_grad_(True) # habilitar gradientes sobre la entrada
4
5 logits = model(x)
6 score = logits[torch.arange(len(y)), y].sum() # puntuacion de la
   clase objetivo
7
8 score.backward() # retropropagar hasta los pixeles de entrada
9 saliency = x.grad.detach().abs() # valor absoluto del gradiente
10 saliency = saliency
11   .max(dim=1)
12   .values # maximo sobre todos los canales: mapa 2D final

```

Extracto de código 4.2: Cálculo del mapa de saliencia en `compute_saliency()` (`src/grad.py`). El gradiente sobre la entrada x es la implementación directa de la fórmula $M_{ij} = \max_c |\partial S_c / \partial I_{ij}^{(c)}|$.

Sin embargo, los mapas de saliencia presentan limitaciones importantes. Al operar directamente con gradientes crudos de la red, los mapas resultantes suelen ser visualmente ruidosos y sensibles a variaciones locales de la imagen [3].

Además, en zonas donde la función de activación tiene gradiente nulo (por ejemplo, ReLU en su región no activa), los mapas pueden volverse poco informativos. Por ello, conviene interpretarlos como una medida de sensibilidad local y no como una explicación completa del modelo.

Otros métodos de visualización

Junto a los mapas de saliencia, la literatura propone otros métodos que ofrecen perspectivas complementarias sobre el comportamiento de las redes convolucionales.

Grad-CAM (del inglés *Gradient-weighted Class Activation Mapping*) [24] pondera los mapas de características de la última capa convolucional por los gradientes de la clase predicha con respecto a esos mapas, produciendo un mapa de calor localizado sobre la imagen de entrada. A diferencia del mapa de saliencia básico, Grad-CAM opera sobre activaciones de capas intermedias en lugar de sobre los píxeles de entrada, lo que proporciona mapas más suaves y menos sensibles al ruido local.

Visualización por deconvolución [4] invierte el paso hacia adelante de la red para proyectar las activaciones de una capa intermedia de vuelta al espacio de la imagen de entrada. Para ello aplica el proceso inverso de las operaciones convolucionales y de activación, permitiendo visualizar qué estructuras de la imagen original activan con mayor intensidad un determinado filtro.

Gradientes Integrados resuelve una de las limitaciones del mapa de saliencia básico: la sensibilidad al punto concreto donde se evalúa el gradiente. El método acumula los gradientes a lo largo de una trayectoria lineal desde una imagen de referencia, normalmente negra, hasta la imagen de entrada, obteniendo una atribución por píxel más estable y consistente.

Estas metodologías resultan especialmente relevantes en aplicaciones críticas, como la medicina, la conducción autónoma o los sistemas de seguridad, donde es fundamental justificar las decisiones tomadas por el modelo [20, 19, 21].

5. Experimentación

Este capítulo recorre tres preguntas que guían los experimentos realizados sobre el conjunto de datos MNIST. La primera compara el rendimiento entre una red neuronal densa y una red convolucional en condiciones ideales, estableciendo la línea base de la comparativa. La segunda indaga qué aprende internamente la red convolucional con imagen limpia: qué detectan sus filtros, cómo evoluciona la representación entre capas y qué píxeles de la imagen de entrada determinan la predicción. La tercera somete ambos modelos a ruido gaussiano progresivo para revelar qué arquitectura es más robusta, por qué lo es y por qué esa robustez no queda recogida en los mapas de saliencia.

5.1. Entorno experimental y herramientas de desarrollo

Los experimentos se desarrollaron en Windows 11, sobre un equipo con procesador AMD Ryzen 7 3700X (8 núcleos) y tarjeta gráfica NVIDIA GeForce RTX 4060 Ti. El lenguaje utilizado fue Python 3.14 en un entorno virtual.

Las librerías principales son `torch` y `torchvision` [22], para la definición, entrenamiento e inferencia de modelos; `numpy`, `matplotlib` y `scikit-learn`, para el análisis numérico, la visualización de resultados y el cálculo de métricas estadísticas. Los notebooks se gestionaron con `jupyterlab` y `jupyter`, que permite mantenerlos como scripts de texto y facilitar el control de versiones con `git`. El código fue redactado principalmente en Visual Studio Code.

5.2. Conjunto de datos: MNIST

El conjunto de datos MNIST es un benchmark ampliamente utilizado en la literatura de aprendizaje automático [11]. Contiene un total de 70.000 imágenes en escala de grises de dígitos manuscritos (del 0 al 9), de tamaño 28×28 píxeles.



Figura 5.1: Captura de ejemplos aleatorios del conjunto de entrenamiento de MNIST.

Este dataset lo segmentaremos en un subconjunto de entrenamiento, del que obtendremos otro subconjunto de validación; y un subconjunto de pruebas. Para ello vamos a utilizar PyTorch [22], una librería de Python para el desarrollo de modelos de aprendizaje profundo optimizada para implementar tensores usando la GPU y la CPU de nuestro ordenador. PyTorch tiene interfaces que nos permiten extraer datasets precargados en sus repositorios de forma muy sencilla.

```
1 from torch.utils.data import DataLoader, random_split
2 from torchvision import datasets
3
4 train_dataset = datasets.MNIST(root=datasets_path, transform=
    ToTensor(), download=True)
5 test_dataset = datasets.MNIST(root=datasets_path, train=False,
    transform=ToTensor(), download=True)
```

Extracto de código 5.1: Descarga del dataset MNIST.

En 5.1, la línea 4 descarga y transforma el conjunto de entrenamiento en tensor mediante *ToTensor()*. Luego, repetimos en la línea 5 para el conjunto de pruebas.

```
1 train_size = int(0.8 * len(train_dataset))
2 val_size = len(train_dataset) - train_size
3 train_subset, val_subset = random_split(train_dataset, [train_size,
    val_size])
4
5 batch_size = 64
6 train_dataloader = DataLoader(train_subset, batch_size=batch_size,
    shuffle=True)
7 val_dataloader = DataLoader(val_subset, batch_size=batch_size)
8 test_dataloader = DataLoader(test_dataset, batch_size=batch_size)
```

Extracto de código 5.2: Segmentación del dataset MNIST a conjunto de entrenamiento, validación y pruebas.

A continuación, en 5.2, la línea 3 divide el conjunto original de entrenamiento en dos subconjuntos: entrenamiento (80 %) y validación (20 %). Finalmente, en las líneas 6 a 8 se crean los correspondientes *Dataloaders*, que se encargan de organizar los datos en batches (a los que se le aplica un *shuffle* para el entrenamiento) en cada época, facilitando así el flujo de datos durante las fases de entrenamiento, validación y evaluación del modelo. Dejando nuestro dataset MNIST distribuido como muestra la figura 5.2.

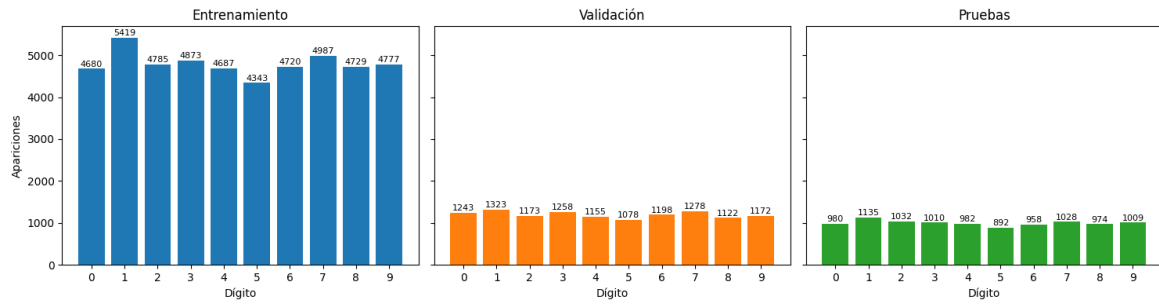


Figura 5.2: Distribución de muestras de MNIST en los conjuntos de entrenamiento, validación y pruebas

5.3. Métricas de Evaluación

Para analizar el rendimiento de los modelos utilizaremos varias métricas complementarias, ya que ninguna por sí sola cuenta toda la historia [14]. A continuación se definen las que se emplean a lo largo de este capítulo. Todas estas métricas pueden encontrarse implementadas en la base de código, concretamente en el notebook 02.metrics.py.

Exactitud (TA). Es la proporción de predicciones correctas sobre el total de ejemplos:

$$TA = \frac{VP + VN}{VP + VN + FP + FN} \quad (5.1)$$

donde *VP* son los verdaderos positivos, *VN* los verdaderos negativos, *FP* los falsos positivos y *FN* los falsos negativos. En clasificación multiclase se calcula simplemente como el número de aciertos dividido entre el total de muestras. Es intuitiva, pero puede resultar engañosa cuando las clases están muy desbalanceadas.

F1-score. Combina precisión (VPP) y sensibilidad (TVP) en una sola medida mediante la media armónica, lo que la hace más útil que la exactitud cuando queremos un equilibrio entre falsos positivos y falsos negativos:

$$VPP = \frac{VP}{VP + FP}, \quad TVP = \frac{VP}{VP + FN}, \quad F1 = 2 \cdot \frac{VPP \cdot TVP}{VPP + TVP} \quad (5.2)$$

Pérdida de entropía cruzada. Es la función de pérdida que usamos durante el entrenamiento. Mide cuánto se aleja la distribución predicha $\hat{y}$ de la distribución real y :

$$PEC = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (5.3)$$

donde C es el número de clases e y_i es la etiqueta en codificación *one-hot*. Un valor bajo indica que el modelo asigna probabilidades altas a las clases correctas [1].

Tiempo de inferencia por muestra. Tiempo medio necesario para que el modelo clasifique una única entrada. Se calcula dividiendo el tiempo total invertido en clasificar N muestras entre N . Es relevante en aplicaciones donde la latencia importa.

Matriz de confusión. Tabla que muestra cómo se distribuyen las predicciones entre las clases: las filas representan las etiquetas reales y las columnas las predicciones. Permite identificar con qué clases se confunde más el modelo.

Prueba de McNemar

La **prueba de McNemar** es un test estadístico no paramétrico que nos permite saber si dos modelos tienen un rendimiento significativamente diferente sobre el mismo conjunto de pruebas. Se define como:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (5.4)$$

donde b es el número de muestras que el modelo A falla pero el B acierta, y c el caso contrario. Si χ^2 supera el umbral crítico para un nivel de significación α dado, podemos descartar que la diferencia sea fruto del azar.

Estabilidad de los mapas de saliencia

Para cuantificar cuánto cambia el mapa de saliencia de un modelo y una imagen al añadir ruido, usaremos dos métricas. La **correlación de Spearman** (ρ) mide si el orden de importancia de los píxeles se mantiene; un valor cercano a 1 significa que los mismos píxeles siguen siendo relevantes, mientras que un valor cercano a 0 indica que la explicación ha cambiado completamente. La **distancia L2 normalizada** mide la diferencia absoluta entre los saliency maps, normalizada por la magnitud del original.

Top-k confusiones

El análisis de **top-k confusiones** lista los pares de clases que el modelo confunde con más frecuencia, calculando para cada clase verdadera su tasa de error:

$$\text{Tasa de Error}_i = \frac{\text{Número de errores de la clase } i}{\text{Total de ejemplos de la clase } i} \quad (5.5)$$

Esto ayuda a entender qué dígitos son más difíciles de distinguir para el modelo.

5.4. Implementación de los modelos

En esta sección se describen los dos modelos que protagonizan los experimentos. El primero es un MLP, una red neuronal densa que actúa como línea base; el segundo es la red convolucional cuyas representaciones internas se analizarán a lo largo del capítulo.

5.4.1. Red Neuronal Densa (FCNet)

Como punto de partida se implementó una red neuronal densa (FCN, del inglés *Fully Connected Network*) denominada FCNet que actuará como línea base de la experimentación. No incorpora ninguna operación espacial, simplemente aplanar cada imagen en un vector y la procesa con capas completamente conectadas. Esto nos permite comparar directamente qué aportan las capas convolucionales frente a una arquitectura que no las tiene.

El primer paso es aplanar la imagen de 28×28 píxeles en un vector de 784 componentes, ya que las capas densas esperan entradas unidimensionales. A partir de ahí, la red consta de dos capas ocultas de 512 neuronas con activación ReLU. Termina con una capa de salida de 10 neuronas (una por clase de dígito). El código completo de FCNet puede consultarse en el Anexo [A](#).

Capa	Dimensión de salida	Parámetros
Aplanado (<i>Flatten</i>)	784	0
Capa densa 1 + ReLU	512	401.920
Capa densa 2 + ReLU	512	262.656
Capa de salida (logits)	10	5.130
Total		669.706

Tabla 5.1: Arquitectura y parámetros de FCNet.

5.4.2. Red Neuronal Convolucional (ConvNet)

Para aprovechar la estructura espacial de las imágenes se implementó el modelo ConvNet, una red neuronal convolucional diseñada para imágenes en escala de grises

de 28×28 píxeles. A diferencia del MLP, esta arquitectura no aplanar la imagen de entrada, la procesa directamente con filtros que se deslizan sobre ella, detectando patrones locales en cada posición.

El bloque convolucional está formado por dos etapas de convolución y *max pooling*. La primera aplica 32 filtros de 3×3 (sin padding), reduciendo la resolución de 28×28 a 13×13 tras el pooling. La segunda aplica 64 filtros de 3×3 , volviendo a reducir hasta 5×5 . Al final del bloque convolucional, el mapa de características se aplanar en un vector de $64 \times 5 \times 5 = 1,600$ componentes, que alimenta un clasificador completamente conexo de tres capas.

Capa	Dimensión de salida	Parámetros
Entrada	$1 \times 28 \times 28$	0
Conv1 (3×3 , 32 filtros) + ReLU	$32 \times 26 \times 26$	320
MaxPool (2×2)	$32 \times 13 \times 13$	0
Conv2 (3×3 , 64 filtros) + ReLU	$64 \times 11 \times 11$	18.496
MaxPool (2×2)	$64 \times 5 \times 5$	0
Aplanado (<i>Flatten</i>)	1.600	0
Capa densa 1 + ReLU	120	192.120
Capa densa 2 + ReLU	84	10.164
Capa de salida (logits)	10	850
Total		221.950

Tabla 5.2: Arquitectura y parámetros de ConvNet.

Un aspecto relevante es que ConvNet tiene menos de la mitad de parámetros que FCNet y aun así obtiene mejor rendimiento gracias a que las capas convolucionales extraen características espaciales que las capas densas no pueden capturar directamente. Además, el diseño incluye métodos para recuperar las activaciones de las capas intermedias, lo que resulta clave para los experimentos de visualización e interpretabilidad que se presentan en secciones posteriores. El código completo de ConvNet puede consultarse en el Anexo A.

5.5. Comparativa FCNet vs ConvNet

El propósito de esta sección es establecer una línea base de rendimiento entre ambas arquitecturas en condiciones ideales, es decir, con imágenes limpias sin perturbación. Esta comparativa es necesaria antes de someter los modelos a condiciones adversas. Si una arquitectura no supera a la otra en condiciones normales, cualquier diferencia observada bajo ruido podría explicarse simplemente por un peor ajuste general, no por la arquitectura en sí. Una vez entrenados ambos modelos, se evaluaron sobre el conjunto de pruebas de MNIST empleando las métricas definidas en la sección anterior. Los resultados se obtuvieron con el notebook 02_metrics.py.

Entrenamiento de los modelos

Ambos modelos se entrenaron con la misma configuración de hiperparámetros para garantizar que la comparativa refleje únicamente las diferencias arquitectónicas y no las del proceso de entrenamiento. Se empleó el optimizador **Adam** con una tasa de aprendizaje de $lr = 10^{-3}$. Adam mantiene estimaciones adaptativas de la media y la varianza de los gradientes pasados, lo que acelera la convergencia y reduce la necesidad de ajustar manualmente la tasa de aprendizaje a lo largo del entrenamiento [2].

La función de pérdida empleada es la **entropía cruzada categórica** (`nn.CrossEntropyLoss`), que combina internamente una transformación *softmax* con la pérdida de log-verosimilitud negativa. Esta formulación es la adecuada para clasificación multiclase cuando el modelo produce logits sin activación explícita en la capa de salida [1].

El proceso de entrenamiento se prolongó durante **10 épocas** con un tamaño de *batch* de **64** muestras. El estado del modelo se guarda automáticamente como punto de control tras cada época.

En cada época, el bucle de entrenamiento itera sobre los minibatches del conjunto de entrenamiento. Para cada uno realiza cinco operaciones en orden secuencial. Primero, el paso hacia adelante produce los logits del modelo. Segundo, la pérdida de entropía cruzada se calcula sobre esos logits. Tercero, los gradientes acumulados de la iteración anterior se vacían. Cuarto, la retropropagación propaga el gradiente de la pérdida hacia atrás a través de todos los parámetros. Quinto, Adam actualiza los pesos usando los gradientes recién calculados. Al finalizar todos los minibatches de la época, el modelo se evalúa en modo inferencia sobre el conjunto de validación sin calcular gradientes, registrando la pérdida de validación que permite monitorizar el sobreajuste.

```
1 # Configuración del entrenamiento
2 optimizer = optim.Adam(modelo.parameters(), lr=1e-3)
3 loss_fn = nn.CrossEntropyLoss()
4
5 trainer = Trainer(
6     model=modelo,
7     optimizer=optimizer,
8     loss_fn=loss_fn,
9     train_loader=train_dataloader, # 80 % del conjunto original
10    val_loader=val_dataloader,      # 20 % para validación
11    device=device,
12 )
13 trainer.fit() # 10 épocas, batch size 64
14
15 # Pasos del entrenamiento
16 preds = self.model(x) # (1) paso hacia adelante
17 loss = self.loss_fn(preds, y) # (2) calcular pérdida
18
19 self.optimizer.zero_grad(set_to_none=True) # (3) limpiar gradientes
    anteriores
```



```

20 loss.backward() # (4) retropropagacion
21 self.optimizer.step() # (5) actualizar pesos con Adam

```

Extracto de código 5.3: Configuración del entrenamiento y pasos del bucle principal (notebooks/03_saliency_map.py, src/models.py).

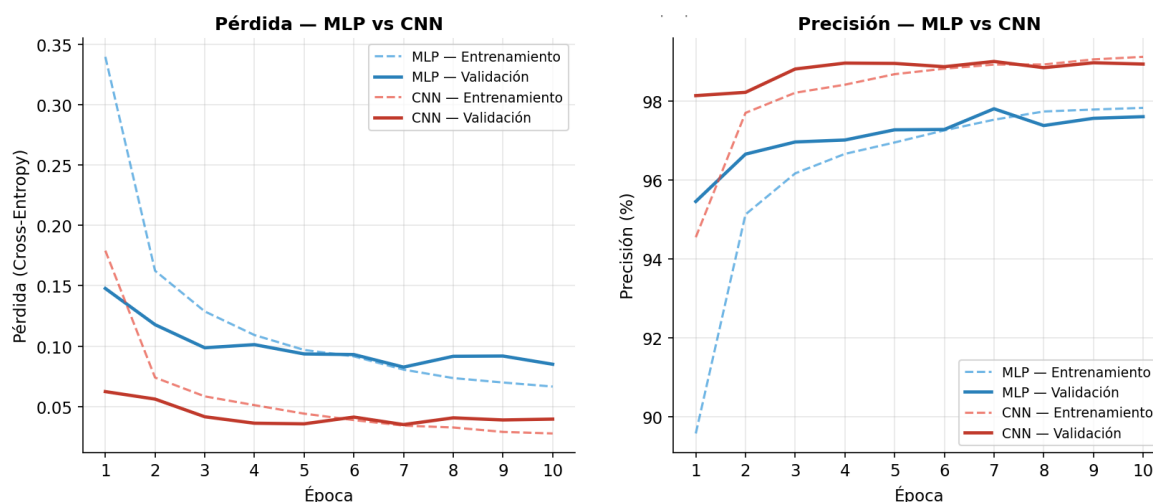


Figura 5.3: Curvas de aprendizaje comparativas de FCNet y ConvNet. Evolución de la pérdida (izquierda) y la precisión (derecha) en entrenamiento y validación a lo largo de las épocas.

Exactitud y F1-score. La ConvNet obtuvo una exactitud superior a la FCNet con un número de parámetros significativamente menor, lo que pone de manifiesto la mayor eficiencia de las representaciones convolucionales para datos con estructura espacial. La tabla 5.3 resume los resultados sobre el conjunto de pruebas.

Modelo	Exactitud (%)	F1 (macro)	PEC	Inf. (ms/muestra)
FCNet	0.9796	0.9794	0.0871	0.0715
ConvNet	0.9902	0.9901	0.0369	0.0740

Tabla 5.3: Comparativa de rendimiento de FCNet y ConvNet sobre el conjunto de pruebas de MNIST en condiciones de imagen limpia.

Tiempo de inferencia. Los modelos convolucionales cuentan con una clara ventaja debido a que cuentan con la aceleración por unidades de procesamiento gráfico (GPU, del inglés *Graphics Processing Unit*), donde las operaciones de convolución están especialmente optimizadas. En este caso, el tiempo de inferencia ha sido prácticamente el mismo por muestra en ambos modelos. Esto se debe, en gran medida, a que la complejidad tanto de las arquitecturas, como del conjunto de datos es relativamente baja.

Matriz de confusión. En ambos modelos, los principales errores se concentran en pares de dígitos visualmente similares: {3, 8}, {4, 9} y {5, 6}. Estos errores son similares a los que podría cometer un ser humano con caligrafía borrosa o

descuidada, lo que nos dice que los modelos han aprendido buenas representaciones de los dígitos.

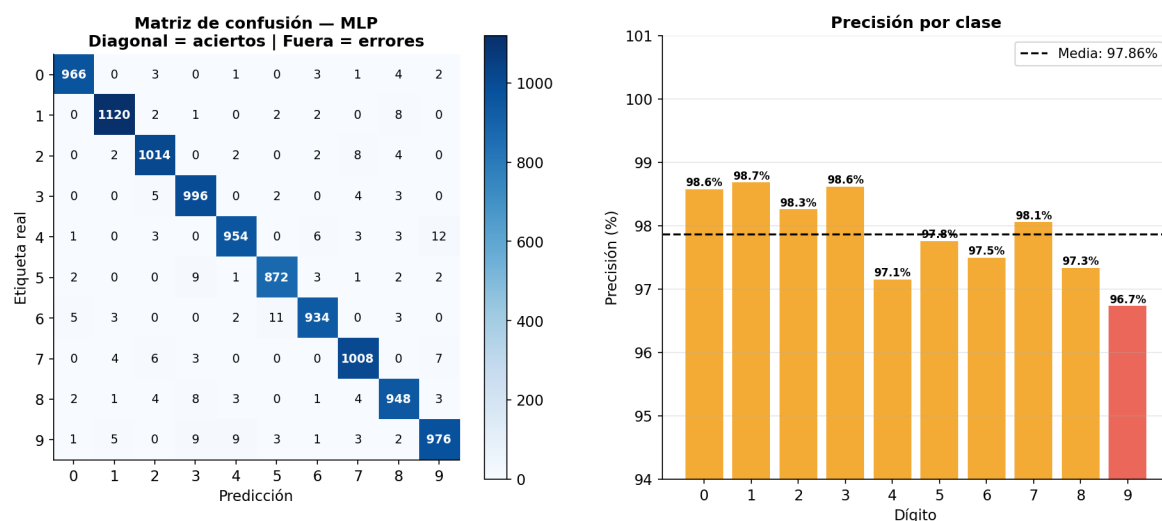


Figura 5.4: Matriz de confusión de ConvNet sobre el conjunto de pruebas (izquierda) y precisión por clase (derecha). El dígito 9 ostenta la peor precisión de predicción.

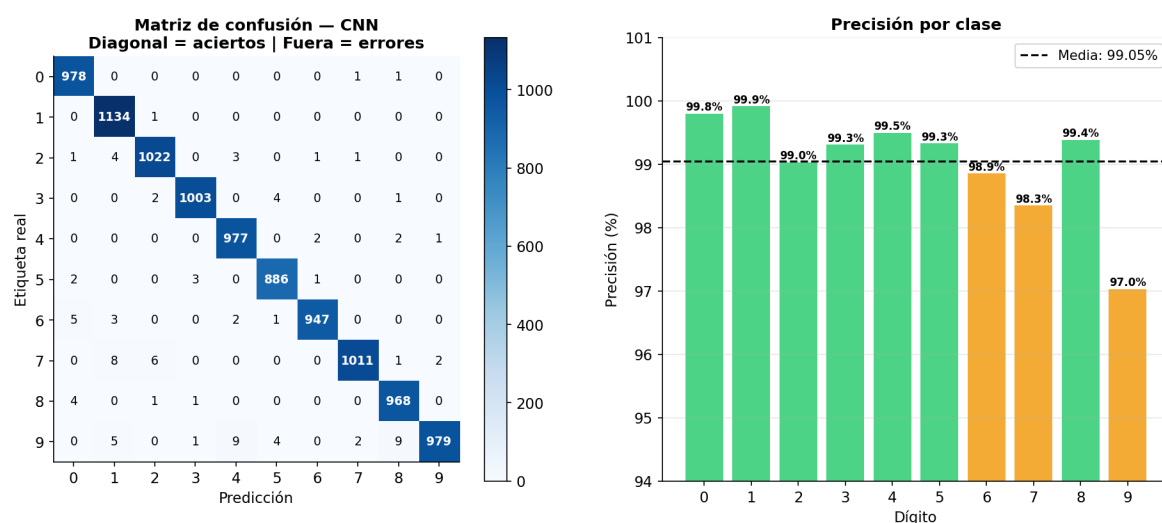


Figura 5.5: Matriz de confusión de ConvNet sobre el conjunto de pruebas (izquierda) y precisión por clase (derecha). Los dígitos 6, 7 y 9 presentan la mayor tasa de error, por debajo del 99 %.

En la figura 5.6 podemos ver los dígitos que han causado mayor nivel de confusión al modelo. La fila etiquetada como "Bloque 1" hace referencia a los filtros con más activaciones en la primera capa de convolución. Mientras que la fila etiquetada con "Bloque 2" se refiere a la segunda capa convolucional. Por último, podemos ver la distribución de la predicción en cada caso.

Los dígitos ambiguos, escritura muy irregular, trazos poco definidos, activan varios filtros de forma similar para dos clases distintas, lo que lleva a la red a

confundirse entre ellas. Las probabilidades en estos casos suelen estar repartidas entre dos clases similares, por ejemplo, 4 y 9. Los mapas del Bloque 1 muestran qué características locales ha detectado la red: bordes, curvas, regiones de alta intensidad. En los errores, a menudo estas características son compatibles con más de un dígito. Por otra lado, los mapas del Bloque 2 son más abstractos y difíciles de interpretar directamente, pero reflejan cómo la red combina las características del Bloque 1 para formar representaciones de alto nivel.

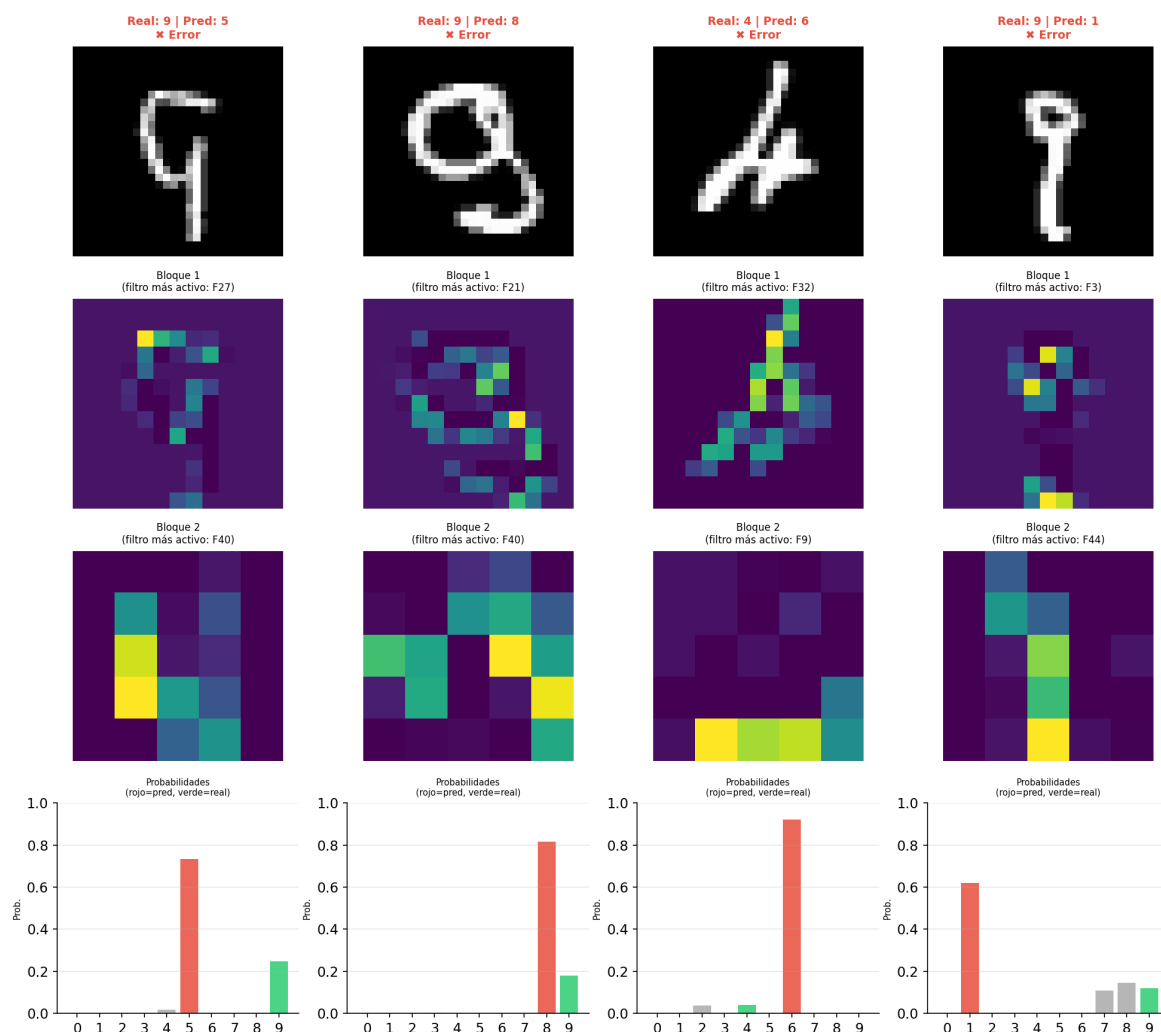


Figura 5.6: Comparación de los pares más confundidos en la CNN.

Que la CNN cometa errores en imágenes que también resultarían ambiguas para un ser humano es, en cierto modo, una señal de que el modelo ha aprendido representaciones genuinamente útiles, no simples atajos estadísticos.

Prueba de McNemar. Confirma que las diferencias de rendimiento entre FCNet y ConvNet son estadísticamente significativas con χ^2 superior al umbral crítico. Este resultado valida que la diferencia observada no es debida al azar [14].

Establecida la superioridad de la ConvNet en condiciones ideales, surge la pregunta de qué aprende internamente esta red para lograr esa eficiencia con

menos parámetros. La siguiente sección analiza las representaciones que construye capa a capa sobre una imagen limpia, antes de someter ambos modelos a condiciones adversas.

5.6. Análisis de representaciones internas

Antes de someter los modelos a condiciones adversas, conviene entender qué representaciones construye la ConvNet cuando la entrada es una imagen limpia. Este análisis se apoya en tres herramientas complementarias. Los mapas de características responden a la pregunta de qué patrones detectan los filtros de cada capa convolucional. La visualización de activaciones por capa muestra cómo evoluciona la representación de una misma imagen al propagarse de Conv1 a Conv2. Los mapas de saliencia identifican qué píxeles de la imagen de entrada han determinado la predicción final. Las tres perspectivas son complementarias: la primera mira hacia adentro de la red capa a capa, la tercera mira hacia la entrada, y la segunda conecta ambas visiones mostrando la transición entre niveles de abstracción.

5.6.1. Mapas de características

Se han registrado los mapas de activación de las capas Conv1 y Conv2 mediante ganchos de propagación hacia adelante (*forward hooks*) de PyTorch. Un gancho es una función que se ejecuta automáticamente cada vez que el módulo al que está asociado completa su paso hacia adelante, permitiendo capturar la salida de cualquier capa sin modificar el modelo.

```
1 capturas = {}
2
3 def crear_gancho(nombre):
4     def _gancho(modulo, entrada, salida):
5         capturas[nombre] = salida.detach().cpu() # guardar mapa de
6             activacion
7     return _gancho
8
9 # Registrar ganchos en Conv1 y Conv2
10 h1 = modelo.conv1.register_forward_hook(crear_gancho("conv1"))
11 h2 = modelo.conv2.register_forward_hook(crear_gancho("conv2"))
12
13 with torch.no_grad():
14     _ = modelo(imagen.unsqueeze(0)) # paso hacia adelante
15
16 h1.remove() # eliminar ganchos tras usarlos
17 h2.remove()
```

Extracto de código 5.4: Extracción de mapas de activación con ganchos de PyTorch (src/models.py, notebooks/05_examples.py).

Los filtros de Conv1 producen 32 mapas de tamaño 26×26 , cada uno con la respuesta de un filtro de 3×3 ante toda la imagen. En esta primera capa, los patrones que provocan activaciones altas son locales y simples, por ej. trazos verticales, horizontales y diagonales, así como transiciones abruptas de intensidad correspondientes a los bordes del dígito. En otras palabras, las capas iniciales actúan como detectores de características elementales.

Los filtros de Conv2 producen 64 mapas de tamaño 11×11 aplicados sobre la salida ya procesada de Conv1. Al combinar la información de múltiples detectores de bordes, esta capa responde a estructuras más complejas: curvas, esquinas y combinaciones de trazos que distinguen entre dígitos morfológicamente similares, como el 4 y el 9, o el 3 y el 8.

En conjunto, los mapas de características muestran que la ConvNet construye una jerarquía de representaciones, desde características locales de bajo nivel en Conv1, hasta representaciones más abstractas en Conv2. La siguiente subsección examina esta jerarquía comparando las activaciones de ambas capas ante la misma imagen.

5.6.2. Visualización de activaciones por capa

La visualización por capa consiste en obtener, para la misma imagen de entrada, la salida completa de cada capa en orden secuencial y observar la transición entre sus representaciones. A diferencia de comparar filtros individuales, este análisis muestra la evolución global de la información a lo largo de la red.

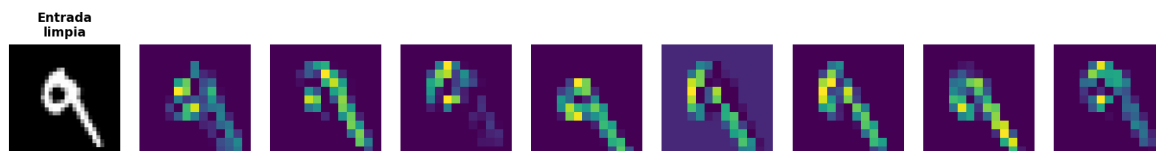


Figura 5.7: Mapas de activación para los 8 filtros más activos de *Conv1* y *Conv2* sobre el número nueve.

Los mapas de Conv1 conservan una alta similitud estructural con la imagen de entrada: muchos filtros producen activaciones que recuerdan a versiones filtradas del dígito original, resaltando contornos o regiones de alta variación local. La dimensión espacial (26×26) es todavía próxima a la original (28×28), por lo que la relación entre activaciones y regiones de la imagen es fácil de interpretar y relacionar [2].

Por otro lado, los mapas de Conv2 presentan un aspecto diferente. Al ser aplicados sobre la salida ya procesada de Conv1, su dimensión espacial ha descendido a 11×11 tras el pooling, y las activaciones son más dispersas y abstractas. Este comportamiento refleja la especialización progresiva que caracteriza a las redes convolucionales [4, 2].

Finalmente, se comprobará mediante los mapas de saliencia cuáles de esas representaciones determinan la predicción final del modelo.

5.6.3. Mapas de saliencia

Los mapas de características muestran qué detecta la red en sus capas internas, pero no son capaces de determinar qué píxeles de la imagen de entrada causaron una predicción concreta. Los mapas de saliencia cubren esa perspectiva al proyectar la sensibilidad de la predicción de vuelta a la imagen original mediante el gradiente.

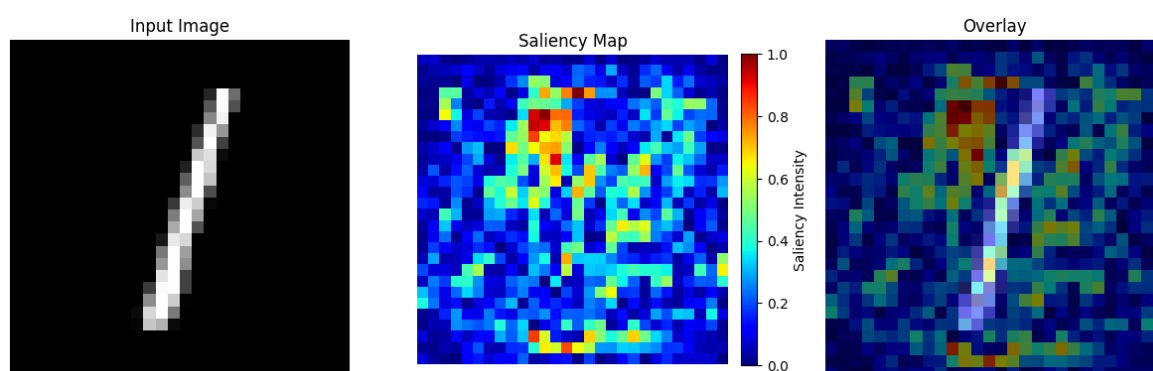


Figura 5.8: Mapa de saliencia del dígito uno sobre un MLP.

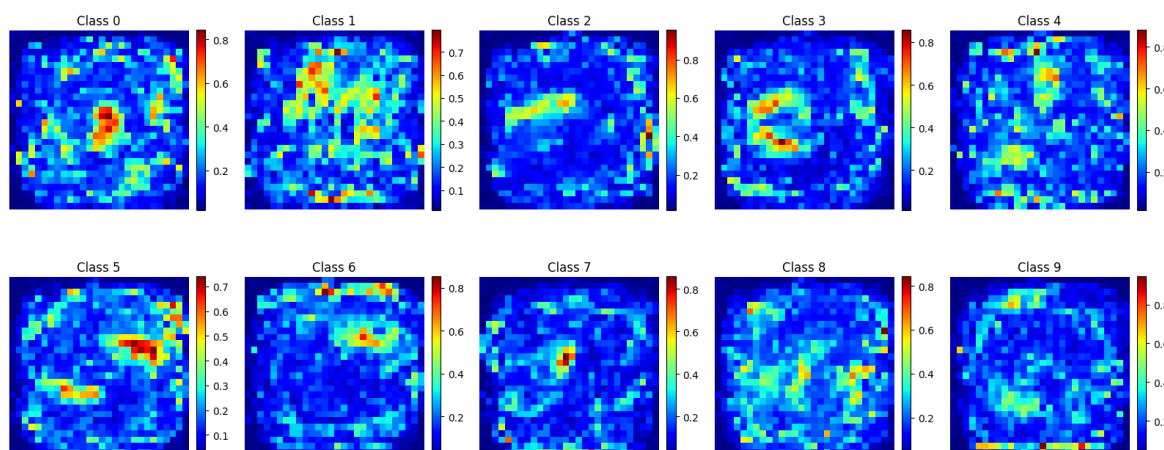


Figura 5.9: Mapa de saliencia promediado de cada uno de los dígitos de MNIST sobre un MLP.

En condiciones de imagen limpia, los mapas de saliencia de la ConvNet tienden a concentrar los valores altos sobre las regiones del trazo del dígito, especialmente en las zonas que definen su forma característica. Mientras que los mapas de la FCNet, aunque también destacan la región del dígito, tienen una distribución algo más difusa, consistente con el hecho de que todas las neuronas ocultas reciben simultáneamente toda la imagen aplanada. Comparativamente, los mapas de la CNN resultan más focalizados y menos ruidosos sobre la imagen limpia, lo cual

refleja que las capas convolucionales han aprendido a aislar las regiones relevantes antes de que la información llegue al clasificador denso [3, 2].

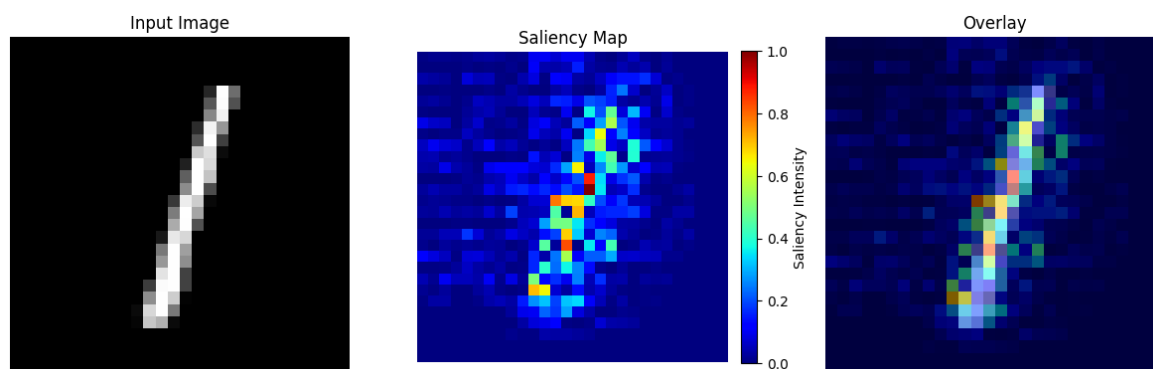


Figura 5.10: Mapa de saliencia del dígito uno sobre una CNN.

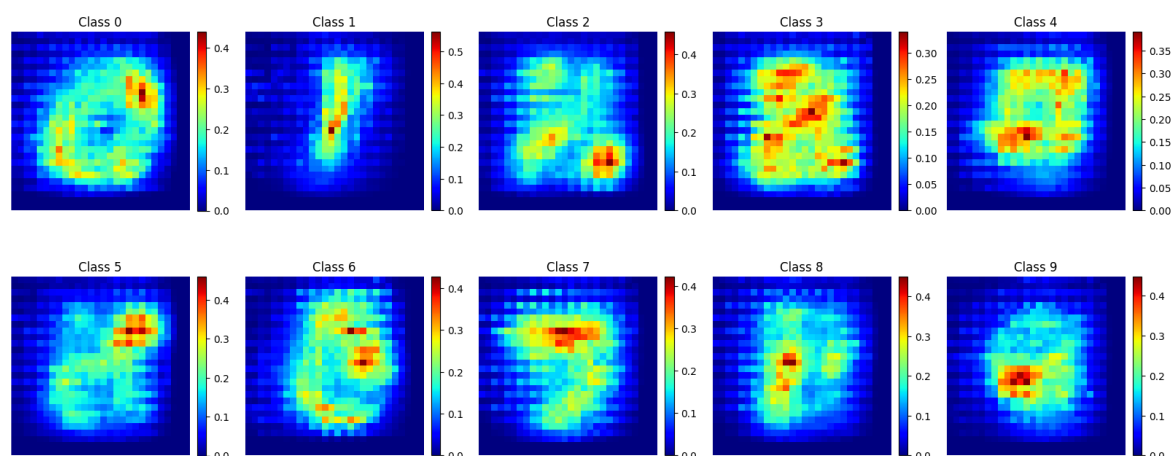


Figura 5.11: Mapa de saliencia promediado de cada uno de los dígitos de MNIST sobre un MLP.

El análisis con imagen limpia ha mostrado que la ConvNet construye representaciones jerárquicas y que sus mapas de saliencia señalan con razonable precisión las zonas del dígito relevantes para la predicción (fig. 5.11). Por otro lado, en el mapa de saliencia generado por *FCNet* sobre el dígito uno (fig. 5.8) no se puede decir que exista una relación clara entre los píxeles de la imagen de entrada y la predicción. No es sino promediando los mapas de saliencia que se obtiene un contorno que permite discernir aproximadamente cada dígito.

5.7. Análisis bajo ruido gaussiano

Habiendo entendido cómo funciona cada modelo con imagen limpia, el objetivo de esta sección es someter ambas arquitecturas a condiciones adversas para revelar cuál de los modelos es más robusto y por qué. La perturbación en las

imágenes del conjunto de pruebas consiste en añadir ruido gaussiano con una intensidad creciente. Se emplea una semilla aleatoria fija en todos los experimentos para garantizar que ambos modelos se evalúan sobre las mismas perturbaciones.

```
1 torch.manual_seed(semilla) # semilla fija para reproducibilidad
2 noise = torch.randn_like(x) * sigma # ruido por pixel
3 noisy = (x + noise).clamp(0, 1) # recortar al rango valido [0, 1]
```

Extracto de código 5.5: (notebooks/03_saliency_map.py).]Aplicacion de ruido gaussiano con semilla fija y recorte al rango [0,1] (notebooks/03_saliency_map.py).

$$\tilde{x} = \text{clip}(x + \epsilon, 0, 1), \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (5.6)$$

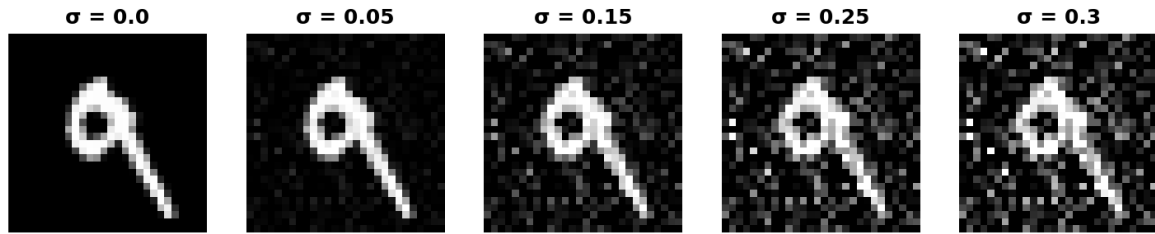


Figura 5.12: Efecto del ruido gaussiano sobre un dígito del conjunto de pruebas para valores de σ crecientes, desde la imagen original ($\sigma = 0,0$) hasta una perturbación severa ($\sigma = 0,3$).

Se evaluaron ambos modelos para $\sigma \in \{0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$. Los experimentos se encuentran en los notebooks 03_saliency_map.py y 05_examples.py.

5.7.1. Resultados de exactitud bajo ruido

La tabla 5.4 muestra la exactitud de ambos modelos para cada nivel de σ , junto con la diferencia en puntos porcentuales. En ella podemos ver qué modelo empeora su precisión conforme el ruido crece en las imágenes.

σ	FCNet (%)	ConvNet (%)	Diferencia (pp)
0.00	97.96 %	99.02 %	1.06 %
0.05	97.85 %	98.99 %	1.14 %
0.10	97.22 %	98.96 %	1.74 %
0.15	95.92 %	98.87 %	2.95 %
0.20	92.57 %	98.62 %	6.05 %
0.25	86.96 %	98.19 %	11.23 %
0.30	75.58 %	97.98 %	22.44 %

Tabla 5.4: Exactitud de FCNet y ConvNet bajo ruido gaussiano para cada nivel de σ .

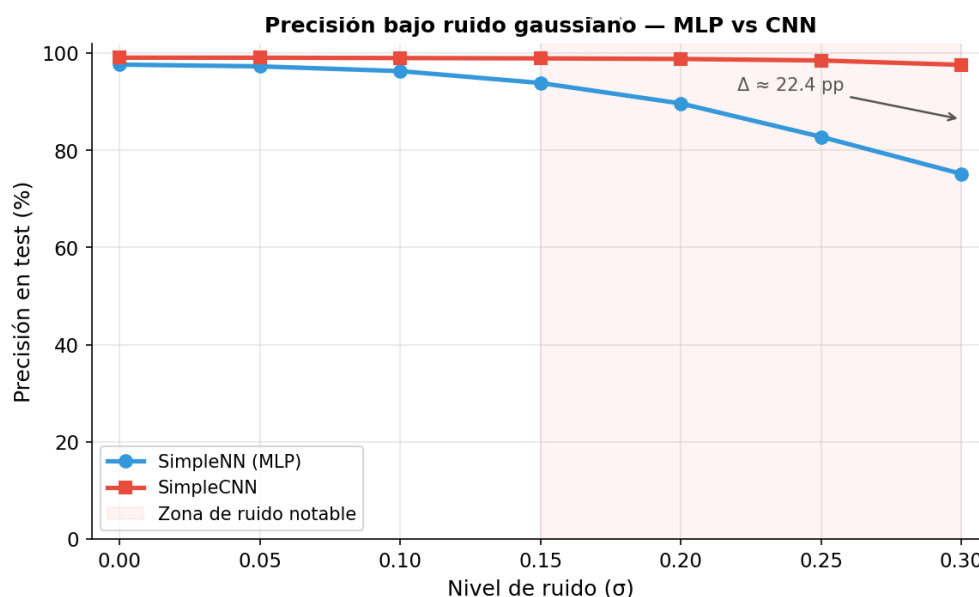


Figura 5.13: Exactitud de FCNet y ConvNet en función del nivel de ruido gaussiano σ . La diferencia entre ambos modelos supera los 22 puntos porcentuales a $\sigma = 0.30$.

La exactitud de la FCNet cae de forma pronunciada al aumentar σ , mientras que la ConvNet mantiene una exactitud significativamente estable en todo el rango evaluado. Con $\sigma = 0.30$ la brecha entre ambos modelos supera los 20 puntos porcentuales. Esta diferencia no se observa en condiciones de imagen limpia ($\sigma = 0.00$), lo que indica que la robustez al ruido es una propiedad de la arquitectura convolucional, no una consecuencia de un mejor ajuste general.

La degradación de la FCNet comienza a ser apreciable a partir de $\sigma \approx 0.05$, mientras que la ConvNet mantiene una exactitud elevada hasta $\sigma \approx 0.15$. La brecha máxima se alcanza en $\sigma = 0.30$.

5.7.2. Robustez de la CNN frente al ruido

Los resultados anteriores muestran que la ConvNet es más robusta ante el ruido en las imágenes. Hay tres características arquitectónicas que actúan de forma conjunta y son responsables de esta diferencia [1, 16].

1. Filtros locales. Cada filtro convolucional procesa únicamente una pequeña ventana de la imagen (en este modelo, 3×3 píxeles). El ruido gaussiano añade perturbaciones independientes a cada píxel. Cuando un filtro detecta un borde calcula una suma ponderada local de esos 9 píxeles: las perturbaciones en píxeles vecinos tienden a cancelarse mutuamente al promediarse, y la señal estructural del borde domina sobre ellas mientras el nivel de ruido sea moderado.

En la FCNet, por el contrario, cada neurona oculta recibe los 784 píxeles de la imagen aplanada con pesos distintos. El ruido en cualquier píxel, aunque sea un píxel del fondo sin información relevante, se propaga directamente al valor de

activación de esa neurona sin ningún promediado espacial. Esto hace que el ruido ensucie los cálculos internos del MLP de forma mucho más grave.

2. MaxPooling como supresor de ruido. La operación de max pooling selecciona el máximo de una ventana 2×2 . Si la activación más alta de esa ventana corresponde a una estructura real del dígito (un borde, una curva), el ruido en los otros tres píxeles de la ventana es irrelevante para el valor que sale del pooling, porque el máximo sigue siendo la señal verdadera. Esta propiedad actúa como un filtro de ruido tras cada bloque convolucional [16].

El MLP no aplica ninguna operación análoga. Todas las activaciones individuales, incluso las más afectadas por el ruido, se transmiten a la siguiente capa densa con sus pesos originales.

3. Jerarquía de abstracción. La primera capa convolucional aprende detectores de bordes y texturas locales cuya respuesta es robusta, es decir, si el trazo del dígito domina en la ventana de 3×3 , el filtro lo detecta con independencia del ruido en los píxeles vecinos. La segunda capa combina esos detectores para construir representaciones más abstractas. O lo que es lo mismo, el ruido que no fue eliminado en la primera capa se diluye aún más en la segunda. Al llegar al clasificador, la representación del dígito está considerablemente más limpia de lo que estaba en los píxeles originales [4, 2].

El MLP no dispone de esta cadena de filtrado, las únicas capas de procesamiento entre la entrada y la salida son las capas densas, que no tienen mecanismo alguno para suprimir el ruido antes de formar sus representaciones internas.

La figura 5.14 ilustra los tres mecanismos de forma conjunta. El MLP conecta todos los píxeles a cada neurona (izquierda), mientras que los filtros convolucionales solo procesan una ventana local de 3×3 (centro). El MaxPooling selecciona el máximo de una ventana 2×2 , de modo que la señal real sobrevive aunque los píxeles vecinos contengan ruido (derecha).

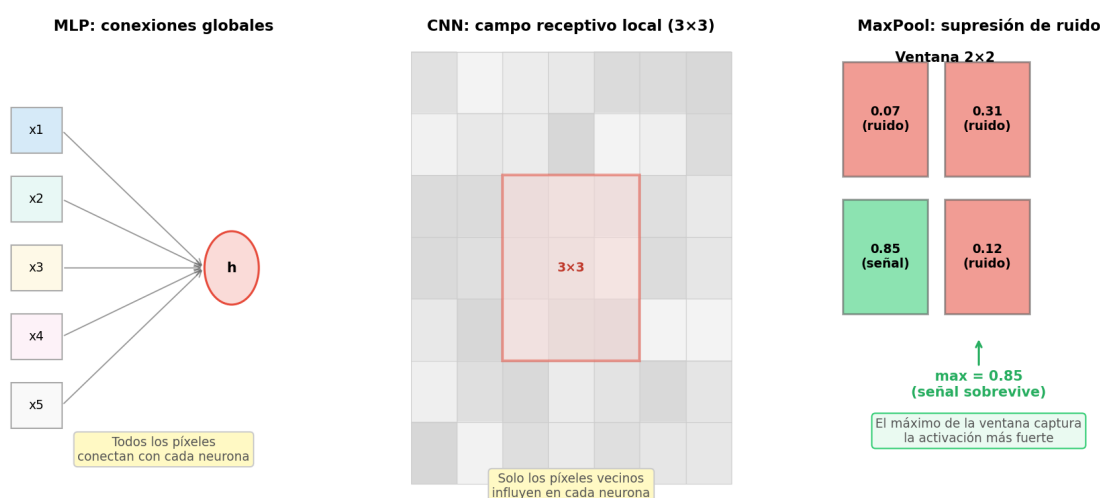


Figura 5.14: Ilustración de los mecanismos de robustez de la CNN.

5.7.3. Fragilidad de los mapas de saliencia bajo ruido

Como hemos visto en secciones anteriores, el mapa de saliencia es una herramienta capaz de contarnos la relevancia de los píxeles para el modelo. Si introducimos ruido a las imágenes, haría falta ver qué repercusión tiene esta modificación en los mapas de saliencia de los modelos.

Para cuantificar cuánto cambia el mapa de saliencia al añadir ruido, se calcularon dos métricas para cada nivel de σ , con 20 repeticiones con semillas distintas:

- **Correlación de Spearman (ρ):** mide si el *orden de importancia* de los píxeles se conserva. Un valor cercano a 1 indica que los mismos píxeles siguen siendo relevantes; cercano a 0 indica que la explicación ha cambiado completamente.
- **Distancia L2 normalizada:** mide la diferencia absoluta entre el mapa original y el ruidoso, normalizada por la magnitud del original.

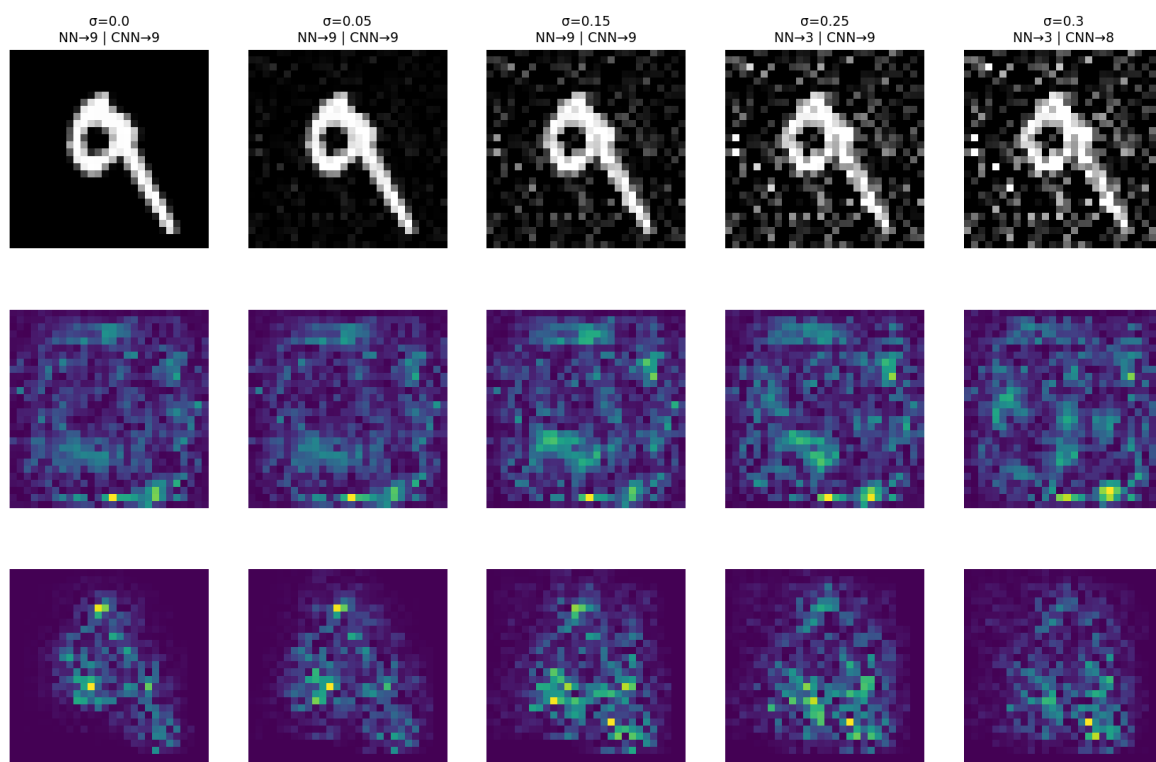


Figura 5.15: Saliency maps de FCNet (fila central) y ConvNet (fila inferior) sobre el mismo dígito a distintos niveles de ruido σ .

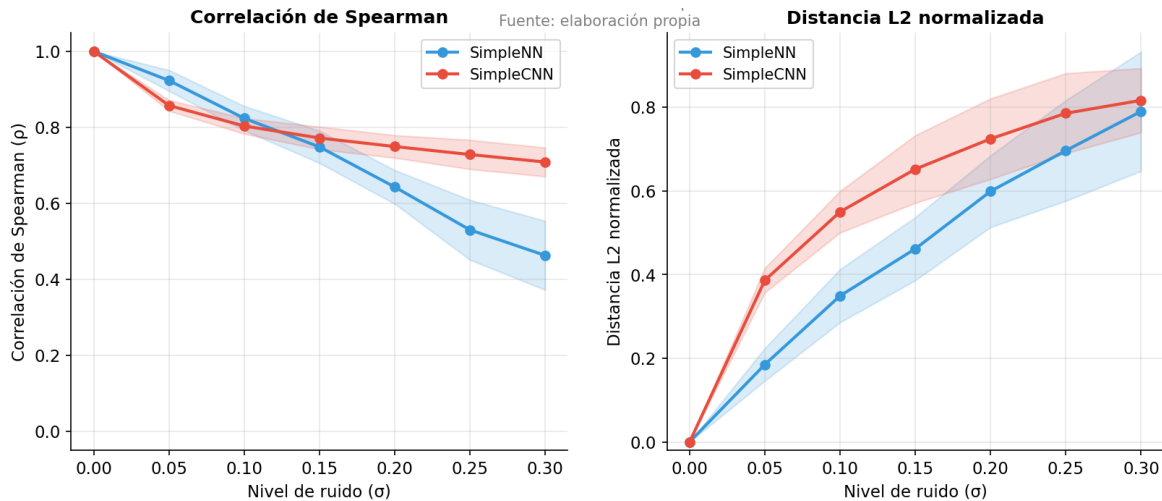


Figura 5.16: Estabilidad del saliency map ante ruido gaussiano: correlación de Spearman (izquierda) y distancia L2 normalizada (derecha), promediadas sobre 20 repeticiones por nivel de σ .

La correlación de Spearman del saliency de la FCNet cae rápidamente con el nivel de ruido, lo que quiere decir que los píxeles de relevancia en el mapa de saliencia no son siempre los mismos. La ConvNet muestra mayor estabilidad, pero también experimenta una degradación apreciable a niveles de $\sigma \geq 0.15$.

Si la estabilidad de la clasificación dependiera de que la CNN utilice siempre los mismos píxeles para discriminar una clase u otra, cabría esperar que los mapas de saliencia de la CNN permanecieran estables. El modelo sigue acertando, luego debería seguir mirando los mismos sitios. Sin embargo, los mapas de saliencia cambian considerablemente con el ruido, lo que indica que la CNN no basa su robustez en píxeles concretos sino en algo más profundo, las representaciones que construye internamente capa a capa.

5.7.4. Persistencia de las representaciones internas bajo ruido

Para contrastar el comportamiento del mapa de saliencia con lo que ocurre realmente dentro de la red, se visualizaron los mapas de activación de las capas convolucionales bajo ruido y se midió su similitud con los de la imagen limpia. Los mapas de activación se obtienen registrando la salida de cada filtro convolucional ante una imagen de entrada. A diferencia del mapa de saliencia, que trabaja en el espacio de los píxeles de entrada, los mapas de características revelan qué estructuras intermedias ha detectado la red.

El resultado fue que los mapas de características de Conv1 mantienen una similitud alta con los de la imagen limpia incluso para niveles de ruido moderados ($\sigma \leq 0.15$). Los patrones detectados por los filtros (bordes, contornos, regiones de alta intensidad) persisten a pesar de las perturbaciones en los píxeles originales. En Conv2, la similitud también se mantiene, aunque empeora al tratarse de representaciones más abstractas.

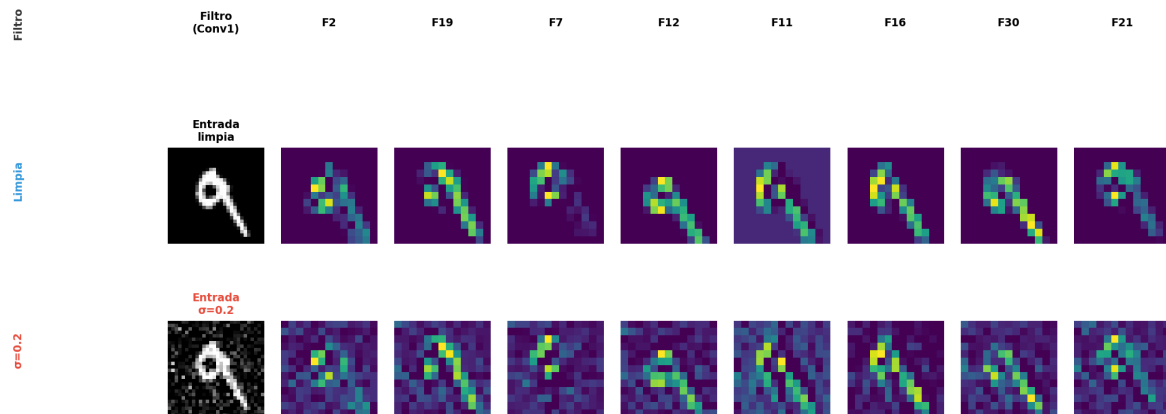


Figura 5.17: Mapas de activación de los 8 filtros más activos de Conv1 para la imagen limpia (fila superior) y con ruido $\sigma = 0.2$ (fila inferior). Los patrones estructurales detectados se mantienen pese a las perturbaciones en los píxeles de entrada.

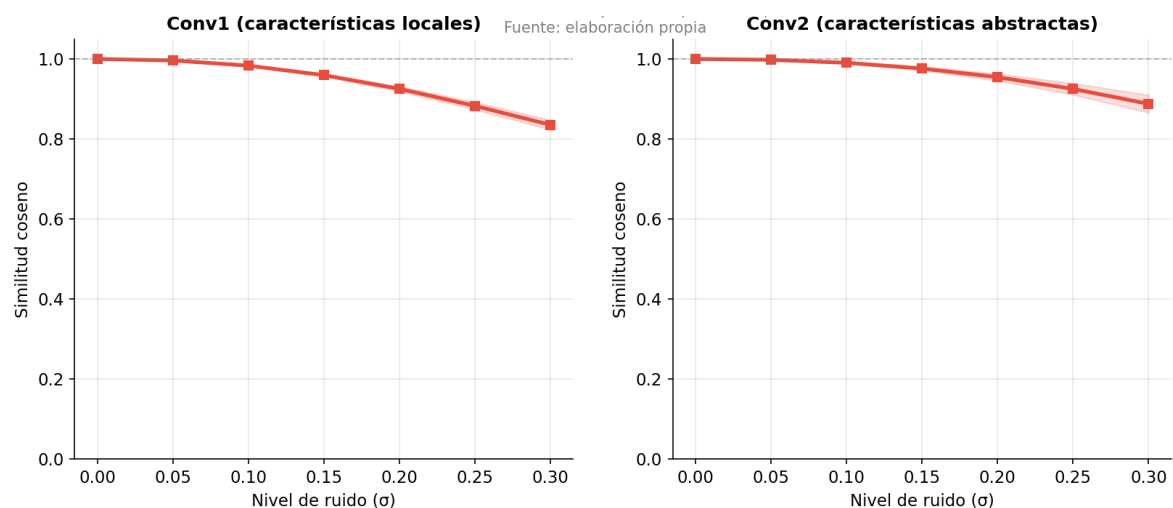


Figura 5.18: Similitud entre los mapas de activación de la imagen limpia y la ruidosa en Conv1 (características locales) y Conv2 (características abstractas), promediada sobre 20 repeticiones. Ambas capas mantienen valores superiores a 0.8 hasta $\sigma = 0.25$.

Por tanto, podemos concluir que los filtros locales de Conv1 promedian el ruido de los píxeles dentro de cada ventana de 3×3 y el MaxPooling posterior elimina las perturbaciones residuales; el resultado es que los bordes y contornos del dígito se detectan de forma casi idéntica con imagen limpia y con la imagen ruidosa. En Conv2 el ruido que sobrevivió a la primera capa vuelve a diluirse al combinarse múltiples detectores de Conv1. Cuando la representación llega al clasificador final, la jerarquía de abstracción ha suprimido la mayor parte de la perturbación, de modo que la predicción se mantiene correcta aunque los píxeles originales hayan cambiado. El mapa de saliencia no captura este fenómeno porque trabaja con gradientes sobre los píxeles de entrada, no sobre las representaciones intermedias, productos de las convoluciones, donde reside la robustez del modelo al ruido.

6. Conclusiones y Líneas Futuras

Este trabajo ha recorrido el camino que va desde los fundamentos de la neurona artificial hasta el análisis experimental del comportamiento interno de las redes convolucionales. El objetivo no ha sido únicamente medir el rendimiento de los modelos, sino entender por qué funcionan como funcionan. Los experimentos sobre clasificación de dígitos MNIST con perturbaciones gaussianas nos han permitido llegar a cuatro conclusiones, que van desde la ventaja en eficiencia y precisión de las representaciones convolucionales frente a las densas, pasando por la robustez de la CNN ante perturbaciones en la imagen y la estabilidad de sus representaciones intermedias bajo ruido, hasta los límites de los mapas de saliencia como herramienta de diagnóstico del funcionamiento interno de los modelos.

6.1. Eficiencia y precisión de las representaciones convolucionales

La red convolucional ConvNet superó consistentemente a la red densa FCNet en exactitud sobre el conjunto de pruebas de MNIST, utilizando menos de la mitad de parámetros. Esta ventaja no es exclusiva de este estudio, ni de las arquitecturas elegidas. El resultado reproduce, a pequeña escala, el mismo principio que LeCun et al. demostraron con LeNet-5 [16] y que Krizhevsky et al. profundizaron con AlexNet en ImageNet [6]. En todos estos casos la compartición de pesos en las capas convolucionales permite extraer características espaciales de forma eficiente, algo que una capa densa solo puede lograr aprendiendo pesos redundantes para cada posición de la imagen.

La prueba de McNemar confirmó que las diferencias de rendimiento son estadísticamente significativas, lo que descarta que la ventaja de la ConvNet sea consecuencia del proceso de entrenamiento o de la partición de datos concreta utilizada [14]. La superioridad de la red convolucional es, por tanto, una propiedad estructural de su arquitectura y no una casualidad experimental.

6.2. Robustez arquitectónica frente a perturbaciones gaussianas

Bajo ruido gaussiano con $\sigma \in [0.00, 0.30]$, la exactitud de la FCNet cae de forma pronunciada mientras que la ConvNet mantiene su rendimiento de manera mucho más estable. A $\sigma = 0.30$ la diferencia entre ambos modelos supera los 20 puntos porcentuales. Esta diferencia no es arbitraria, es la consecuencia de tres mecanismos arquitectónicos de la red convolucional que actúan de forma acumulada.

El primero es el conjunto de filtros locales del modelo. Cada filtro convolucional procesa solo una ventana de 3×3 píxeles, lo que permite que las perturbaciones gaussianas independientes de cada píxel se promedien parcialmente dentro de esa ventana, favoreciendo la extracción de la señal estructural del borde sobre el ruido. El segundo mecanismo es el MaxPooling, que al retener el máximo de una ventana 2×2 descarta las activaciones débiles causadas por el ruido y mantiene las activaciones fuertes asociadas a estructuras reales del dígito [16]. El tercero es la jerarquía de abstracción. El ruido se diluye capa a capa, de modo que la representación final es considerablemente más limpia que los píxeles originales. Goodfellow et al. mencionan que las representaciones jerárquicas son intrínsecamente más robustas a perturbaciones locales que las representaciones planas de una sola capa [1], lo que es exactamente lo que se observa al comparar las curvas de exactitud bajo ruido de ambos modelos.

6.3. Representaciones internas como indicador de robustez

En el trabajo se han empleado tres herramientas de visualización con propósitos distintos. Estas son los mapas de características, el análisis de activaciones por capa y los mapas de saliencia. La comparación de sus comportamientos bajo ruido es clarificadora, debido a que cada una opera en un espacio diferente del modelo, sus respuestas son diferentes ante la misma perturbación.

Los mapas de características permiten ver qué detectan los filtros convolucionales a lo largo de las capas de la red. En la primera capa aparecen detectores de bordes y texturas locales, mientras que en la segunda, combinaciones más abstractas de esos patrones. Esta estructura jerárquica, descrita por Zeiler y Fergus [4] y detallada por Chollet [2], se puede apreciar en los experimentos realizados sobre MNIST y permite entender qué información extrae la red en cada nivel de abstracción.

El análisis de las activaciones por capa ayuda a construir una perspectiva completa sobre las extracciones parciales de los mapas de características. Esa imagen nos permite comprobar si las representaciones que construye la red permanecen estables cuando la imagen de entrada se degrada. Los mapas de activación de Conv1 y Conv2 mantienen una similitud alta con los de la imagen limpia incluso a $\sigma = 0.20$, lo que confirma que los filtros convolucionales siguen detectando las mismas estructuras del dígito con o sin ruido moderado. Esa estabilidad de las representaciones intermedias es la explicación de por qué la clasificación también lo es.

Los mapas de saliencia operan en el espacio de los píxeles de entrada y miden la sensibilidad local de la predicción a cada píxel. Por eso, su comportamiento ante el ruido es distinto al de los mapas de activación. Esta diferencia entre herramientas no es un defecto de los mapas de saliencia, esta característica revela que la robustez de la CNN reside en el espacio de las representaciones y jerarquías intermedias que el modelo aprende, y no en los píxeles de entrada.

6.4. Límites de los mapas de saliencia como herramienta de diagnóstico

Los mapas de saliencia son una herramienta útil para entender qué regiones de la imagen activan al modelo en condiciones normales. Sin embargo, presentan una limitación estructural como instrumento de diagnóstico de la robustez ante perturbaciones en la imagen. La correlación de Spearman entre el mapa de saliencia de una imagen limpia y el de esa misma imagen con ruido cae de forma significativa incluso para niveles moderados de σ , mientras que la predicción del modelo se mantiene correcta.

Si la robustez de la CNN proviniese de que el modelo siempre usa los mismos píxeles discriminativos con independencia del ruido, los mapas de saliencia deberían permanecer estables al añadir perturbaciones, puesto que el modelo sigue clasificando correctamente. Sin embargo, ocurre todo lo contrario, los mapas de saliencia cambian considerablemente bajo ruido moderado mientras que la predicción no varía. Eso demuestra que la estabilidad de la clasificación no proviene de píxeles concretos, sino de que las representaciones intermedias construidas por las capas convolucionales sí son estables. Los mapas de activación de Conv1 y Conv2 lo confirman, pues mantienen una similitud alta con los de la imagen limpia incluso a $\sigma = 0.20$, cuando el mapa de saliencia ya ha cambiado considerablemente.

Los mapas de saliencia trabajan en el espacio de entrada, donde el ruido sí tiene efecto directo, y por eso su inestabilidad ante perturbaciones no implica que el modelo haya fallado, sino que el fenómeno que se quiere diagnosticar, la robustez, ocurre en una capa de abstracción a la que el gradiente sobre la entrada no tiene acceso directo. Para diagnosticar la robustez de una CNN, los mapas de activación de las capas convolucionales son una herramienta más adecuada que los gradientes sobre la imagen de entrada [23].

El rendimiento de una CNN no se comprende mirando solo sus métricas de clasificación. Hace falta examinar sus representaciones internas para entender por qué el modelo clasifica bien y a qué tipo de perturbaciones es sensible. La comparación entre herramientas de visualización, en este caso, los mapas de activación que permanecen estables, junto a los mapas de saliencia variables ante el mismo ruido, cuentan una historia que las métricas y números solos no pueden contar.

6.5. Limitaciones

Dataset. Todos los experimentos se realizaron sobre MNIST, un conjunto de datos relativamente sencillo en el que los dígitos aparecen centrados sobre un fondo oscuro uniforme. Los resultados cuantitativos, como la precisión bajo ruido o los valores de correlación de Spearman de los mapas de saliencia, podrían variar en datasets más complejos como CIFAR-10 o subconjuntos de ImageNet, donde las

imágenes tienen mayor variabilidad visual, fondos complejos y objetos con formas diversas. Por otro lado, también sería interesante observar cómo se comportan estos modelos ante datasets con un número limitado de muestras para el entrenamiento. Este aspecto es el más relevante de las limitaciones porque condiciona la generalidad de todos los resultados del trabajo.

Interpretabilidad. El análisis de mapas de saliencia se limitó al método de gradiente básico. Métodos más avanzados como Grad-CAM (del inglés *Gradient-weighted Class Activation Mapping*) [24], Integrated Gradients o SmoothGrad han demostrado mayor estabilidad ante perturbaciones y podrían proporcionar explicaciones más fieles al comportamiento interno de la red. Utilizar solo el método más simple impide determinar si la fragilidad observada es inherente a los mapas de saliencia en general o específica de la variante de gradiente básico empleada.

Modelos. Las arquitecturas utilizadas (FCNet y ConvNet) son intencionadamente sencillas para facilitar la comprensión. Arquitecturas más profundas, con técnicas de regularización avanzadas como Batch Normalization o conexiones residuales, podrían mostrar comportamientos distintos bajo ruido, tanto en términos de robustez como de estabilidad en las herramientas utilizadas.

Tipo de ruido. El estudio se centró exclusivamente en ruido gaussiano aditivo. Otros tipos de perturbación como el ruido salt-and-pepper, las transformaciones geométricas o los cambios de contraste podrían afectar de forma muy distinta a los modelos y alterar las conclusiones sobre la robustez relativa.

6.6. Líneas Futuras

Los resultados obtenidos abren varias líneas de trabajo que quedan fuera del alcance de este trabajo, pero que sería interesante explorar.

Evaluación sobre datasets más complejos. Repetir los experimentos de robustez al ruido sobre CIFAR-10 o ImageNet para verificar si los mecanismos identificados en MNIST se generalizan a imágenes naturales más complejas.

Comparación con métodos de interpretabilidad más robustos. Estudiar si métodos como Grad-CAM, Integrated Gradients o SmoothGrad muestran mayor estabilidad bajo ruido que el mapa de saliencia básico, y si se aproximan mejor al comportamiento que revelan los mapas de características.

Robustez a ataques adversariales. El ruido gaussiano es una perturbación no dirigida. Esto quiere decir que se añade aleatoriamente sin ningún conocimiento del modelo. Los ataques adversariales, como el método del signo del gradiente (FGSM, del inglés *Fast Gradient Sign Method*), generan perturbaciones específicamente diseñadas para engañar al clasificador utilizando información de los propios gradientes del modelo. Los mecanismos de robustez al ruido gaussiano identificados en este trabajo, los filtros locales, el MaxPooling y la jerarquía de abstracción, actúan sobre perturbaciones aleatorias independientemente de la

arquitectura del modelo; no está claro que proporcionen la misma protección ante perturbaciones construidas para explotar precisamente esos gradientes. Explorar si existe robustez entre ambos tipos de perturbación, y bajo qué condiciones tiene implicaciones relevantes para la seguridad de los sistemas de clasificación basados en redes convolucionales.

Arquitecturas más profundas. Estudiar cómo evoluciona la robustez y la fragilidad de los mapas de saliencia y las características extraídas en cada nivel del modelo a medida que se añaden más capas convolucionales, Batch Normalization o conexiones residuales, para identificar qué componentes arquitectónicos contribuyen más a la robustez.

A. Implementación de los Modelos

En este anexo se muestra el código PyTorch de los dos modelos utilizados en los experimentos del capítulo 5. Ambos se encuentran definidos en `src/models.py` dentro del repositorio del proyecto.

FCNet

FCNet es una red neuronal totalmente conectada (*Fully Connected Network*) cuya arquitectura se describe en la sección 5.1. La implementación sigue la API estándar de PyTorch: se hereda de `nn.Module` y se define el flujo de datos en el método `forward`.

```
1 class FCNet(nn.Module):
2
3     def __init__(self):
4         super().__init__()
5         self.flatten = nn.Flatten()
6         self.linear_relu_stack = nn.Sequential(
7             nn.Linear(28 * 28, 512),
8             nn.ReLU(),
9             nn.Linear(512, 512),
10            nn.ReLU(),
11            nn.Linear(512, 10),
12        )
13
14    def forward(self, x):
15        x = self.flatten(x)
16        logits = self.linear_relu_stack(x)
17        return logits
```

Extracto de código A.1: Implementación de FCNet en PyTorch.

ConvNet

ConvNet es la red neuronal convolucional cuya arquitectura se describe en la sección 5.2. Además de las capas convolucionales y densas, incluye los atributos `_acts` y `_grads` para almacenar activaciones y gradientes de la última capa convolucional, lo que permite aplicar técnicas de visualización como los mapas de saliencia.

```
1 class ConvNet(nn.Module):
2
3     def __init__(self):
4         super().__init__()
5         self.conv1 = nn.Conv2d(1, 32, 3)
6         self.conv2 = nn.Conv2d(32, 64, 3)
7         self.pool = nn.MaxPool2d(2, 2)
8         self.fc1 = nn.Linear(64 * 5 * 5, 120)
9         self.fc2 = nn.Linear(120, 84)
10        self.fc3 = nn.Linear(84, 10)
11        self._acts = None
12        self._grads = None
13
14    def forward(self, x):
15        x = self.pool(F.relu(self.conv1(x)))
16        x = self.pool(F.relu(self.conv2(x)))
17        x = torch.flatten(x, 1)
18        x = F.relu(self.fc1(x))
19        x = F.relu(self.fc2(x))
20        x = self.fc3(x)
21        return x
22
23    def get_activations(self):
24        return self._acts
25
26    def get_activations_gradient(self):
27        return self._grads
```

Extracto de código A.2: Implementación de ConvNet en PyTorch.

B. Desarrollo Matemático de la Retropropagación

Antes de explicar el algoritmo de retropropagación vamos a redefinir la sintaxis, inspirándonos en la planteada por Nielsen [2015](#), con el objetivo de facilitar la comprensión de esta sección.

- w_{jk}^l es el peso asociado a la conexión entre la neurona j -ésima de la capa l y la k -ésima de la capa $l - 1$.
- b_j^l es el sesgo asociado a la neurona j -ésima de la capa l .
- z_j^l es la combinación de las entradas a la neurona j -ésima de la capa l .
- a_j^l es la activación de la neurona j -ésima de la capa l .

Aplicando esta nomenclatura, podemos reescribir expresiones anteriores como la combinación de las entradas de una neurona ([B.1](#)) o su activación ([B.2](#)).

$$z_j^l = \sum_k w_{jk}^l a_j^{l-1} + b_j^l \quad (\text{B.1})$$

$$a_j^l = \sigma(z_j^l) \quad (\text{B.2})$$

Además, siguiendo convenciones de la literatura científica, podemos simplificar estas expresiones anteriores convirtiéndolas a la forma matricial, ya no solo porque sea una forma más fácil de escribir, sino porque computacionalmente los ordenadores están mejor preparados para realizar estas operaciones matriciales.

- w^l como la matriz de pesos w_{jk}^l de la capa l .
- b^l como el vector de sesgos b_j^l de la capa l .
- z^l como el vector de combinaciones z_j^l de la capa l .
- a^l como el vector de activaciones o las salidas a_j^l de la capa l .

De esta forma, denotamos a la combinación de las entradas de una neurona [B.3](#) y a su activación [B.4](#) como

$$z^l = w^l a^{l-1} + b^l \quad (\text{B.3})$$

$$a^l = \sigma(z^l) \quad (\text{B.4})$$

donde σ se refiere a un operador elemento a elemento del vector z^l , es decir, $\sigma(z^l)_j = \sigma(z_j^l)$.

Retomando el algoritmo de retropropagación, el objetivo es entender cómo los cambios introducidos en los pesos y sesgos de la red modificarán la salida de la misma. Dicho de otra forma, tendremos que calcular a partir de la salida global de la red, las derivadas parciales de todos y cada uno de los parámetros involucrados en el cálculo. Para ello, la función de coste elegida para la red debe cumplir dos premisas:

En primer lugar, el coste total de la red debe poder expresarse como la media de los costes parciales. Es decir, $C = \frac{1}{n} \sum_i c_i$ para cada salida i de la red. Por ejemplo, en el caso del error cuadrático medio (3.8) sería $c_x = \frac{1}{2}(y - a^L)^2$, donde L denota la última capa de la red.

En segundo lugar, su expresión debe estar escrita en función de los parámetros de la red, o lo que es lo mismo, de las salidas de la red. Para el caso anterior del error cuadrático $C = \frac{1}{2} \sum_j (y_j - a_j^L)^2$. De esta forma, aprovechando la regla de la cadena, podremos propagar hacia atrás el error obtenido en la salida, calculando los costes parciales asociados a cada uno de los parámetros [10].

Ecuaciones detrás de la retropropagación

Existen cuatro ecuaciones fundamentales para describir el proceso de retropropagación en una red [10]. Todas ellas se basan en el error que hemos mencionado anteriormente. Definimos el error δ_j^l de la neurona j en la capa l como:

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} \quad (\text{B.5})$$

Siguiendo los principios de notación anteriores, podemos expresar el vector de errores de la capa l como δ^l . La retropropagación nos proporcionará un método de calcular δ^l para cada capa de la red para así poder obtener los valores que realmente nos interesan, $\frac{\partial C}{\partial w_{jk}^l}$ y $\frac{\partial C}{\partial b_j^l}$.

1. Error en la última capa de la red δ^L :

Aplicando la regla de la cadena podemos expresar el error (B.5) respecto a la activación de la capa L ,

$$\delta_j^L = \sum_k \frac{\partial C}{\partial a_k^L} \frac{\partial a_k^L}{\partial z_j^L}, \quad (\text{B.6})$$

donde $\frac{\partial a_k^L}{\partial z_j^L}$ se desvanece para $k \neq j$. Luego, teniendo en cuenta que $a_j^L = \sigma(z_j^L)$, podemos reescribir el segundo término como

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L), \quad (\text{B.7})$$

la forma de esta expresión nos permite trabajar elemento a elemento. Su forma matricial sería

$$\delta^L = \nabla_a C \odot \sigma'(z^L). \quad (\text{B.8})$$

2. Error δ^l en función del error de la siguiente capa δ^{l+1} :

Para la expresión del error definida anteriormente (B.5), es muy simple definir el error en la siguiente capa como $\delta_k^{l+1} = \frac{\partial C}{\partial z_k^{l+1}}$. Ahora, aplicando la regla de la cadena

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} = \sum_k \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l} = \sum_k \frac{\partial z_k^{l+1}}{\partial z_j^l} \delta_k^{l+1}, \quad (\text{B.9})$$

hemos sustituido siguiendo la definición de δ_k^{l+1} . Luego, teniendo en cuenta que

$$z_k^{l+1} = \sum_j w_{kj}^{l+1} a_j^l + b_k^{l+1} = \sum_j w_{kj}^{l+1} \sigma(z_j^l) + b_k^{l+1}, \quad (\text{B.10})$$

podemos derivar, obteniendo

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = w_{kj}^{l+1} \sigma'(z_j^l). \quad (\text{B.11})$$

Por tanto, la expresión final que nos queda es

$$\delta_j^l = \sum_k w_{kj}^{l+1} \delta_k^{l+1} \sigma'(z_j^l), \quad (\text{B.12})$$

que reescrito a la forma matricial, quedaría como

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l). \quad (\text{B.13})$$

3. Ratio de cambio del error respecto a cualquier sesgo de la red:

Representamos el ratio de cambio del error respecto del sesgo como $\frac{\partial C}{\partial b_j^l}$. Aplicando la regla de la cadena podemos llegar a

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial b_j^l}, \quad (\text{B.14})$$

donde, aplicando una sustitución por la definición del error (B.5) nos queda

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \frac{\partial z_j^l}{\partial b_j^l} = \delta_j^l \frac{\partial [\sum_k w_{kj}^l a_k^{l-1} + b_j^l]}{\partial b_j^l} = \delta_j^l, \quad (\text{B.15})$$

reescribiendo matricialmente,

$$\frac{\partial C}{\partial b^l} = \delta^l, \quad (\text{B.16})$$

es decir, la variación del sesgo en cualquier capa es equivalente al error percibido en esa capa.

4. Ratio de cambio del error respecto a cualquier peso de la red:

De forma similar, podemos obtener la variación del error respecto de cualquier peso de la red

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial w_{jk}^l} = \delta_j^l \frac{\partial [\sum_k w_{kj}^l a_k^{l-1} + b_j^l]}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l, \quad (\text{B.17})$$

reescrito matricialmente como

$$\frac{\partial C}{\partial w^l} = a^{l-1} \delta^l. \quad (\text{B.18})$$

El algoritmo de retropropagación

Gracias a las ecuaciones tenemos una forma de calcular el gradiente de la función de coste, ahora vamos a ver los pasos que lleva a cabo la red en el entrenamiento [10]:

1. **Entrada x:** calculamos las activaciones a^1 correspondientes a la primera capa de la red.
2. **Propagación o feedforward:** para cada $l = 2, 3, \dots, L$ calculamos z^l y a^l .
3. **Error de salida:** al terminar una iteración completa sobre la red, calculamos el error en la última capa δ^L .
4. **Retropropagamos el error:** para cada $l = L-1, L-2, \dots, 2$ calculamos el error δ^l en función del error en la capa siguiente δ^{l+1} .
5. **Salida:** calculamos el gradiente de la función de coste, lo que conlleva averiguar la variación del error respecto de los parámetros de la red.

Con este algoritmo podemos calcular el gradiente de la función de coste respecto de los parámetros de la red para una entrada determinada. Sin embargo, en un entorno práctico lo que nos interesa es poder entrenar el modelo actualizando múltiples veces los parámetros hasta alcanzar cierta condición de

parada. Por ello, se emplea lo que se conoce como el *descenso estocástico del gradiente* [1]. Para N ejemplos de entrada $[(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)]$ realizaríamos el anterior algoritmo N veces, de forma que al calcular el gradiente de la función de coste podamos aplicar la regla delta y actualizar los parámetros de la siguiente forma:

$$w^l \leftarrow w^l - \eta \delta^l (a^{l-1})^T \quad (\text{B.19})$$

$$b^l \leftarrow b^l - \eta \delta^l. \quad (\text{B.20})$$

Lista de Acrónimos

ANN Artificial Neural Network (Red Neuronal Artificial).

ATM Automated Teller Machine (Cajero Automático).

CNN Convolutional Neural Network (Red Neuronal Convolucional).

CPU Central Processing Unit (Unidad de Procesamiento Central).

DL Deep Learning (Aprendizaje Profundo).

FGSM Fast Gradient Sign Method.

GPU Graphics Processing Unit (Unidad de Procesamiento Gráfico).

Grad-CAM Gradient-weighted Class Activation Mapping.

IA Inteligencia Artificial.

ILSVRC ImageNet Large Scale Visual Recognition Challenge.

LIME Local Interpretable Model-agnostic Explanations.

ML Machine Learning (Aprendizaje Automático).

MLP Multi-Layer Perceptron (Perceptrón Multicapa).

MNIST Modified National Institute of Standards and Technology.

NIN Network in Network.

ReLU Rectified Linear Unit (Unidad Lineal Rectificada).

RN Red Neuronal.

RNA Red Neuronal Artificial.

SGD Stochastic Gradient Descent (Descenso de Gradiente Estocástico).

SHAP SHapley Additive exPlanations.

TFG Trabajo de Fin de Grado.

Bibliografía

- [1] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT press Cambridge, 2016.
- [2] Francois Chollet and François Chollet. *Deep learning with Python*. Simon and Schuster, 2021.
- [3] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps, 2014. URL <https://arxiv.org/abs/1312.6034>.
- [4] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European Conference on Computer Vision (ECCV)*, pages 818–833, 2014. doi: 10.1007/978-3-319-10590-1_53.
- [5] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023. <https://D2L.ai>.
- [6] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25 (NeurIPS 2012)*, pages 1097–1105, 2012. URL <https://api.semanticscholar.org/CorpusID:195908774>.
- [7] Robert Hecht-Nielsen. Neurocomputing: picking the human brain. *IEEE spectrum*, 25(3):36–41, 1988.
- [8] José Ramón Hilera González and Víctor José Martínez Hernando. *Redes neuronales artificiales: fundamentos, modelos y aplicaciones*. RA-MA Editorial, 1995.
- [9] Amey Thakur and Archit Konde. Fundamentals of neural networks. *International Journal for Research in Applied Science and Engineering Technology*, 9:407–426, 08 2021. doi: 10.22214/ijraset.2021.37362.
- [10] Michael A Nielsen. *Neural networks and deep learning*, volume 25. Determination press San Francisco, CA, USA, 2015.
- [11] Ultralytics Inc. Mnist - ultralytics yolo docs, 2024. URL <https://docs.ultralytics.com/es/datasets/classify/mnist/>.
- [12] Eduardo Rivera. Introducción a las redes neuronales artificiales. *Universidad Don Bosco*, 2005.
- [13] Fernando Bombal Gordon, Gabriel Vera Boti, and Luis Rodríguez Marín. *Problemas de análisis matemático*. Alfa Centauro, S.A., 1986.
- [14] Lozano Bagén, T. y Bosch Rué, A. Casas Roma, J. *Deep learning: Principios y fundamentos*. Editorial UOC, 2020.

- [15] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.
- [16] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [17] ImageNet, 2026. URL <https://image-net.org/challenges/LSVRC/>.
- [18] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *International Conference on Learning Representations (ICLR)*, 2014. URL <https://arxiv.org/abs/1312.4400>.
- [19] Andry Chowanda and Rhio Sutoyo. Convolutional neural network for face recognition in mobile phones. *ICIC Express Letters*, 13(7):569–574, 2019.
- [20] SD Kulik and DA Nikonets. Examples of application of the neural networks in techniques for the forensic handwriting expert. *Neurocomputers: design and application*, 9:61–85, 2009.
- [21] Hajira Saleem, Reza Malekian, and Hussan Munir. Neural network-based recent research developments in slam for autonomous ground vehicles: A review. *IEEE Sensors Journal*, 23(13):13829–13858, 2023.
- [22] PyTorch Contributors. PyTorch documentation, 2026. URL https://docs.pytorch.org/docs/stable/index.html?ajs_aid=88edefef-263d-44cf-8f7e-cb9b525c6afd.
- [23] Christoph Molnar. *Interpretable Machine Learning*. 3 edition, 2025. ISBN 978-3-911578-03-5. URL <https://christophm.github.io/interpretable-ml-book>.
- [24] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017. doi: 10.1109/ICCV.2017.74.